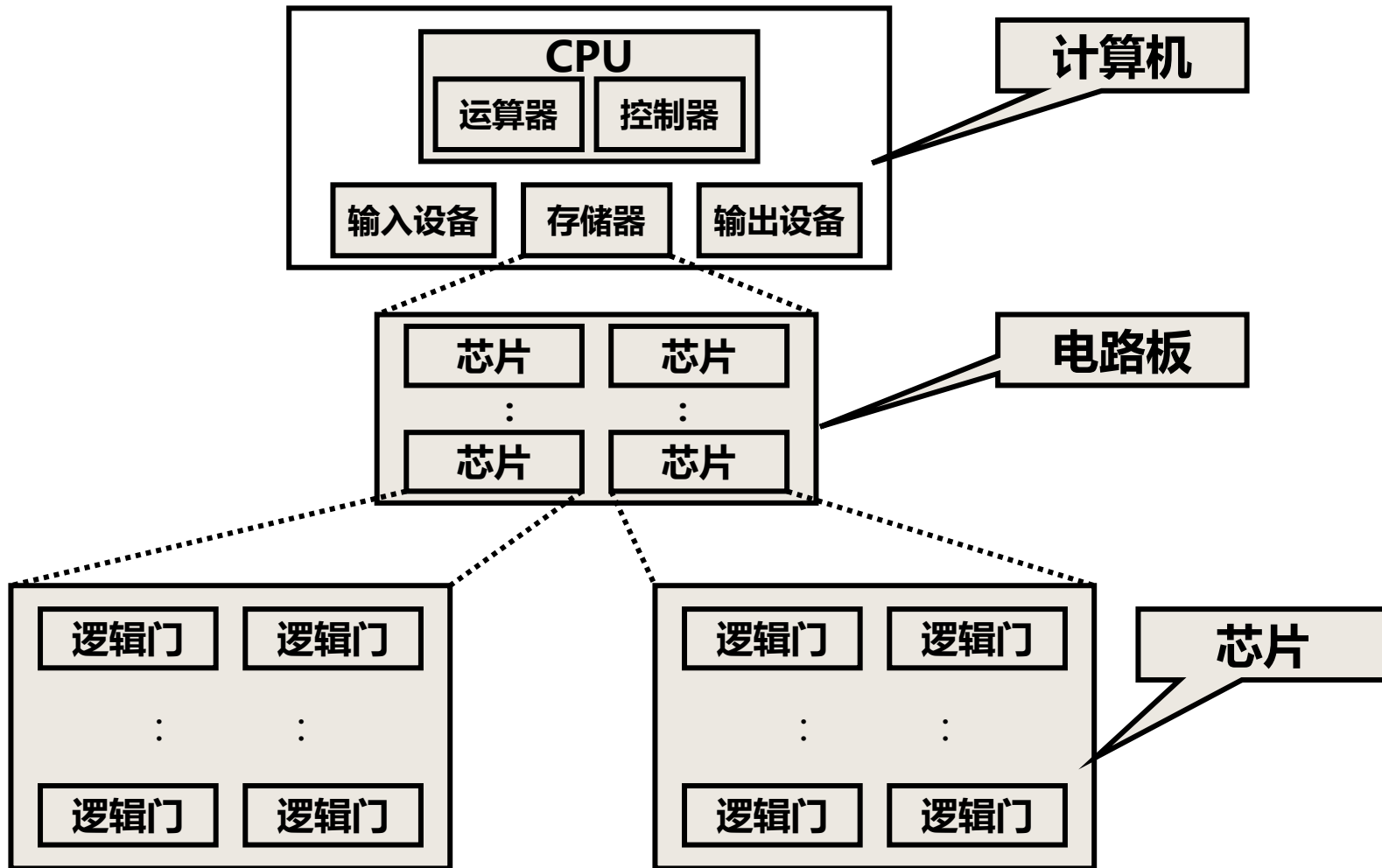


2. 数字电路基础

计算机中的逻辑电路



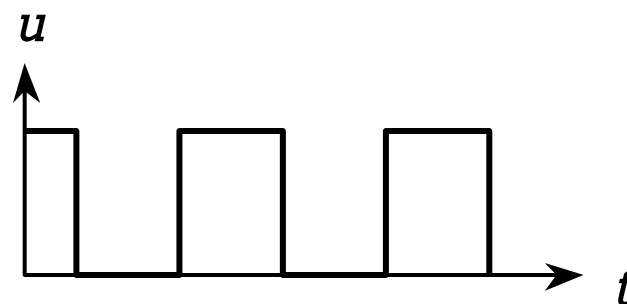
概览

- 2.1 数字电路和数字逻辑概述
 - 2.1.1 逻辑代数基础
 - 2.1.2 门电路
- 2.2 计算机中的数字电路
 - 2.2.1 组合逻辑：加法器，ALU，译码器，数据选择器，ROM
 - 2.2.2 时序逻辑：触发器，计数器，寄存器和RAM

2.1 数字电路与数字逻辑概述

■ 数字电路的特点

- ① 工作信号是二进制的数字信号，在时间和数值上是离散的（不连续），反映在电路上就是低电平和高电平两种状态（即0和1两个逻辑值）。
- ② 和模拟电路相比，组成数字电路的元器件的精度要求不高，只要在工作时能够可靠地区分0和1两种状态即可。
- ③ 数字量更便于加工、处理、传输、存储等，可靠，抗干扰能力强。
- ④ 在数字电路中，研究的主要问题是电路的逻辑功能，即输入信号的状态和输出信号的状态之间的关系。



数字信号波形

2.1.1 逻辑代数基础

- 事物往往存在两种对立的状态，在逻辑代数中可以抽象地表示为 0 和 1，称为**逻辑0状态**和**逻辑1状态**。
- **逻辑常量**：0和1，不表示数量的大小，而是表示两种对立的逻辑状态。
- **逻辑变量**：逻辑代数中的变量，取值只逻辑0和逻辑1两种
- **逻辑代数**：按一定的逻辑关系进行运算的代数，是分析和设计数字电路的数学工具。在逻辑代数中，只有**0**和**1**两种逻辑值，**与、或、非**三种基本逻辑运算，还有**与或、与非、与或非、异或**等几种导出逻辑运算。

逻辑运算-与逻辑

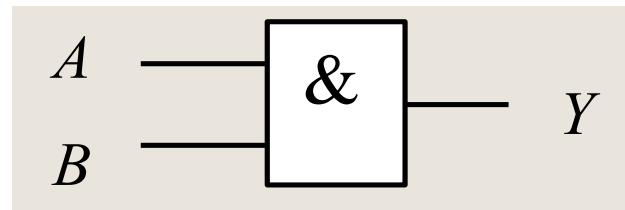
□ **与逻辑的定义**：仅当决定事件（Y）发生的所有条件（A，B，C，...）均满足时，事件（Y）才能发生。表达式为：

$$Y = A B C \dots$$

把所有可能的条件组合，及其对应结果——列出来的表格叫做**真值表**。

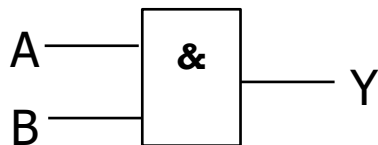
<i>A</i>	<i>B</i>	<i>Y</i>
0	0	0
0	1	0
1	0	0
1	1	1

逻辑符号



$$Y = A B$$

国标



国际常用



逻辑运算-或逻辑

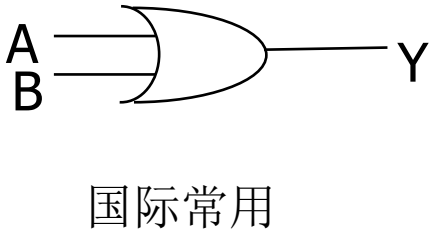
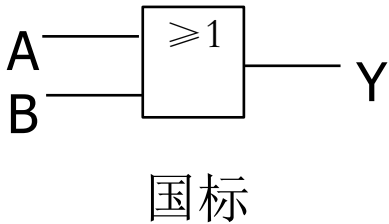
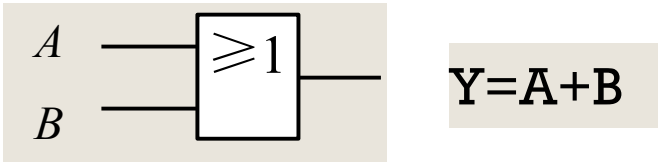
或逻辑的定义：当决定事件（Y）发生的各种条件（A，B，C，...）中，只要有一个或多个条件具备，事件（Y）就发生。表达式为：

$$Y = A + B + C + \dots$$

真值表

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

逻辑符号



逻辑运算-非逻辑

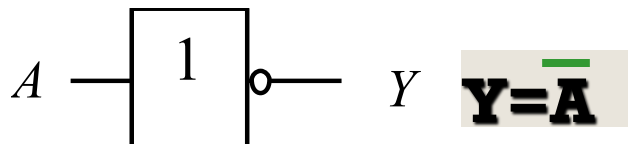
非逻辑：指的是逻辑的否定。当决定事件（Y）发生的条件（A）满足时，事件不发生；条件不满足，事件反而发生。表达式为：

$$Y = \overline{A}$$

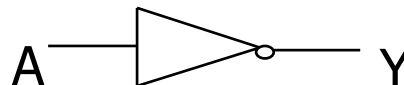
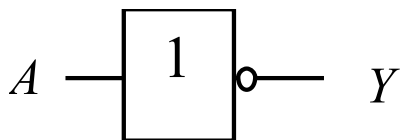
真值表

A	Y
0	1
1	0

逻辑符号



国标



国际常用

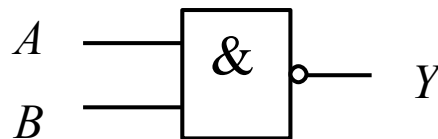
其他常用的逻辑运算

(1) **与非运算**：逻辑表达式为：

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

真值表

$$Y = \overline{AB}$$



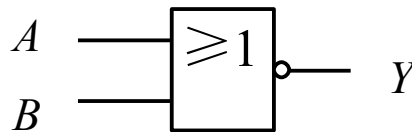
与非门的逻辑符号

(2) **或非运算**：逻辑表达式为：

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

真值表

$$Y = \overline{A + B}$$



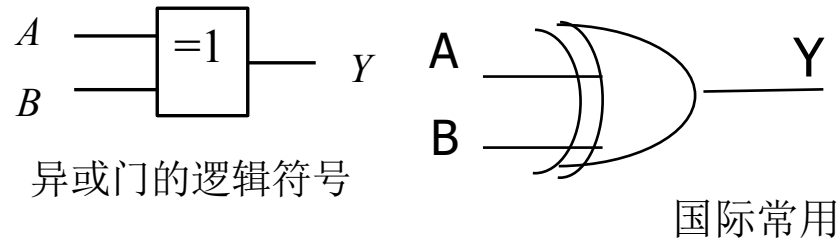
或非门的逻辑符号

(3) **异或运算**：逻辑表达式为：

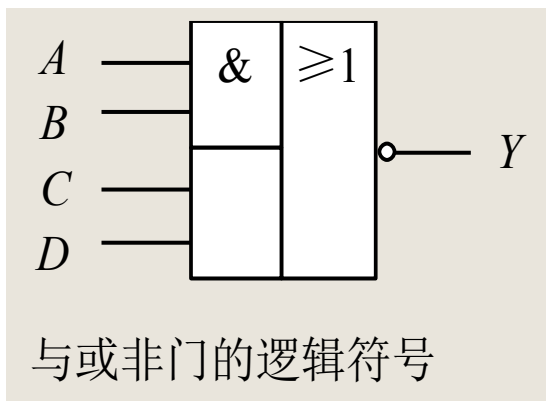
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

真值表

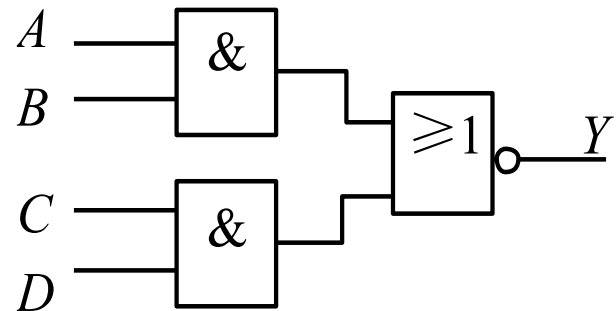
$$Y = \bar{A}B + A\bar{B} = A \oplus B$$



(4) **与或非运算**：逻辑表达式为：



$$Y = \overline{AB + CD}$$



与或非门的等效电路

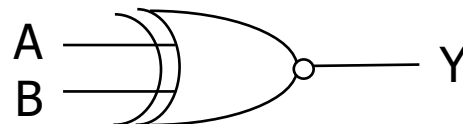
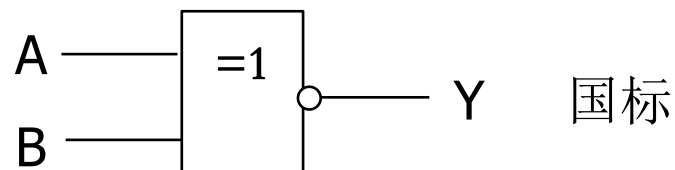
(5) 同或运算

■ 逻辑表达式: $Y = AB + \overline{A}\overline{B} = A \odot B$

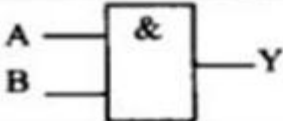
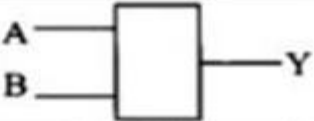
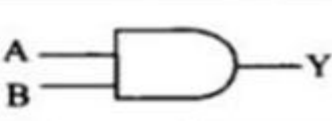
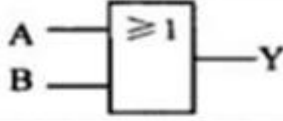
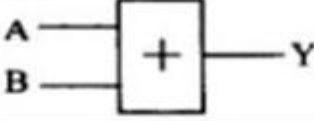
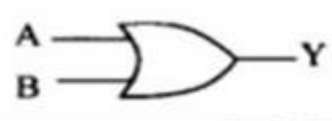
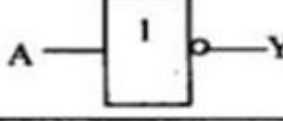
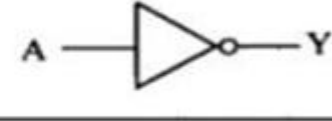
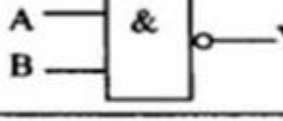
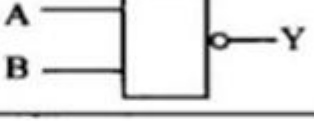
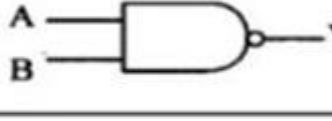
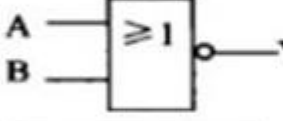
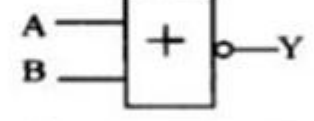
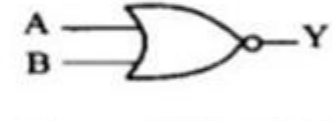
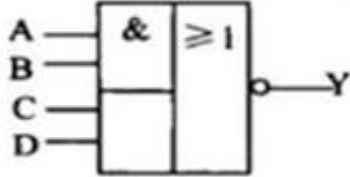
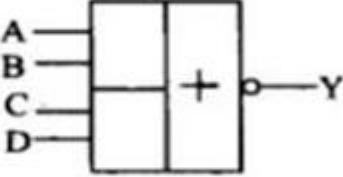
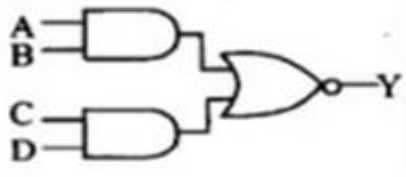
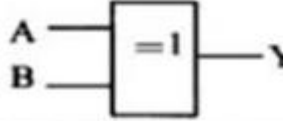
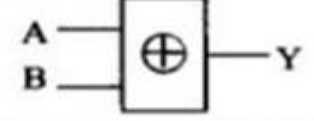
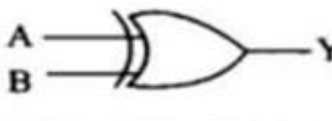
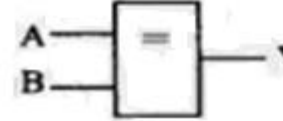
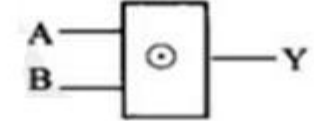
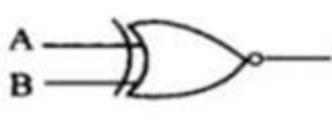
同或逻辑真值表

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

同或逻辑符号



国际常用

名称	国标符号	曾用符号	国外流行符号
与			
或			
非			
与非			
或非			
与或非			
异或			
同或			

逻辑函数

- **逻辑表达式**：由逻辑变量和与、或、非3种运算符连接起来所构成的式子，称为逻辑表达式。
- 在逻辑表达式中，字母上面没有非运算符的叫做**原变量**，有非运算符的叫做**反变量**。
- **逻辑函数**：如果对应于输入逻辑变量A、B、C、...的每一组确定值，输出逻辑变量Y就有唯一确定的值，则称Y是A、B、C、...的逻辑函数。记为

$$Y = f(A, B, C, \dots)$$

逻辑函数的相等

设有两个逻辑函数 $Y_1 = f(A, B, C, \dots)$ $Y_2 = g(A, B, C, \dots)$

如果对应于输入变量A、B、C、...的任何一组变量取值，输出变量 Y_1 和 Y_2 的值都相同，则称 Y_1 和 Y_2 是相等的，记为 $Y_1 = Y_2$ 。

若两个逻辑函数相等，则它们的真值表一定相同；反之，若两个函数的真值表完全相同，则这两个函数一定相等。

因此，要证明两个逻辑函数是否相等，只要分别列出它们的真值表，看看它们的真值表是否相同即可。

证明等式：

$$\overline{AB} = \overline{A} + \overline{B}$$

A	B	AB	$\overline{AB}$	$\overline{A}$	$\overline{B}$	$\overline{A+B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

逻辑代数的公式与定理

- 常量之间的关系定理
- 常量和变量的关系定理
- 与普通代数相似的定理
- 逻辑代数的一些特殊定理
- 关于等式的三个规则

常量之间的关系

$$1) \quad 0 \cdot 0 = 0$$

$$2) \quad 0 \cdot 1 = 0$$

$$3) \quad 1 \cdot 1 = 1$$

$$4) \quad \bar{0} = 1$$

$$1') \quad 1 + 1 = 1$$

$$2') \quad 1 + 0 = 1$$

$$3') \quad 0 + 0 = 0$$

$$4') \quad \bar{1} = 0$$

变量与常量之间的关系

$$5) \quad A \cdot 1 = A$$

$$6) \quad A \cdot 0 = 0$$

$$7) \quad A \cdot \bar{A} = 0$$

$$5') \quad A + 0 = A$$

$$6') \quad A + 1 = 1$$

$$7') \quad A + \bar{A} = 1$$

与普通代数类似的定理

交换律

$$8) \quad A \cdot B = B \cdot A$$

$$8') \quad A + B = B + A$$

结合律

$$9) \quad (A \cdot B) \cdot C = A \cdot (B \cdot C)$$

$$9') \quad (A + B) + C = A + (B + C)$$

分配律

$$10) \quad A \cdot (B + C) = A \cdot B + A \cdot C$$

$$10') \quad A + B \cdot C = (A + B) \cdot (A + C)$$

逻辑代数特殊的定理

同一律

$$11) \quad A \cdot A = A$$

$$11') \quad A + A = A$$

德摩根定理

$$12) \quad \overline{A \cdot B} = \bar{A} + \bar{B}$$

$$12') \quad \overline{A + B} = \bar{A} \cdot \bar{B}$$

还原律

$$13) \quad \overline{\bar{A}} = A$$

其他公式

$$14) \quad \bar{A}B + AB = B$$

$$15) \quad A + AB = A$$

$$16) \quad A + \bar{A}B = A + B$$

$$17) \quad AB + \bar{A}C + BC = AB + \bar{A}C$$

$$18) \quad \overline{A\bar{B} + \bar{A}B} = AB + \bar{A}\bar{B}$$

关于等式的三个规则

- **代入规则**：任何一个含有变量A的等式，如果将所有出现A的位置都用同一个逻辑函数代替，则等式仍然成立。这个规则称为代入规则。

例如，已知等式 $\overline{AB} = \overline{A} + \overline{B}$ ，用函数Y=AC代替等式中的A，根据代入规则，等式仍然成立，即有：

$$\overline{(AC)B} = \overline{AC} + \overline{B} = \overline{A} + \overline{B} + \overline{C}$$

关于等式的三个规则

- **反演规则**：对于任何一个逻辑表达式Y，如果将表达式中的所有“ $\cdot$ ”换成“ $+$ ”，“ $+$ ”换成“ $\cdot$ ”，“ 0 ”换成“ 1 ”，“ 1 ”换成“ 0 ”，**原变量换成反变量，反变量换成原变量**，那么所得到的表达式就是函数Y的反函数 $\bar{Y}$ （或称补函数）。这个规则称为反演规则。例如：

$$Y = A\bar{B} + C\bar{D}E$$



$$\bar{Y} = (\bar{A} + B)(\bar{C} + D + \bar{E})$$

$$Y = A + B + \overline{\bar{C} + D + \bar{E}}$$



$$\bar{Y} = \bar{A} \cdot \bar{B} \cdot C \cdot \overline{\overline{D} \cdot \bar{E}}$$

注意：反演规则遵循的两个规则

1. 遵守“先括号、然后与、最后或”的运算优先次序；
2. 不属于单个变量上的反号应保留不变。

关于等式的三个规则

- **对偶规则**：对于任何一个逻辑表达式Y，如果将表达式中的所有“.”换成“+”，“+”换成“.”，“0”换成“1”，“1”换成“0”，而**变量保持不变**，则可得到的一个新的函数表达式Y'，Y'称为函数Y的对偶函数。例如：

$$Y = A\bar{B} + C\bar{D}E \longrightarrow Y' = (A + \bar{B})(C + \bar{D} + E)$$

$$Y = A + B + \overline{\overline{C} + \overline{D} + \overline{E}} \longrightarrow Y' = A \cdot B \cdot \overline{\overline{C} \cdot \overline{D} \cdot \overline{E}}$$

对偶规则的意义：**如果两个函数相等，则它们的对偶函数也相等**。例如：

$$A \cdot B + A \cdot \bar{B} = A \longrightarrow (A + B) \cdot (A + \bar{B}) = A$$

$$A(B + C) = AB + AC \longrightarrow A + BC = (A + B)(A + C)$$

逻辑函数的化简

- 逻辑函数的标准与或式
- 公式法化简
- 图形法化简
- 含约束项的逻辑函数化简

逻辑函数的标准与或式

逻辑函数的最小项：如果一个函数的某个乘积项，包含了函数的全部变量，其中每个变量都以原变量或反变量的形式出现，且作为因子仅出现一次，这样的乘积项就叫做最小项。 n 变量函数有 2^n 个最小项。

例如：3个变量A、B、C可组成8个最小项：

$$\overline{A}\overline{B}\overline{C}、\overline{A}\overline{B}C、\overline{A}B\overline{C}、\overline{A}BC、A\overline{B}\overline{C}、A\overline{B}C、AB\overline{C}、ABC$$

最小项的表示方法：通常用符号 m_i 来表示最小项。下标 i 的确定：确定变量顺序，反变量当做0，原变量当做1，则与这个二进制数相对应的十进制数，就是这个最小项的下标 i 。例如：3个变量A、B、C的8个最小项可以分别表示为

$$\begin{aligned} m_0 &= \overline{A}\overline{B}\overline{C}、m_1 = \overline{A}\overline{B}C、m_2 = \overline{A}B\overline{C}、m_3 = \overline{A}BC \\ m_4 &= A\overline{B}\overline{C}、m_5 = A\overline{B}C、m_6 = AB\overline{C}、m_7 = ABC \end{aligned}$$

逻辑函数的标准与或式

任何一个逻辑函数都可以表示成**唯一**的一组最小项之和，称为**标准与或表达式**，也称为**最小项表达式**。

对于不是最小项表达式的与或表达式，可利用公式 $A + \overline{A} = 1$ 和 $A(B + C) = AB + AC$ ，来配项展开成最小项表达式。

$$\begin{aligned} Y &= \overline{A} + BC \\ &= \overline{A}(B + \overline{B})(C + \overline{C}) + (A + \overline{A})BC \\ &= \overline{A}BC + \overline{A}B\overline{C} + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + ABC + \overline{A}BC \\ &= \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + ABC \\ &= m_0 + m_1 + m_2 + m_3 + m_7 \\ &= \sum m(0,1,2,3,7) \end{aligned}$$

如果列出了函数的真值表，则只要将函数值为1的那些最小项相加，便是函数的最小项表达式。

A	B	C	Y	最小项	
0	0	0	0	m_0	
0	0	1	1	m_1	$m_1 = \bar{A}\bar{B}C$
0	1	0	1	m_2	$m_2 = \bar{A}B\bar{C}$
0	1	1	1	m_3	$m_3 = \bar{A}BC$
1	0	0	0	m_4	
1	0	1	1	m_5	$m_5 = A\bar{B}C$
1	1	0	0	m_6	
1	1	1	0	m_7	

$$\begin{aligned}
 Y &= m_1 + m_2 + m_3 + m_5 = \sum m(1,2,3,5) \\
 &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C
 \end{aligned}$$

逻辑函数的最简与或式

最简与或式：乘积项最少、并且每个乘积项中的变量也最少的与或表达式。

$$\begin{aligned} Y &= \overline{A}B\overline{E} + \overline{A}B + A\overline{C} + A\overline{C}E + B\overline{C} + B\overline{C}D \\ &= \overline{A}B + A\overline{C} + B\overline{C} \\ &= \overline{A}B + A\overline{C} \longrightarrow \text{最简与或表达式} \end{aligned}$$

逻辑函数的公式法化简

逻辑函数的公式化简法就是运用逻辑代数的基本公式、定理和规则来化简逻辑函数，这里主要针对与或式的化简。

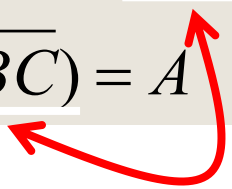
① 并项法

利用公式 $A + \bar{A} = 1$ ，将两项合并为一项，并消去一个变量。

运用分配律


$$Y_1 = \underline{ABC} + \underline{\bar{A}BC} + \underline{B\bar{C}} = \underline{(A + \bar{A})BC} + \underline{B\bar{C}}$$
$$= \underline{BC} + \underline{B\bar{C}} = \underline{B(C + \bar{C})} = B$$

运用分配律

$$Y_2 = \underline{ABC} + \underline{A\bar{B}} + \underline{A\bar{C}} = \underline{ABC} + \underline{A(\bar{B} + \bar{C})}$$
$$= \underline{ABC} + \underline{A\bar{B}\bar{C}} = \underline{A(BC + \bar{B}\bar{C})} = A$$


运用摩根定律

② 吸收法

利用公式 $A + A B = A$ ，消去多余的项。

$$Y_1 = \underline{\bar{A}B} + \bar{A}BCD(E + F) = \bar{A}B$$

运用摩根定律


$$\begin{aligned} Y_2 &= A + \overline{\overline{B}} + \overline{\overline{CD}} + \overline{\overline{ADB}} = A + BCD + \underline{AD} + B \\ &= (\underline{A + AD}) + (\underline{B + BCD}) = A + B \end{aligned}$$

如果一个乘积项是另外一个乘积项的因子，则另外一个乘积项是多余的。

③ 消去法

利用公式16 $A + \overline{A}B = A + B$ ，消去乘积项中多余的因子。

$$\begin{aligned} Y &= \overline{A}B + AC + BD \\ &= \overline{A} + AC + \overline{B} + BD \\ &= \overline{A} + \overline{B} + C + D \end{aligned}$$

④ 配项消去法

(1) 利用公式17(推论) $A B + \overline{A} C + BC = A B + \overline{A} C$, 将 $B C$ 项消去。

$$\begin{aligned} Y_1 &= A\overline{B} + AC + ADE + \overline{C}D \\ &= A\overline{B} + (AC + \overline{C}D + ADE) \\ &= A\overline{B} + AC + \overline{C}D \end{aligned}$$

$$\begin{aligned} Y_2 &= AB + \overline{B}C + AC(\overline{DE} + FG) \\ &= AB + \overline{B}C \end{aligned}$$

(2) 利用公式11' $A + A = A$, 为某项配上其所能合并项。

$$\begin{aligned} Y &= ABC + AB\overline{C} + A\overline{B}C + \overline{A}BC \\ &= (ABC + AB\overline{C}) + (\underline{ABC} + A\overline{B}C) + (\underline{ABC} + \overline{A}BC) \\ &= AB + AC + BC \end{aligned}$$

逻辑函数的图形法化简

逻辑函数的图形化简法：将逻辑函数用卡诺图来表示，利用卡诺图来化简。

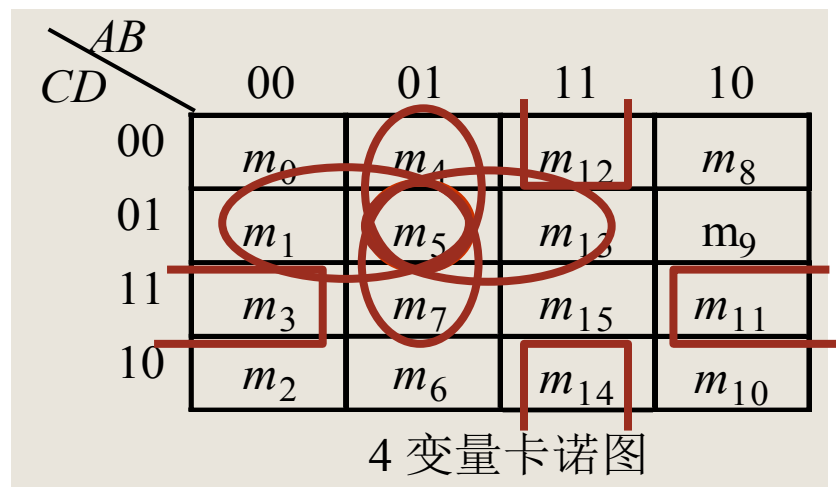
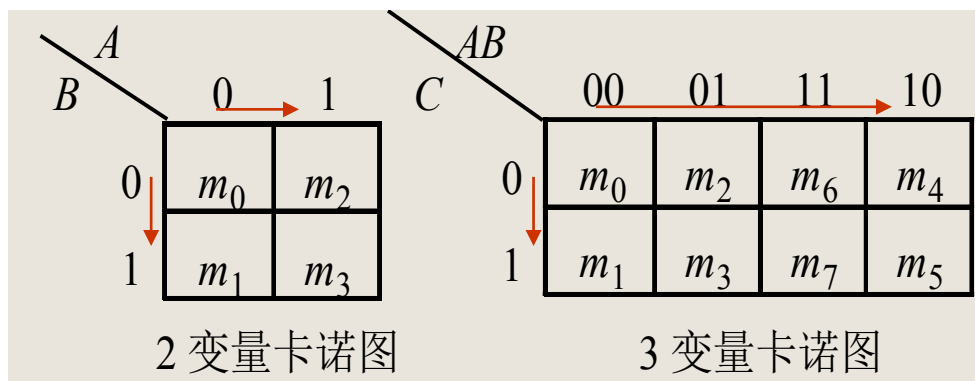
- 表示最小项的**卡诺图**：将 n 变量的全部最小项各用一个小方块表示，并使具有**逻辑相邻性的最小项在几何位置上也相邻**地排列起来，所得到的图形叫 n 变量最小项的卡诺图。（**相邻项**是指两个最小项，只有一个因子互为反变量，其余因子均相同，又称为逻辑相邻项）。

$\begin{array}{c} AB \\ \swarrow \searrow \\ C \end{array}$		00	01	11	10
		0	1	1	0
0	m_0	m_2	m_6	m_4	
1	m_1	m_3	m_7	m_5	

3 变量卡诺图

逻辑函数的卡诺图

卡诺图：将逻辑函数的最小项排列成矩阵形式，并且使矩阵的横方向和纵方向的逻辑变量的取值，按照**格雷码(循环码)**的顺序排列。



卡诺图的特点是：任意两个逻辑相邻的最小项，在图中也是相邻的。

- 逻辑函数是以真值表或者以最小项表达式给出：在卡诺图中，那些使真值表中函数值为1的最小项、或给定逻辑函数中的最小项，相对应的方格内填入1，其余的方格内填入0。

$$Y(A, B, C, D) = \sum m(1, 3, 4, 6, 7, 11, 14, 15)$$

$AB \backslash CD$					
		00	01	11	10
m_1 m_3	00	0	1	0	0
	01	1	0	0	0
	11	1	1	1	1
	10	0	1	1	0

m_4 (points to cell 01, 00)
 m_{11} (points to cell 11, 11)
 m_7 (points to cell 01, 10)
 m_6 (points to cell 11, 01)
 m_{15} (points to cell 11, 11)
 m_{14} (points to cell 11, 10)

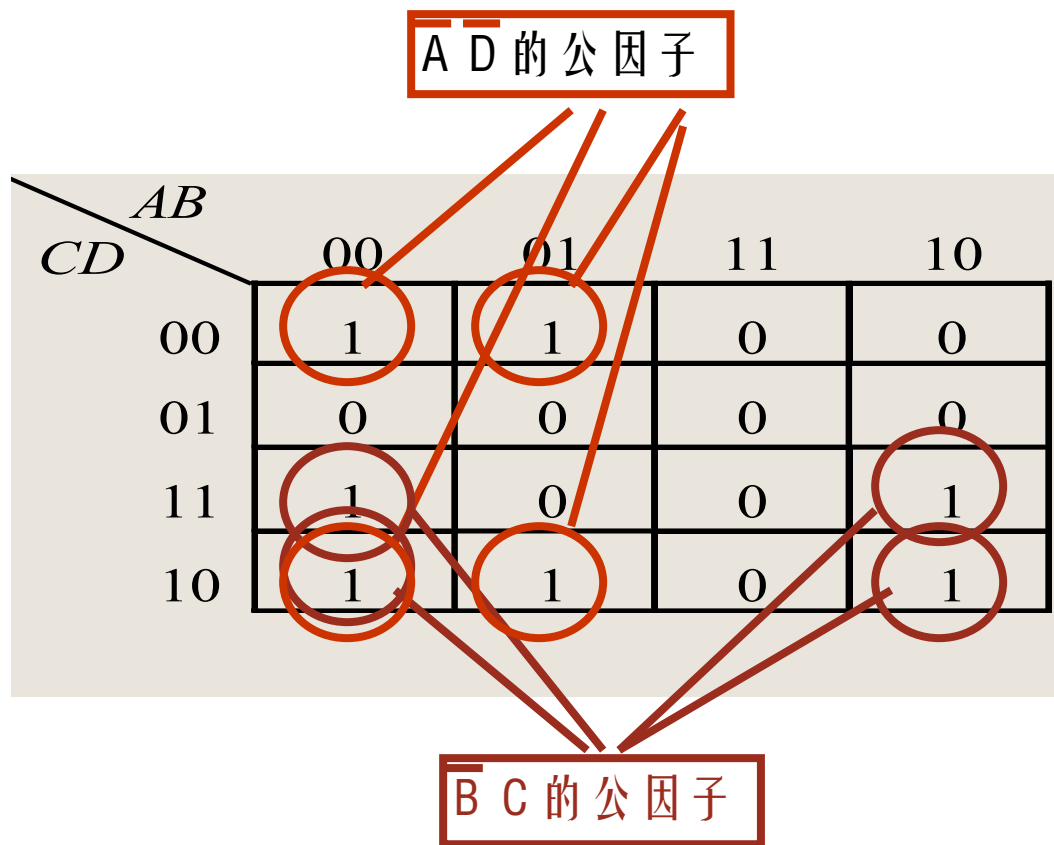
- 逻辑函数以一般的逻辑表达式给出：先将函数变换为与或表达式（不必变换为最小项之和的形式），然后在卡诺图上，找出包含每一个乘积项的那些最小项（该乘积项就是这些最小项的公因子），相对应的方格内填入1，其余的方格内填入0。

$$Y = \overline{(A + D)(B + \overline{C})}$$

变换为与
或表达式

$$Y = \overline{A}\overline{D} + \overline{B}C$$

说明：如果求得了函数Y的反函数 $\overline{Y}$ ，则对 $\overline{Y}$ 中所包含的各个最小项，在卡诺图相应方格内填入0，其余方格内填入1，即可得Y的卡诺图。



利用卡诺图化简

(1) 任何两个 (2^1 个) 标1的相邻最小项, 可以合并为一项, 并消去一个变量 (消去互为反变量的因子, 保留公因子)。

$\begin{array}{c} AB \\ \swarrow C \end{array}$		00	01	11	10
		0	1	0	1
0	1	0	0	1	0
1	0	1	1	0	0

$$\cancel{\bar{A}\bar{B}\bar{C}} + \cancel{A\bar{B}\bar{C}} = \bar{B}\bar{C}$$

$$\cancel{\bar{A}BC} + \cancel{ABC} = BC$$

$\begin{array}{c} AB \\ \swarrow CD \end{array}$		00	01	11	10
		00	01	00	01
00	0	1	0	0	0
01	0	0	0	0	1
11	0	0	0	0	1
10	0	1	0	0	0

$$\cancel{A\bar{B}\bar{C}D} + \cancel{A\bar{B}CD} = A\bar{B}D$$

$$\cancel{\bar{A}B\bar{C}\bar{D}} + \cancel{\bar{A}BC\bar{D}} = \bar{A}B\bar{D}$$

(2) 任何4个 (2^2 个) 标1的相邻最小项, 可以合并为一项, 并消去2个变量。(消去互为反变量的因子, 保留公因子)。

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	1	1	0

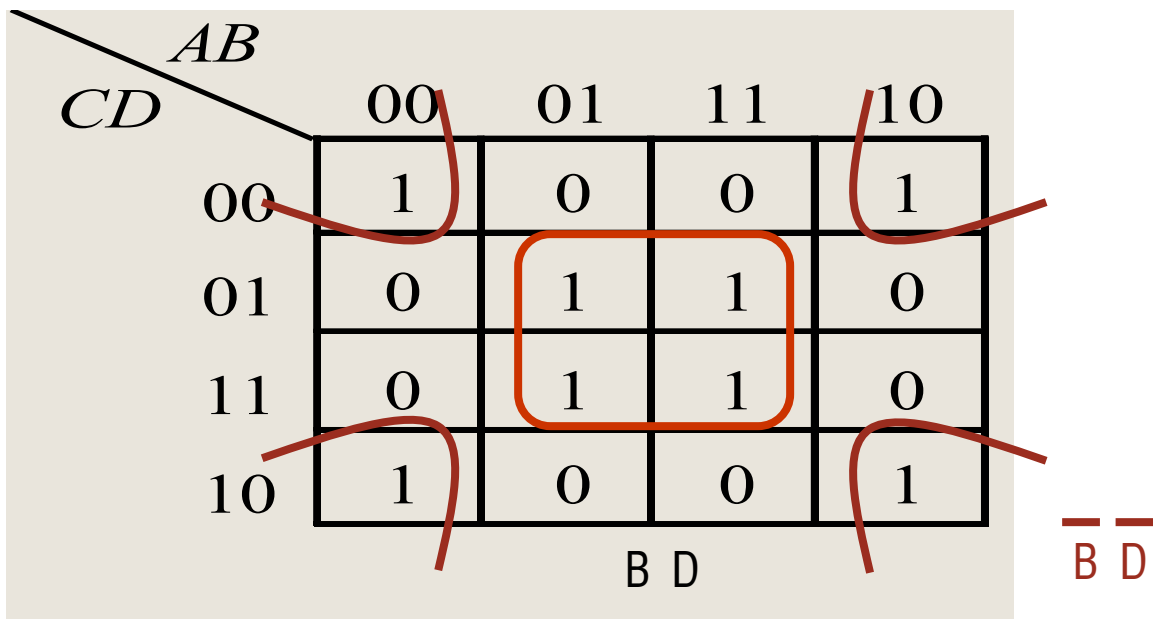
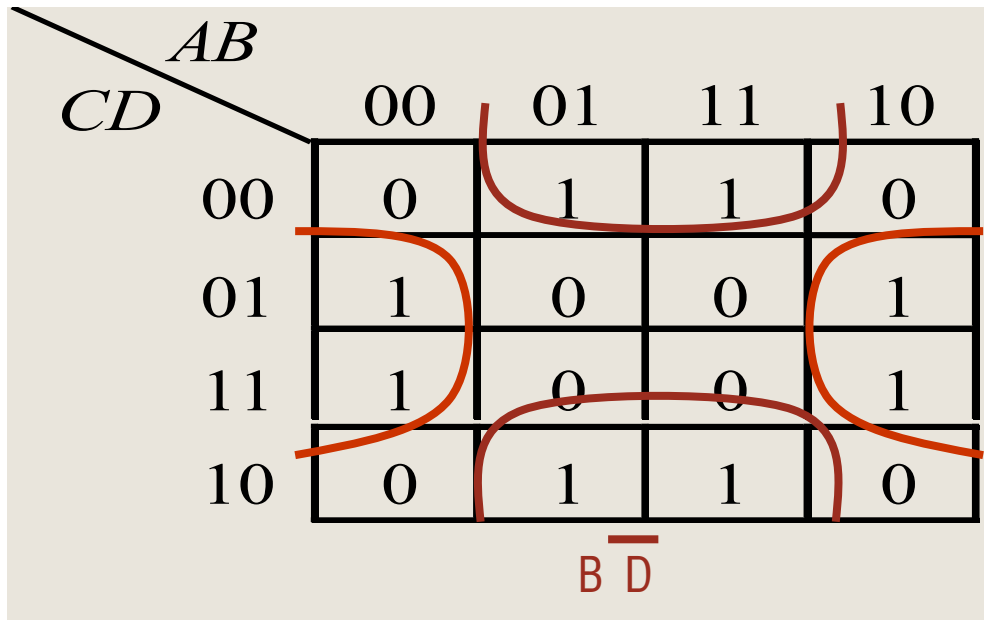
$$\begin{aligned}
 & \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + A\overline{B}C \\
 &= (\overline{A}\overline{B} + \overline{A}B + A\overline{B} + AB)\overline{C} \\
 &= \overline{C}
 \end{aligned}$$

$$\overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C} + ABC = (\overline{A}\overline{C} + \overline{A}C + A\overline{C} + AC)B = B$$

		AB			
		00	01	11	10
CD	00	0	1	0	0
	01	1	1	1	1
	11	0	1	1	0
	10	0	1	0	0

$\overline{C}D$

$\overline{A}B$



(3) 任何8个 (2^3 个) 标1的相邻最小项, 可以合并为一项, 并消去3个变量。
(消去互为反变量的因子, 保留公因子)。

$AB \backslash CD$		AB			
		00	01	11	10
00	0	0	0	0	0
01	1	1	1	1	1
11	1	1	1	1	1
10	0	0	0	0	0

D

相邻最小项的数目必须为 2^i 个才能合并为一项, 并消去 i 个变量。包含的最小项数目越多, 即由这些最小项所形成的圈越大, 消去的变量也就越多, 从而所得到的逻辑表达式就越简单。这就是利用卡诺图化简逻辑函数的基本原理。

AB CD		AB			
		00	01	11	10
00	1	0	0	1	
01	1	0	0	1	
11	1	0	0	1	
10	1	0	0	1	

$\overline{B}$

逻辑函数图形化化简步骤

逻辑表达式或真值表

$$Y(A, B, C, D) = \sum m(3, 5, 7, 8, 11, 12, 13, 15)$$

1

卡诺图

1

AB CD					
		00	01	11	10
00	0	0	1	1	
01	0	1	1	0	
11	1	1	1	1	
10	0	0	0	0	

2

①圈越大越好，但每个圈中标1的方格数目必须为 2^i 个。②同一个方格可同时画在几个圈内，但每个圈都要有新的方格，否则它就是多余的。③不能漏掉任何一个标1的方格。

合并最小项

3

最简与或表达式

2

		$\overline{A} \overline{C} \overline{D}$			
$AB \backslash CD$		00	01	11	10
	00	0	0	1	1
	01	0	1	1	0
	11	1	1	1	1
	10	0	0	0	0
		$B D$			

冗余项

3

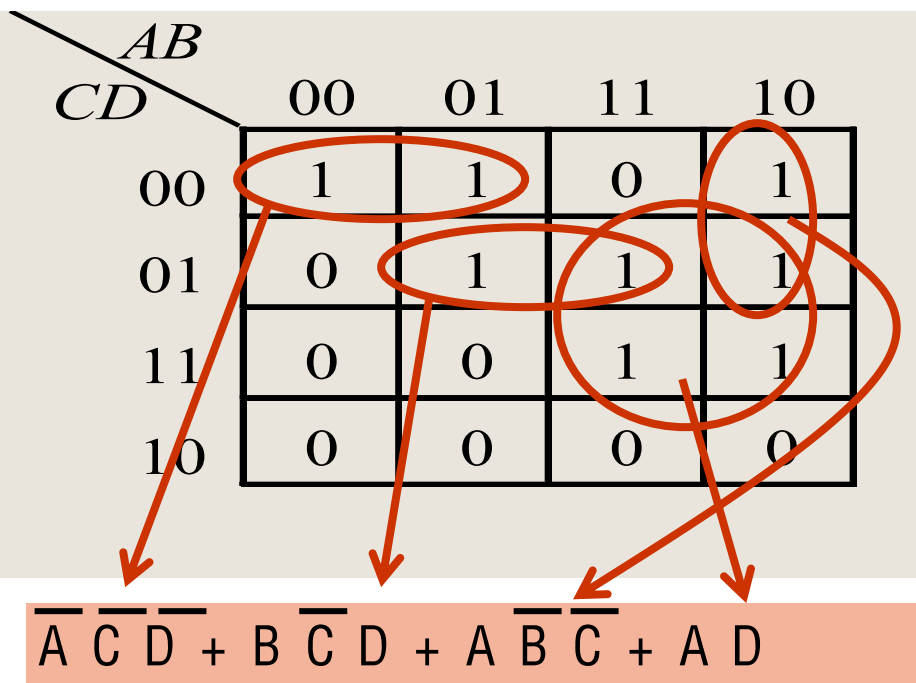
将代表每个圈的乘积项相加

$$Y(A, B, C, D) = BD + CD + A\overline{C}\overline{D}$$

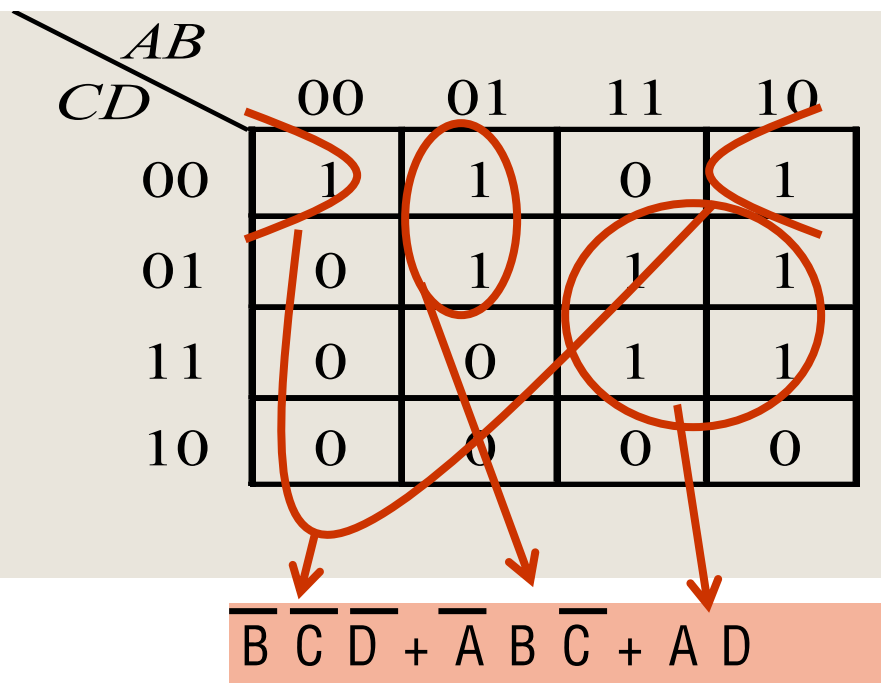
两点说明:

① 在有些情况下, 最小项的圈法不只一种, 得到的各个乘积项组成的与或表达式各不相同, 哪个是最简的, 要经过比较、检查才能确定。

圈个数越少越简

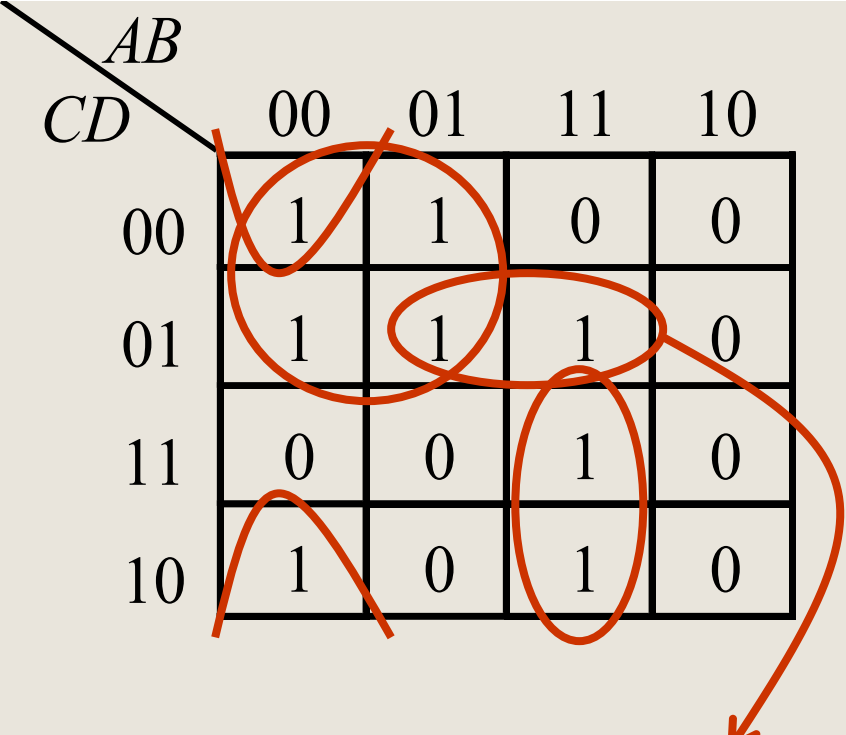


不是最简



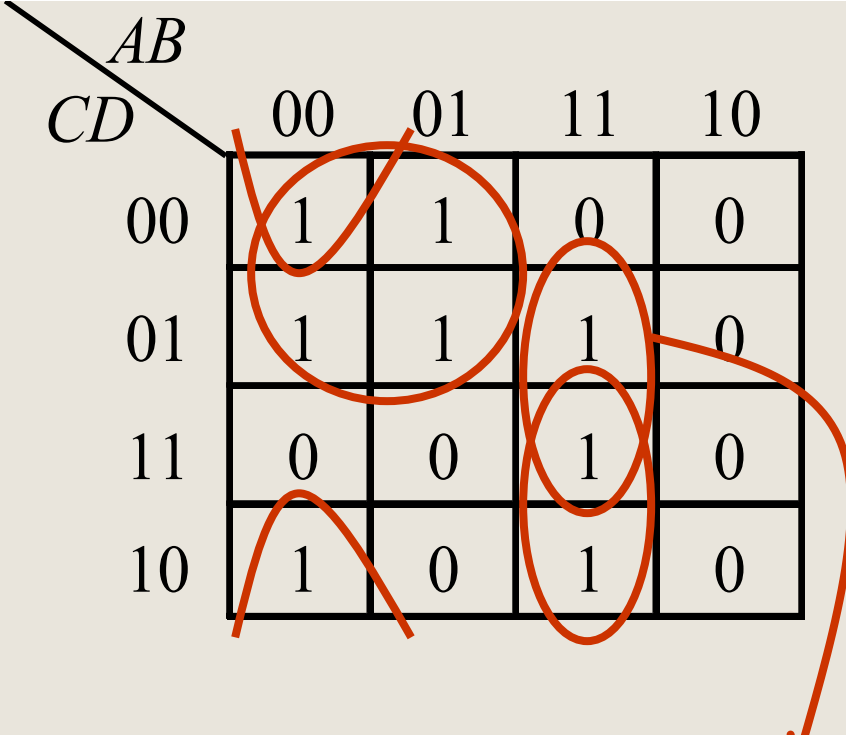
最简

② 在有些情况下，不同圈法得到的与或表达式都是最简形式。即一个函数的最简与或表达式不是唯一的。



	AB			
CD	00	01	11	10
00	1	1	0	0
01	1	1	1	0
11	0	0	1	0
10	1	0	1	0

$\overline{A} \overline{C} + \overline{A} \overline{B} \overline{D} + A B C + \underline{B \overline{C} D}$



	AB			
CD	00	01	11	10
00	1	1	0	0
01	1	1	1	0
11	0	0	1	0
10	1	0	1	0

$\overline{A} \overline{C} + \overline{A} \overline{B} \overline{D} + A B C + \underline{A B D}$

圈的个数相同，每个圈越大越简

包含约束项的逻辑函数化简

约束项：不会出现的变量取值，所对应的最小项。

- 由最小项的性质知道，只有在函数的与或式中出现的最小项，其值才会为1。而约束项是不出现的最小项，所以其值总是等于0。
- **约束条件：**由约束项加起来所构成的值为0的逻辑表达式。因为约束项的值恒为0，所以**约束条件是一个值恒为0的条件等式**。

例：判断一位十进制数是否为偶数。

A B C D	Y	A B C D	Y	说 明
0 0 0 0	1	1 0 0 0	1	
0 0 0 1	0	1 0 0 1	0	
0 0 1 0	1	1 0 1 0	×	不会出现
0 0 1 1	0	1 0 1 1	×	不会出现
0 1 0 0	1	1 1 0 0	×	不会出现
0 1 0 1	0	1 1 0 1	×	不会出现
0 1 1 0	1	1 1 1 0	×	不会出现
0 1 1 1	0	1 1 1 1	×	不会出现

$AB \backslash CD$		AB			
		00	01	11	10
CD	00	1	1	×	1
	01	0	0	×	0
	11	0	0	×	×
	10	1	1	×	×

输入变量A, B, C, D取值为0000-1001时, 逻辑函数Y有确定的值, 根据题意, 偶数时为1, 奇数时为0。

$$Y(A, B, C, D) = \sum m(0, 2, 4, 6, 8)$$

A, B, C, D取值为1010 -1111的情况不会出现或不允许出现, 对应的最小项属于约束项。约束项之和构成的逻辑表达式叫做**约束条件**, 用一个值恒为 0 的条件等式表示。

$$\sum d(10, 11, 12, 13, 14, 15) = 0$$

含有约束条件的逻辑函数可以表示成如下形式:

$$F(A, B, C, D) = \sum m(0, 2, 4, 6, 8) + \sum d(10, 11, 12, 13, 14, 15)$$

在逻辑函数的化简中，充分利用约束项可以得到更加简单的逻辑表达式，其相应的逻辑电路也更简单。

$AB \backslash CD$		00	01	11	10
		00	01	11	10
00	1	1	×	1	
01	0	0	×	0	
11	0	0	×	×	
10	1	1	×	×	

不利用约束项的化简
结果为：

$$Y = \bar{A} \bar{D} + \bar{B} \bar{C} \bar{D}$$

利用约束项的化简结果为：

$$Y = \bar{D}$$

逻辑函数的表示方法

- 真值表
- 卡诺图
- 逻辑表达式
- 逻辑图
- 波形图

① 真值表

真值表：是由变量的所有可能取值组合及其对应的函数值所构成的表格。

真值表列写方法：每一个变量均有0、1两种取值， n 个变量共有 2^n 种不同的取值，将这 2^n 种不同的取值按顺序（一般按二进制递增规律）排列起来，同时在相应位置上填入函数的值，便可得到逻辑函数的真值表。

例如：当 $A=B=1$ 、或 $B=C=1$ 时，函数 $Y=1$ ；否则 $Y=0$ 。

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

② 卡诺图

卡诺图：是由表示变量的所有可能取值组合的小方格所构成的图形。

逻辑函数卡诺图的填写方法：在那些使函数值为1的变量取值组合所对应的小方格内填入1，其余的方格内填入0，便得到该函数的卡诺图。

AB					
		00	01	11	10
C	0	0	0	1	0
	1	0	1	1	0

③ 逻辑表达式

逻辑表达式：是由逻辑变量和与、或、非3种运算符连接起来所构成的式子。

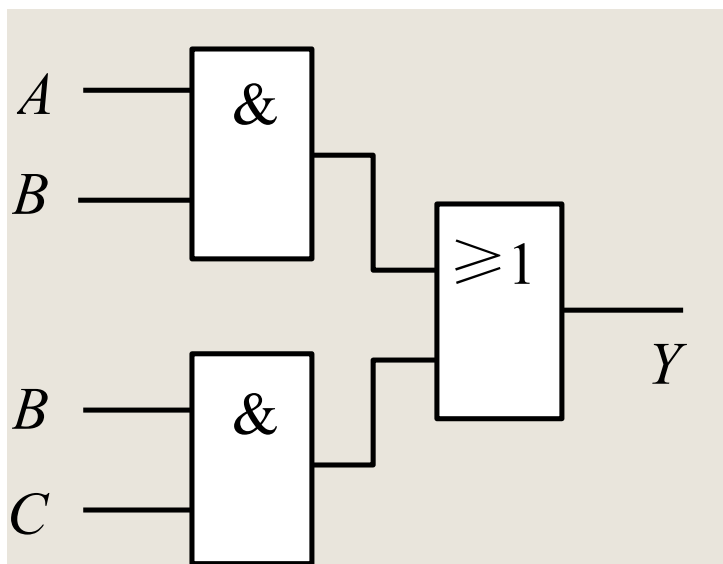
函数的**标准与或表达式**的列写方法：将函数的真值表中那些使函数值为1的最小项相加，便得到函数的标准与或表达式。

$$Y = \bar{A}BC + A\bar{B}\bar{C} + ABC \\ = \sum m(3,6,7)$$

④ 逻辑图

逻辑图：是由表示逻辑运算的逻辑符号所构成的图形。

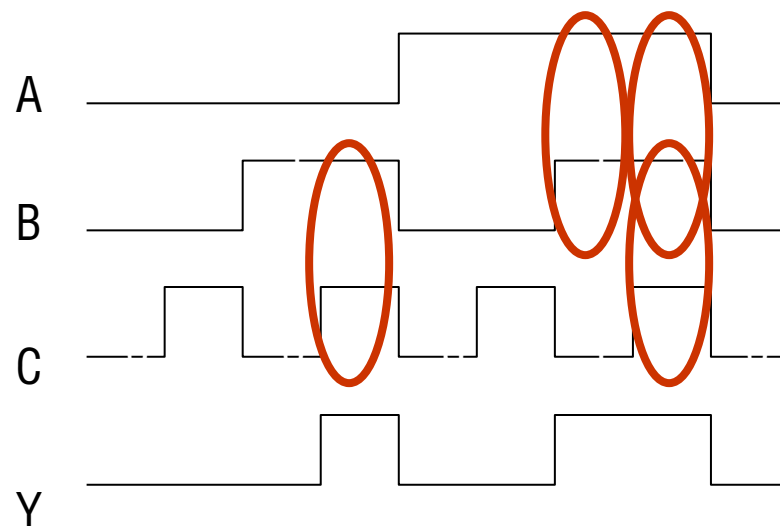
$$Y = A B + B C$$



⑤ 波形图

波形图：是由输入变量的所有可能取值组合的高、低电平及其对应的输出函数值的高、低电平所构成的图形。

$$Y = A B + B C$$



2.1.2 门电路

半导体的导电特性

原子是由带正电的原子核和负电的电子组成，电子分几层围绕原子核不停的运动。在同一原子中，原子核对内层电子吸引力较大，对外层电子吸引力较小。影响物质电性能的主要是外层电子。

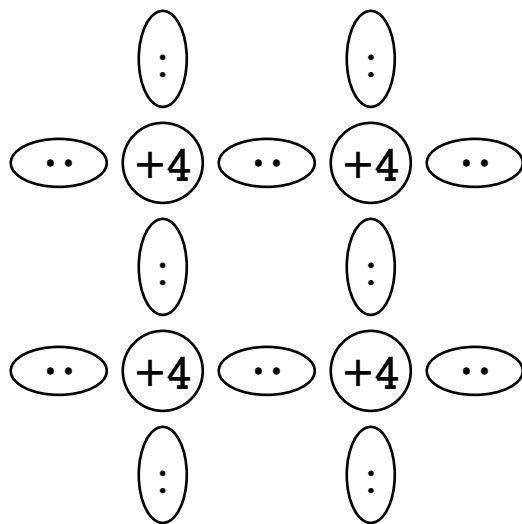
导体：金属的外层电子受原子核的束缚力最小，因此有大量电子能够挣脱束缚成为自由电子。这些自由电子成为载流子，它们在外电场的作用下作定向运动形成电流。所以金属是导体。

绝缘体：绝缘体中，原子的外层电子受原子核的束缚力很大，不容易挣脱出来。由于载流子很少，所以绝缘体导电性能很差。

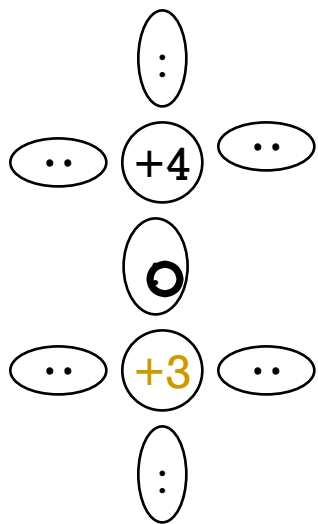
半导体：半导体的结构比较特殊，外层电子既不像导体那样容易挣脱，也不象绝缘体那样束缚很紧。其导电性能介于导体和绝缘体之间。

本征半导体：

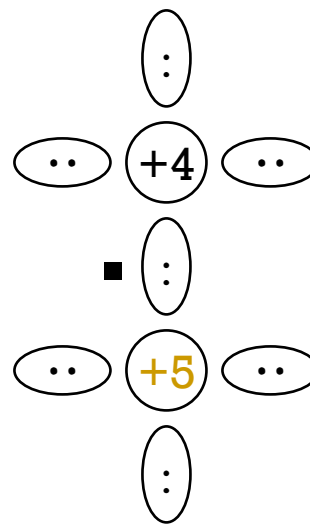
本征半导体是指那些四价元素，例如硅或锗，它们的核外有四个电子，可以构成稳定的晶格结构。每相邻两原子都共用一对电子，电子对中的任一电子既围绕自身原子核运动，也时常出现在相邻原子核所属的轨道上。这样的组合称为“共价键结构”。



本征半导体本身并没有使用价值。但是如果向本征半导体中有目的掺入某些杂质，就可以大大改变它的导电性能。例如，分别掺入三价元素硼和五价元素磷，就可以分别得到P型半导体和N型半导体。



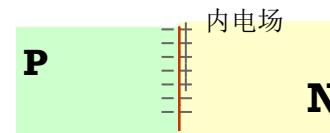
P型半导体
向四价的本征半导体(硅或锗)注入三价原子(硼或铝)，就构成P型半导体。



N型半导体
向四价的本征半导体(硅或锗)注入五价原子(砷或磷)，就构成N型半导体。

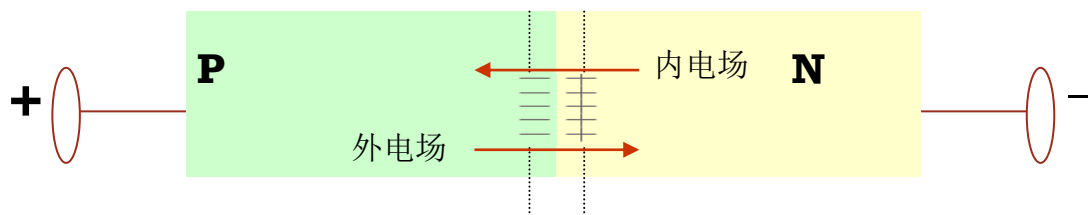
PN结的单向导电性

PN结结构：一块硅片上，用不同的掺杂工艺使其一边形成N型半导体，另一边形成P型半导体，两种半导体的交界面附近的区域为PN结。

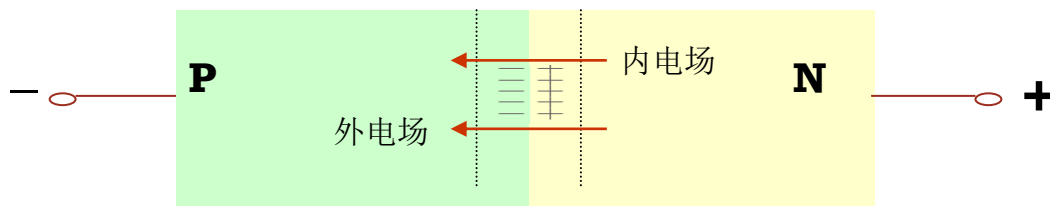


在PN结两端适当加以外部电压，可以实现单向导电。

加正向电压时PN结导通：加正向电压时，内外电场方向相反，阻挡层变窄，相当于PN结的电阻很小。

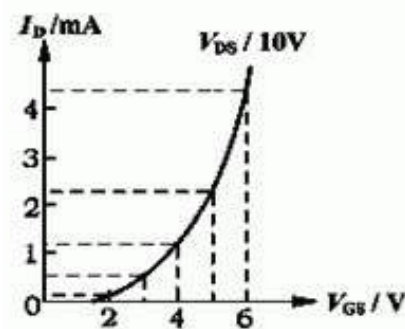
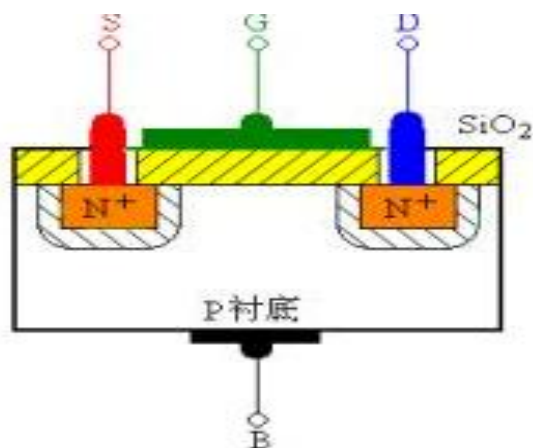


加反向电压时PN结截止：加反向电压时，内外电场方向相同，阻挡层变宽，相当于PN结的电阻很大。



MOS管：由金属-氧化物-半导体构成的，分为N沟道和P沟道MOS管

- N沟道MOS管：在P型衬底上，利用光刻、扩散等方法，制作出两个高掺杂浓度N⁺型区，并引出电极，源极S和漏极D，同时在源极和漏极之间的二氧化硅SiO₂绝缘层上，制作一个金属电极栅极G。

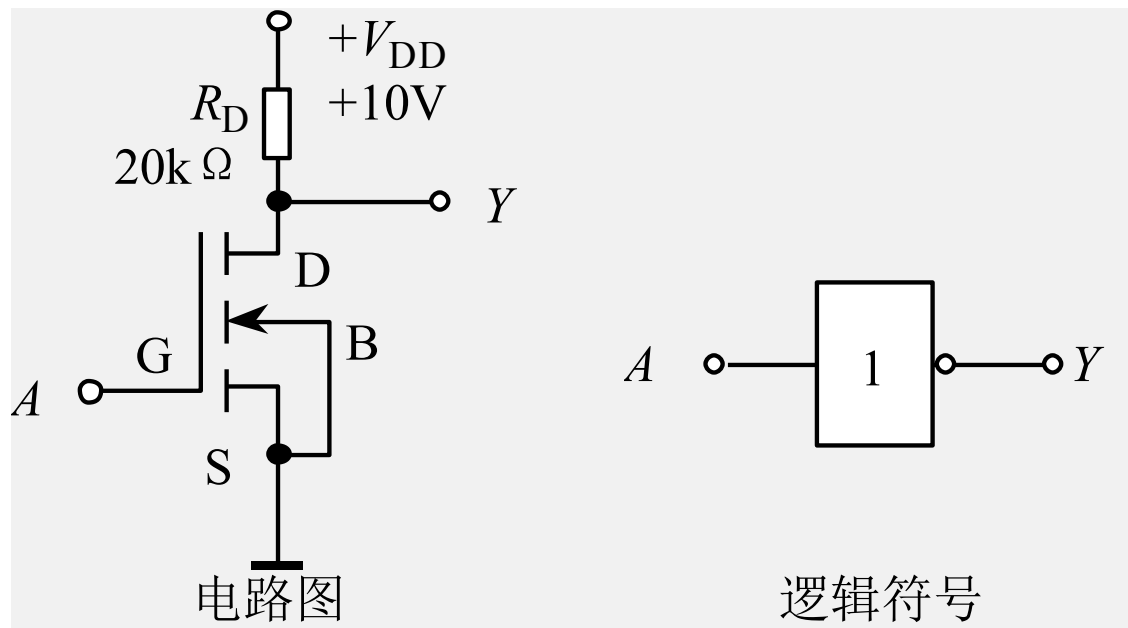


MOS管是通过栅极电压 u_{GS} 控制其工作状态。

- 截止状态： $U_{GS} \leq U_{TN}$, $i_D = 0$
- 导通状态： $U_{GS} > U_{TN}$, $i_D > 0$

- P沟道MOS管：与N沟道增强型MOS管有明显的对偶关系。其衬底是N型硅半导体，漏极和源极是两个高浓度掺杂的P⁺区，而且它的 u_{GS} 、 u_{DS} 、开启电压 U_{TP} 都是负值。

MOS管非门



①当 $u_A = 0V$ 时，由于 $u_{GS} = u_A = 0V$ ，小于开启电压 U_{TN} ，所以MOS管截止。输出电压为 $u_Y = V_{DD} = 10V$ 。

②当 $u_A = 10V$ 时，由于 $u_{GS} = u_A = 10V$ ，大于开启电压 U_{TN} ，所以MOS管导通，导通电阻很小，输出电压为 $u_Y \approx 0V$ 。

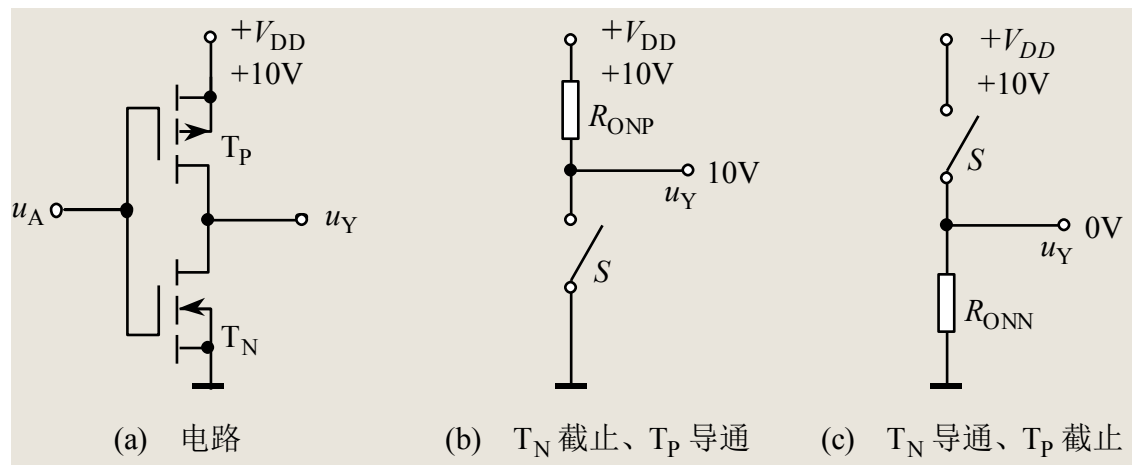
$$Y = \overline{A}$$

CMOS集成电路

- **CMOS** (Complementary Metal Oxide Semiconductor) 是互补对称式金属-氧化物-半导体电路的缩写, CMOS具有电压控制、功耗极小、连接方便等一系列优点, 是目前应用最广泛的集成电路之一。
- 用P沟道增强型MOS管和N沟道增强型MOS管, 按照互补对称形式连接构成。

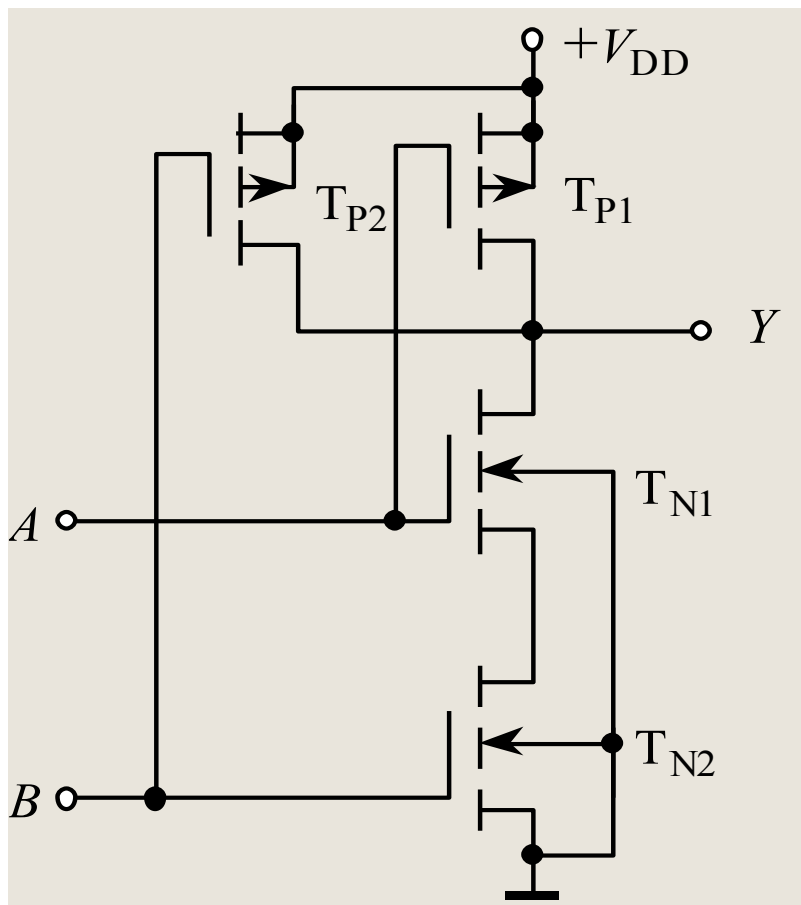
CMOS非门

$$Y = \overline{A}$$



- (1) $u_A = 0V$ 时, T_N 截止, T_P 导通。输出电压 $u_Y = V_{DD} = 10V$ 。
- (2) $u_A = 10V$ 时, T_N 导通, T_P 截止。输出电压 $u_Y = 0V$ 。

CMOS与非门

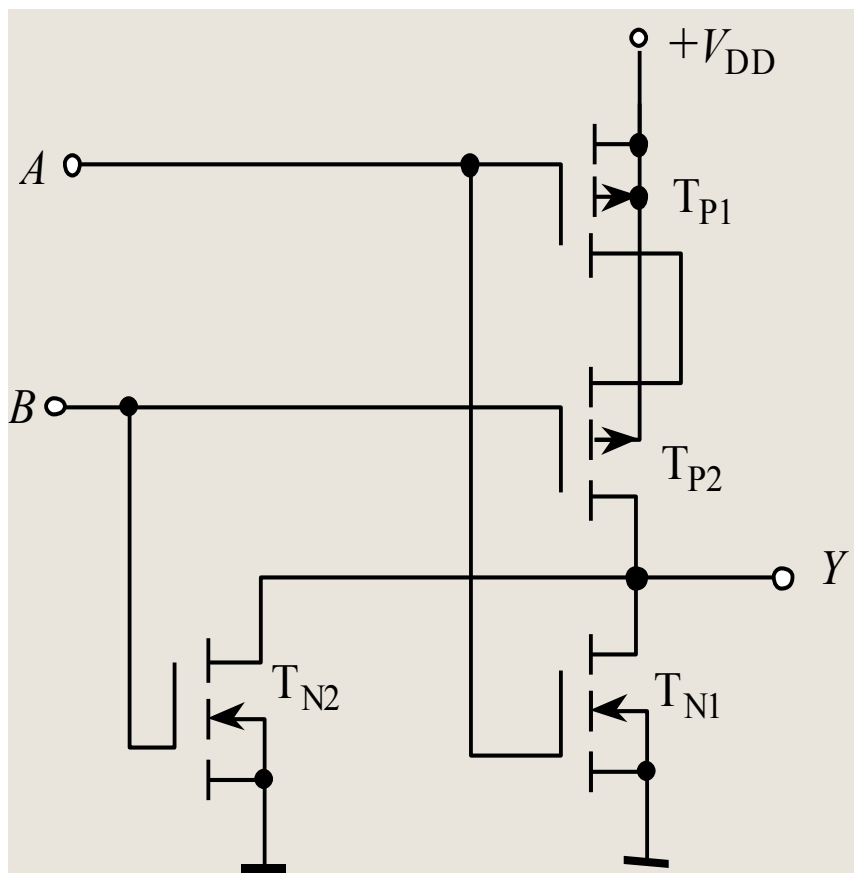


① A 、 B 当中有一个或全为低电平时， T_{N1} 、 T_{N2} 中有一个或全部截止， T_{P1} 、 T_{P2} 中有一个或全部导通，输出 Y 为高电平。

② 只有当输入 A 、 B 全为高电平时， T_{N1} 和 T_{N2} 才会都导通， T_{P1} 和 T_{P2} 才会都截止，输出 Y 才会为低电平。

$$Y = \overline{A \cdot B}$$

CMOS或非门

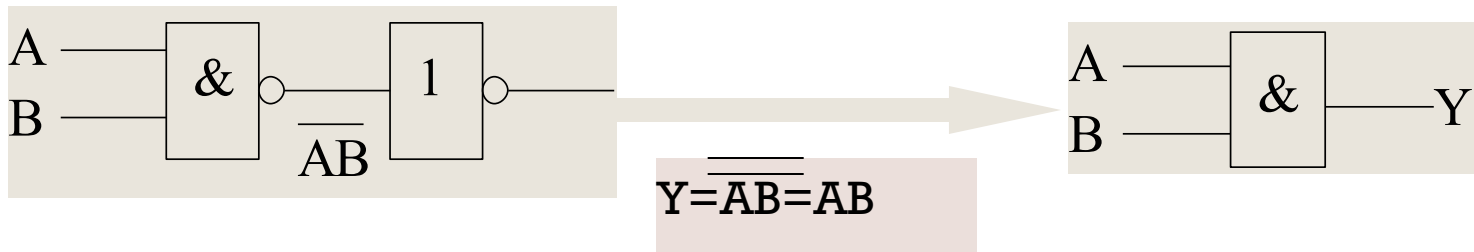


①只要输入 A 、 B 当中有一个或全为高电平， T_{P1} 、 T_{P2} 中有一个或全部截止， T_{N1} 、 T_{N2} 中有一个或全部导通，输出 Y 为低电平。

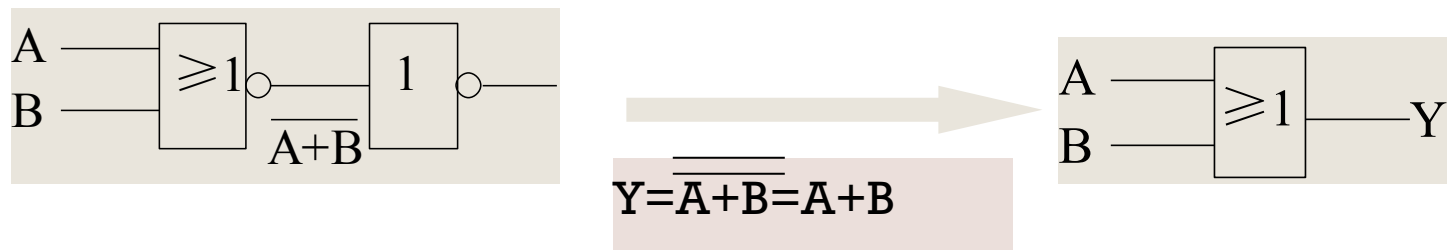
②只有当 A 、 B 全为低电平时， T_{P1} 和 T_{P2} 才会都导通， T_{N1} 和 T_{N2} 才会都截止，输出 Y 才会为高电平。

$$Y = \overline{A + B}$$

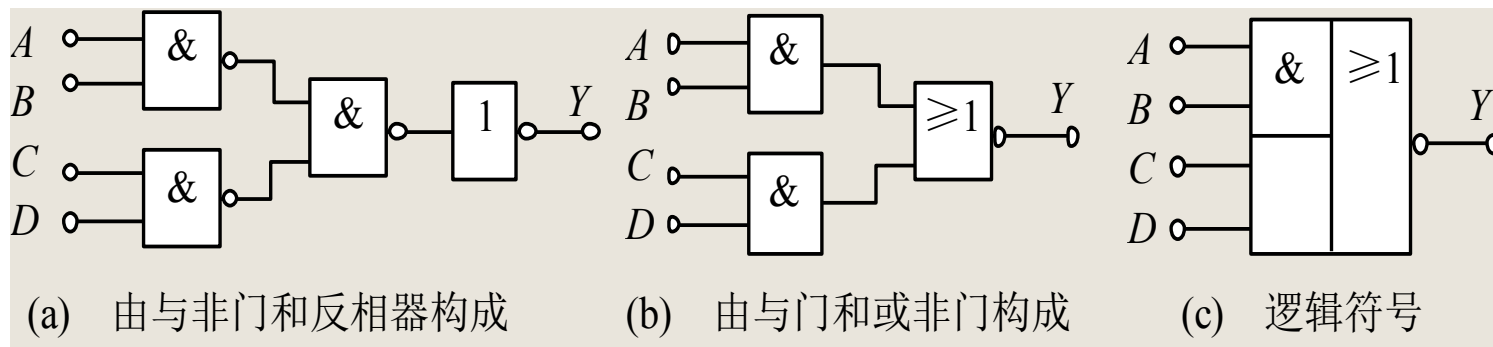
与门



或门

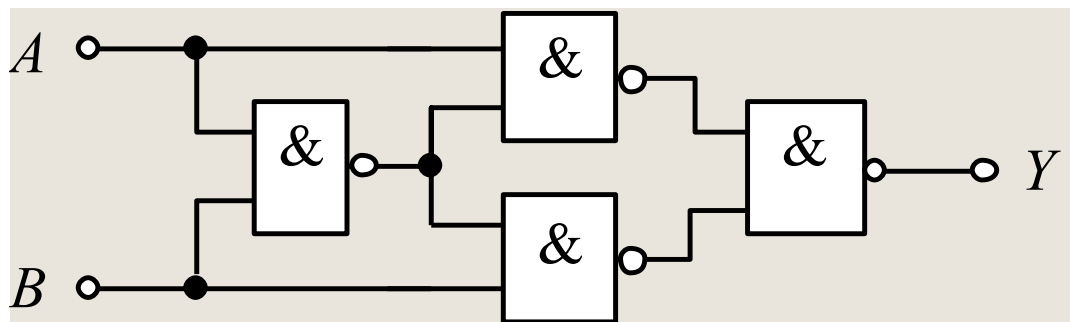


CMOS与或非门



$$Y = \overline{\overline{A \cdot B \cdot C \cdot D}} = \overline{A \cdot B + C \cdot D}$$

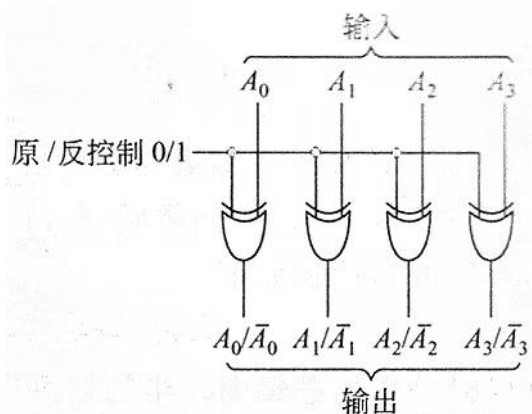
CMOS异或门



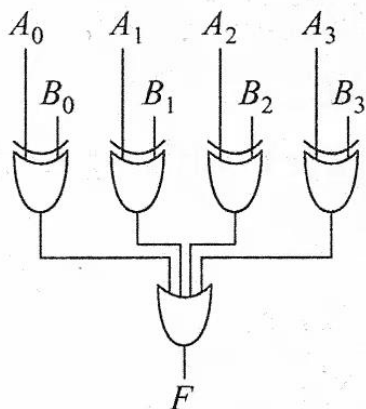
$$\begin{aligned} Y &= \overline{\overline{A \cdot AB} \cdot \overline{AB \cdot B}} \\ &= \overline{A\overline{B}} + \overline{\overline{A}B} \\ &= A \oplus B \end{aligned}$$

异或门的简单应用

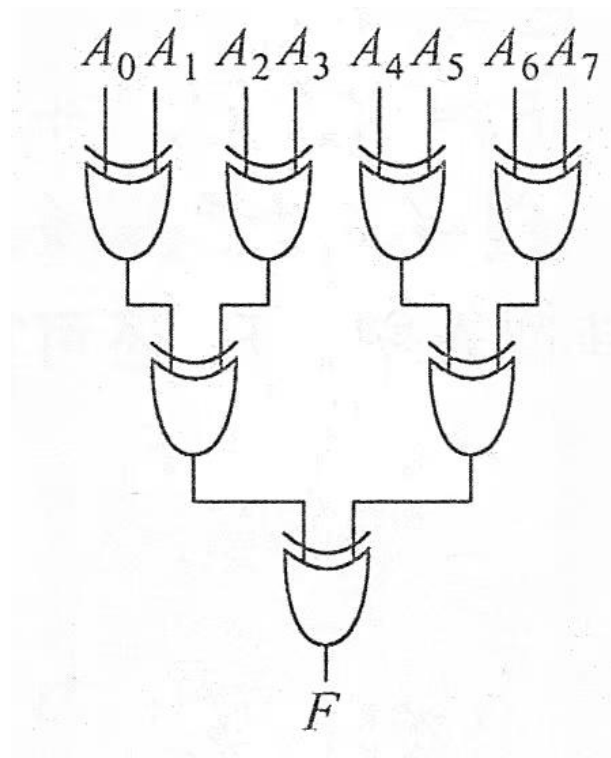
■ 可控原/反码输出电路



■ 数码比较器



■ 奇偶检测电路



2.2 计算机中的数字逻辑电路

- 计算机的基本组成（冯诺依曼体系结构）
 - 控制器、运算器、存储器、输入设备、输出设备
 - 每部分都是由集成电路板组成，每个电路板上是由许多集成电路芯片组成，每个集成电路的基础电路是逻辑门电路（gate）。
- 数字逻辑电路可以分为
 - 组合逻辑电路
 - 时序逻辑电路

2.2.1 组合逻辑电路

- 加法器
- 译码器
- 数据选择器
- 只读存储器 (ROM)
- 算术逻辑单元 (ALU)

组合逻辑电路

组合逻辑电路：输出仅由输入决定，与电路当前状态无关；电路结构中**无**反馈环路（无记忆）



$$Y_0 = F_0(I_0, I_1, \dots, I_{n-1})$$

$$Y_1 = F_1(I_0, I_1, \dots, I_{n-1})$$

...

$$Y_{m-1} = F_{m-1}(I_0, I_1, \dots, I_{n-1})$$

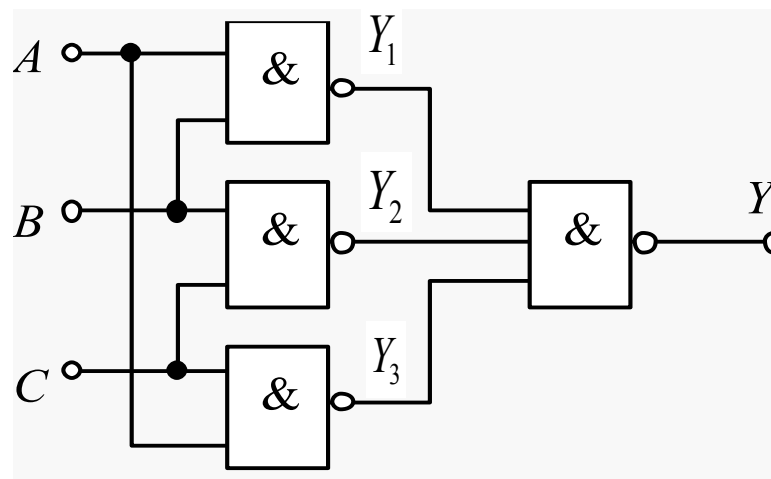
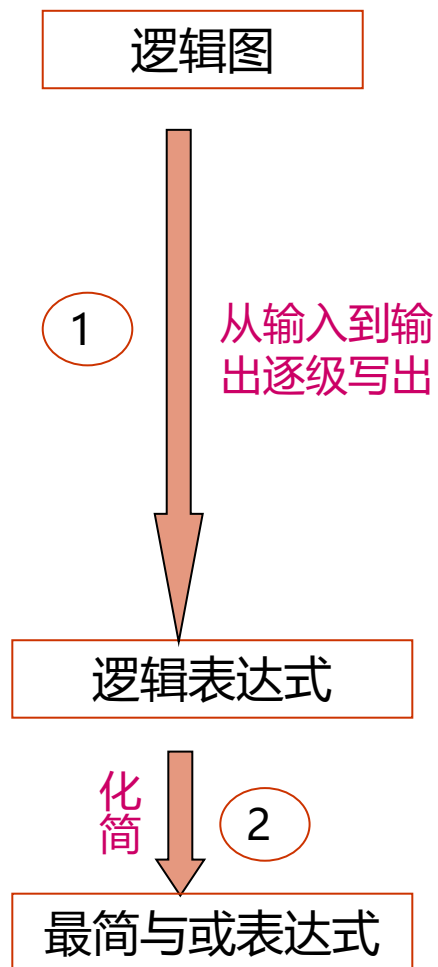
$$Y(t_n) = F[I(t_n)]$$

t_n 时刻的稳定输出 $Y(t_n)$,仅取决于 t_n 时刻的输入 $I(t_n)$

组合逻辑电路的分析

- **电路分析**：已知逻辑电路，寻找输出与输入之间的逻辑关系，确定电路功能的过程。
- 组合逻辑电路的**分析步骤**：
 - 根据给定的逻辑图，从输入到输出逐级写出逻辑函数表达式；
 - 用公式法或卡诺图法化简逻辑函数表达式；
 - 根据化简的逻辑函数表达式列出真值表（可省略）；
 - 对逻辑函数表达式或真值表进行分析，概括出给定组合逻辑电路的逻辑功能。

组合逻辑电路的基本分析方法



1

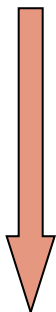
$$\left. \begin{aligned} Y_1 &= \overline{AB} \\ Y_2 &= \overline{BC} \\ Y_3 &= \overline{CA} \end{aligned} \right\} Y = \overline{Y_1 Y_2 Y_3} = \overline{\overline{AB} \overline{BC} \overline{AC}}$$

2

$$Y = AB + BC + CA$$

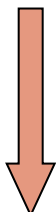
$$Y = AB + BC + CA$$

最简与或表达式



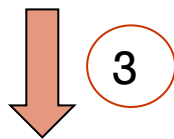
3

真值表



4

说明电路的功能



<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

4



当输入A、B、C中有2个或3个为1时，输出Y为1，否则输出Y为0。

可以认为电路是一种3人表决用的组合电路：只要有2票或3票同意，表决就通过。

组合逻辑电路的设计

- **电路设计：**根据所要求实现的逻辑功能，设计出相应的逻辑电路。
 - 通常以电路简单、所用器件最少为目标。
- **设计步骤：**
 1. 分析设计要求，对问题逻辑抽象，确定输入、输出变量。
 2. 对输入和输出变量赋予0、1值，根据输入输出之间的因果关系，列出输入输出对应关系表，即真值表。
 3. 根据真值表写出逻辑表达式或卡诺图，并根据设计要求进行化简和变换。
 4. 根据化简式画出逻辑电路图。

加法器——半加器和全加器

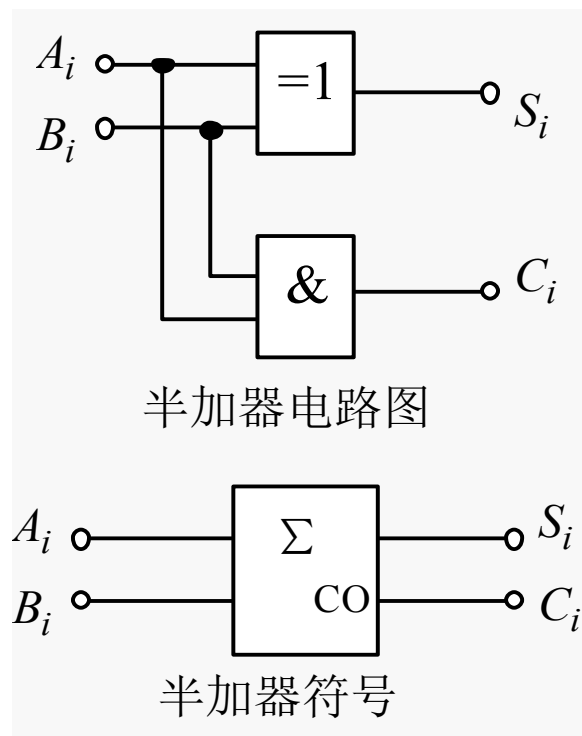
半加器

能对两个1位二进制数进行相加，而求得和及进位的逻辑电路称为**半加器**。

半加器真值表

加数	A_i	B_i	S_i	C_i
			本位的和	向高位的进位
	0	0	0	0
	0	1	1	0
	1	0	1	0
	1	1	0	1

$$S_i = \bar{A}_i B_i + A_i \bar{B}_i = A_i \oplus B_i$$
$$C_i = A_i B_i$$



全加器

能对两个1位二进制数进行相加并考虑低位来的进位，即相当于3个1位二进制数相加，求得和及进位的逻辑电路称为**全加器**。

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$A_i B_i$		C_{i-1}			
		00	01	11	10
0		0	1	0	1
1		1	0	1	0

S_i 的卡诺图

$$S_i = m_1 + m_2 + m_4 + m_7$$

$A_i B_i$		C_{i-1}			
		00	01	11	10
0		0	0	1	0
1		0	1	1	1

C_i 的卡诺图

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$

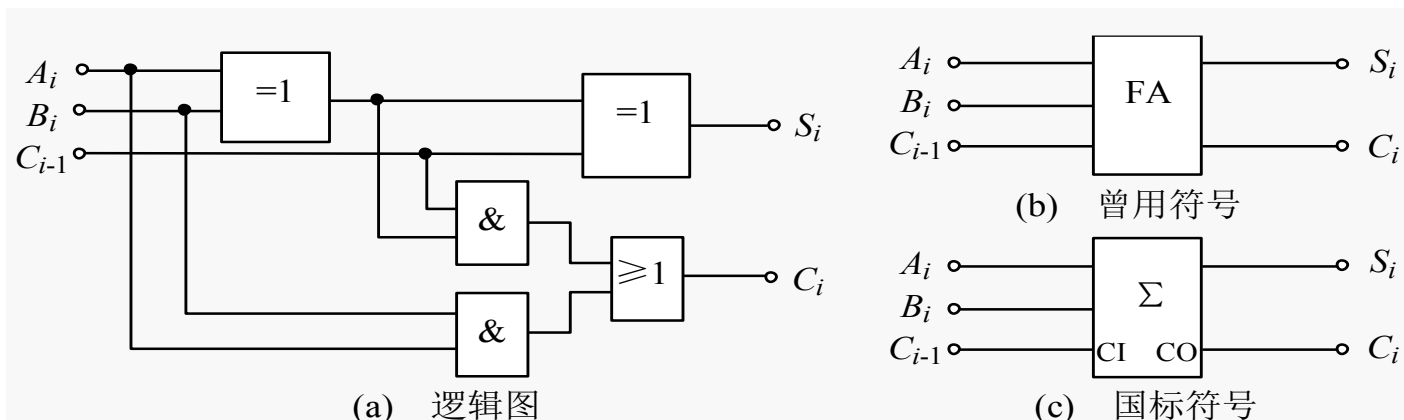
$$C_i = A_i B_i + m_5 + m_3$$

A_i, B_i : 加数, C_{i-1} : 低位来的进位, S_i : 本位的和, C_i : 向高位的进位。

全加器的逻辑图和逻辑符号

$$\begin{aligned}
 S_i &= m_1 + m_2 + m_4 + m_7 = \bar{A}_i \bar{B}_i C_{i-1} + \bar{A}_i B_i \bar{C}_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1} \\
 &= \bar{A}_i (\bar{B}_i C_{i-1} + B_i \bar{C}_{i-1}) + A_i (\bar{B}_i \bar{C}_{i-1} + B_i C_{i-1}) = \bar{A}_i (B_i \oplus C_{i-1}) + A_i \overline{(B_i \oplus C_{i-1})} \\
 &= A_i \oplus B_i \oplus C_{i-1}
 \end{aligned}$$

$$\begin{aligned}
 C_i &= m_3 + m_5 + A_i B_i = \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i B_i = (\bar{A}_i B_i + A_i \bar{B}_i) C_{i-1} + A_i B_i \\
 &= (A_i \oplus B_i) C_{i-1} + A_i B_i
 \end{aligned}$$

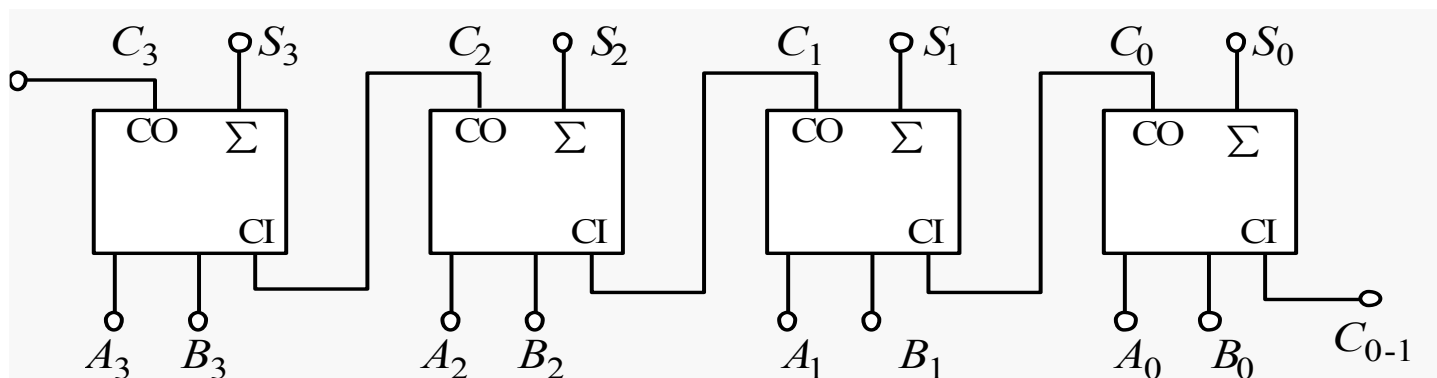


加法器

实现多位二进制数相加的电路称为**加法器**。

4位串行进位加法器

构成：把 n 个全加器串联起来，低位全加器的进位输出连接到相邻的高位全加器的进位输入。



特点：进位信号是由低位向高位逐级传递的，速度不高。

超前进位加法器（并行进位加法器）

超前进位加法器：作加法运算时，进位输入信号由二进制加数直接产生，而不需要等待低位加数相加后的进位，这样的加法器称之为超前进位加法器。

以4位二进制加法器为例：根据全加器的逻辑功能，我们可以得到4位二进制 ($D_3D_2D_1D_0$)加法器中， D_{i+1} ($i=0-3$)位全加器的进位输入信号 C_i 的表达式为：

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1} = (A_i \oplus B_i) C_{i-1} + A_i B_i$$

进位生成项

$$G_i = A_i B_i$$

低位进位传递条件

$$P_i = A_i \oplus B_i$$

进位表达式

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1} = G_i + P_i C_{i-1}$$

和表达式

$$S_i = A_i \oplus B_i \oplus C_{i-1} = P_i \oplus C_{i-1}$$

进位生成项

$$G_i = A_i B_i$$

低位进位传递条件

$$P_i = A_i \oplus B_i$$

进位表达式

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1} = G_i + P_i C_{i-1}$$

和表达式

$$S_i = A_i \oplus B_i \oplus C_{i-1} = P_i \oplus C_{i-1}$$

$$\begin{cases} S_0 = P_0 \oplus C_{0-1} \\ C_0 = G_0 + P_0 C_{0-1} \end{cases}$$

4位超前进位加法器递推公式

$$\begin{cases} S_1 = P_1 \oplus C_0 \\ C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0 + P_1 P_0 C_{0-1} \end{cases}$$

$$\begin{cases} S_2 = P_2 \oplus C_1 \\ C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{0-1} \end{cases}$$

$$\begin{cases} S_3 = P_3 \oplus C_2 \\ C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{0-1} \end{cases}$$

只要 $A_3A_2A_1A_0$ 、 $B_3B_2B_1B_0$ 和 C_{0-1} 给出后，便可以按上式直接确定 $C_3C_2C_1C_0$ 的值，提高运算速度，因为高位的运算不需再等待低位的进位。

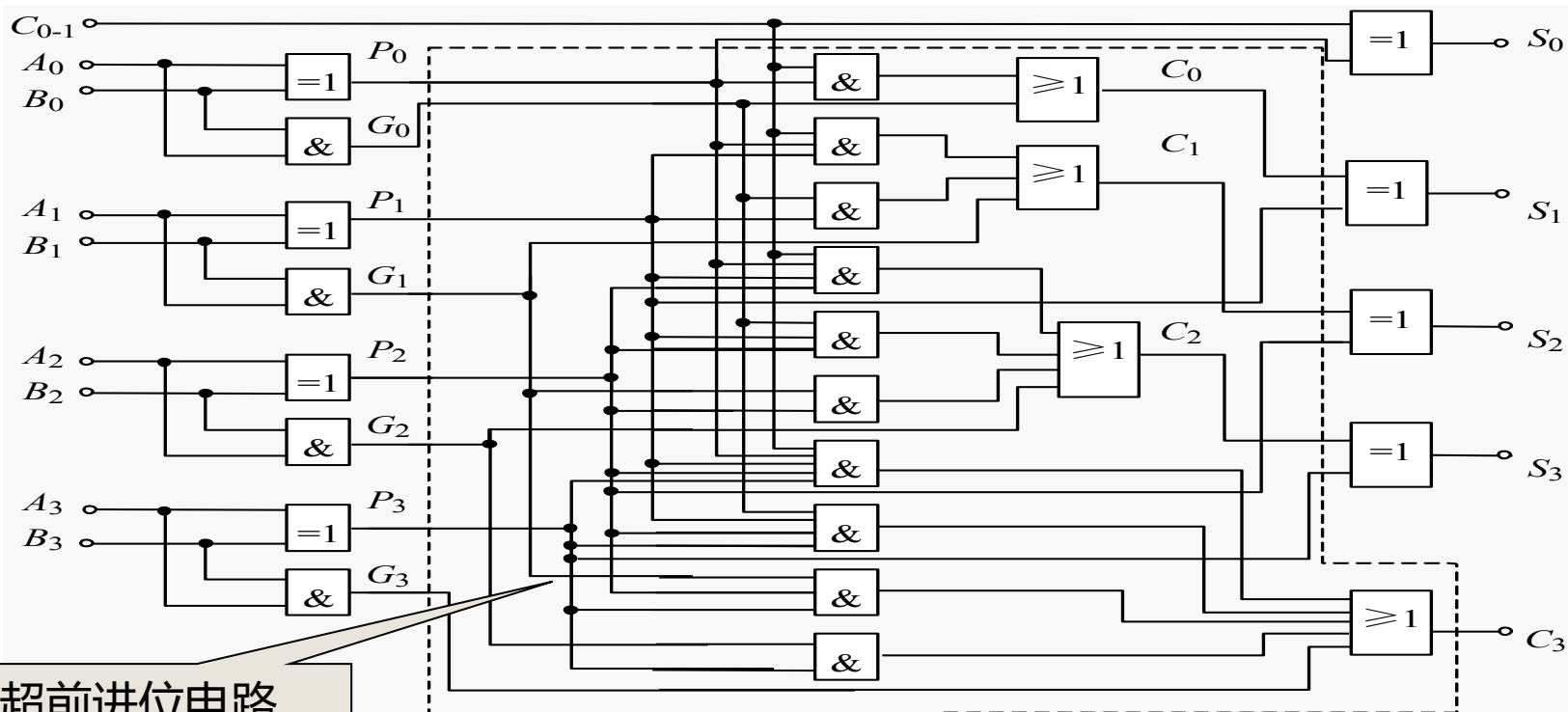
4位超前进位加法器

$$\begin{cases} S_0 = P_0 \oplus C_{0-1} \\ C_0 = G_0 + P_0 C_{0-1} \end{cases}$$

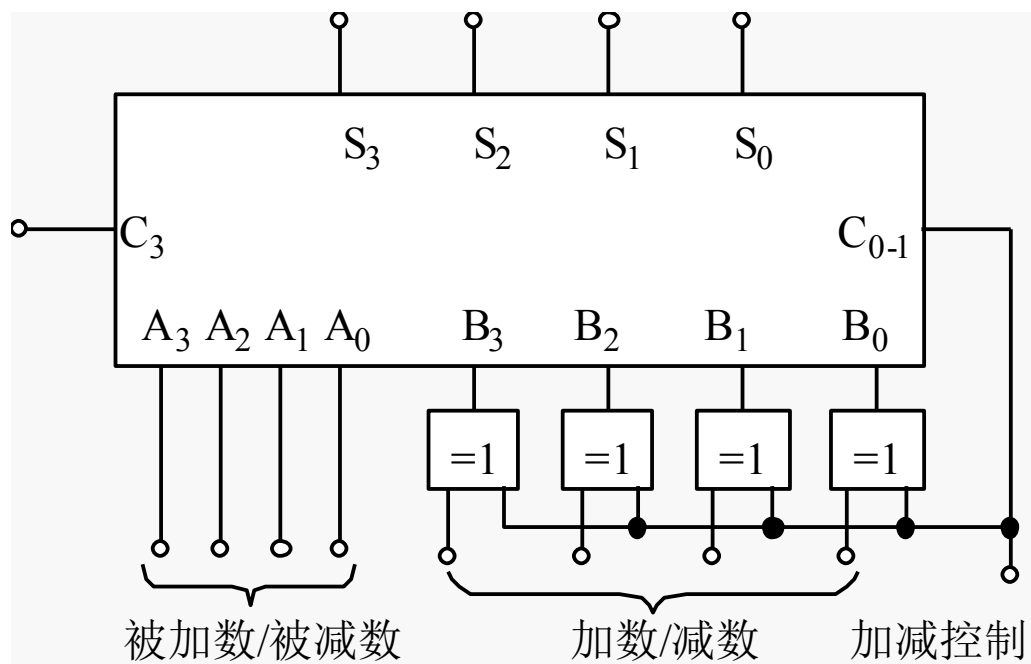
$$\begin{cases} S_1 = P_1 \oplus C_0 \\ C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0 + P_1 P_0 C_{0-1} \end{cases}$$

$$\begin{cases} S_2 = P_2 \oplus C_1 \\ C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{0-1} \end{cases}$$

$$\begin{cases} S_3 = P_3 \oplus C_2 \\ C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{0-1} \end{cases}$$



加减法运算



$C_{0-1} = 0$ 时, $B \oplus 0 = B$, 电路执行 $A + B$ 运算;

$C_{0-1} = 1$ 时, $B \oplus 1 = \overline{B}$, 电路执行 $A - B = A + \overline{B} + 1$ 运算。

译码器

3位二进制译码真值表

- 将二进制代码所表示的信息翻译成对应输出的高低电平信号的过程称为**译码**。实现译码功能的电路称为**译码器 (Decoder)**。
- 地址译码，指令译码，显示译码等应用。

A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

输入：3位二进制代码

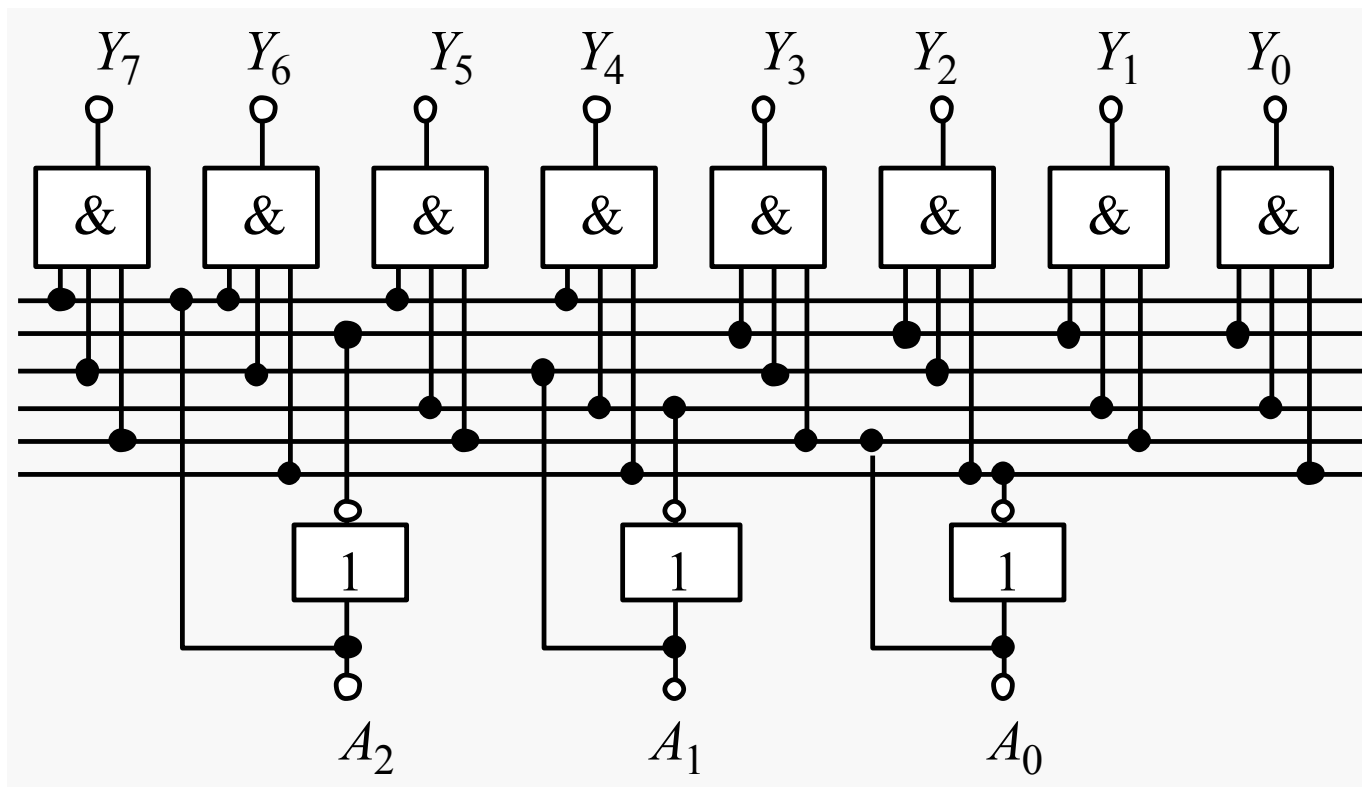
输出：8个互斥的信号

3 线-8 线译码器

逻辑表达式

$$\begin{cases} Y_0 = \bar{A}_2 \bar{A}_1 \bar{A}_0 \\ Y_1 = \bar{A}_2 \bar{A}_1 A_0 \\ Y_2 = \bar{A}_2 A_1 \bar{A}_0 \\ Y_3 = \bar{A}_2 A_1 A_0 \\ Y_4 = A_2 \bar{A}_1 \bar{A}_0 \\ Y_5 = A_2 \bar{A}_1 A_0 \\ Y_6 = A_2 A_1 \bar{A}_0 \\ Y_7 = A_2 A_1 A_0 \end{cases}$$

逻辑图

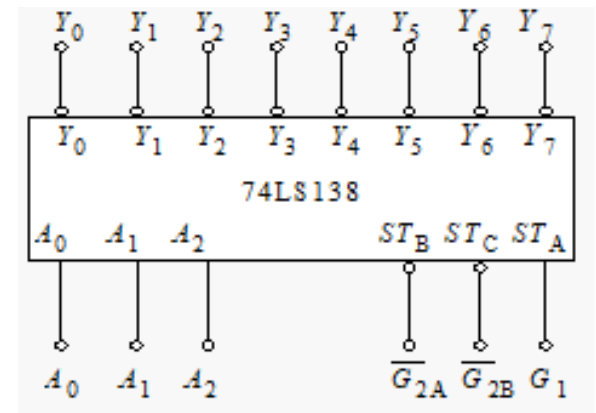


电路特点：与门组成的阵列

集成3线-8线译码器

$$\overline{G}_2 = \overline{G}_{2A} + \overline{G}_{2B}$$

输 入		输 出										
使 能	选 择											
G_1	G_2	A_2	A_1	A_0	$\overline{Y}_7$	$\overline{Y}_6$	$\overline{Y}_5$	$\overline{Y}_4$	$\overline{Y}_3$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$
×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	1	1	1	1	1	1	0	1	1	1
1	0	1	0	0	1	1	1	0	1	1	1	1
1	0	1	0	1	1	1	0	1	1	1	1	1
1	0	1	1	0	1	0	1	1	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1	1	1



输入：自然二进制码

输出：低电平有效

数据选择器

- 在多路数据传送过程中，能够根据需要将其任一路挑选出来的电路，叫做数据选择器，或多路选择器（MUX）。
- 多路选择器可应用于数据通路，包括ALU中操作数的选择，寄存器选择输入数据来源等。

4选1数据选择器

真值表

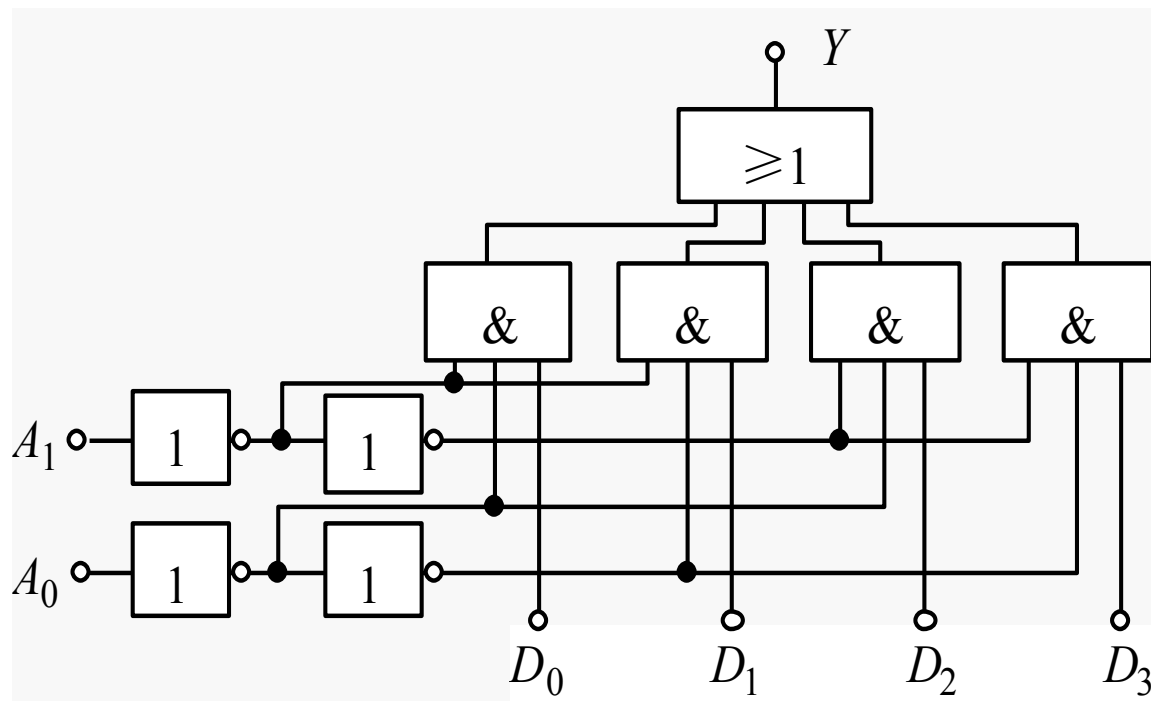
输入数据	输入			输出
	D	A_1	A_0	Y
	D_0	0	0	D_0
	D_1	0	1	D_1
	D_2	1	0	D_2
	D_3	1	1	D_3

地址变量

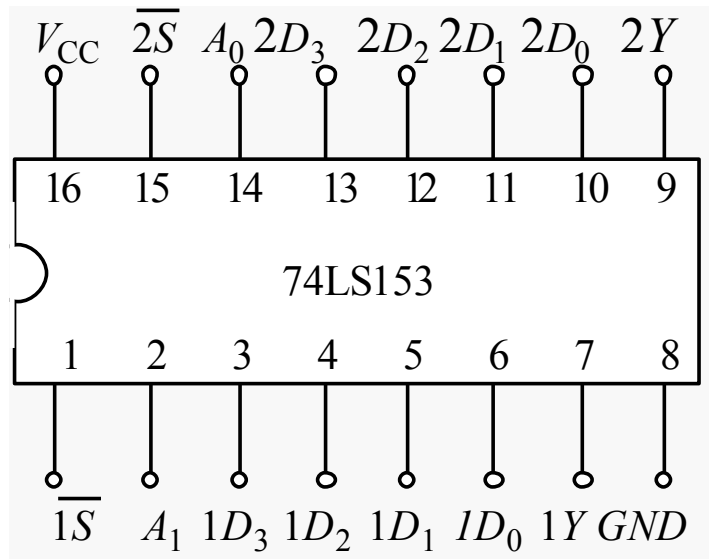
由地址码决定从4路输入中选择哪1路输出。

$$Y = D_0 \bar{A}_1 \bar{A}_0 + D_1 \bar{A}_1 A_0 + D_2 A_1 \bar{A}_0 + D_3 A_1 A_0 = \sum_{i=0}^3 D_i m_i$$

逻辑图



集成双4选1数据选择器74LS153



输 入				输 出
S	D	A_1	A_0	Y
1	×	×	×	0
0	D_0	0	0	D_0
0	D_1	0	1	D_1
0	D_2	1	0	D_2
0	D_3	1	1	D_3

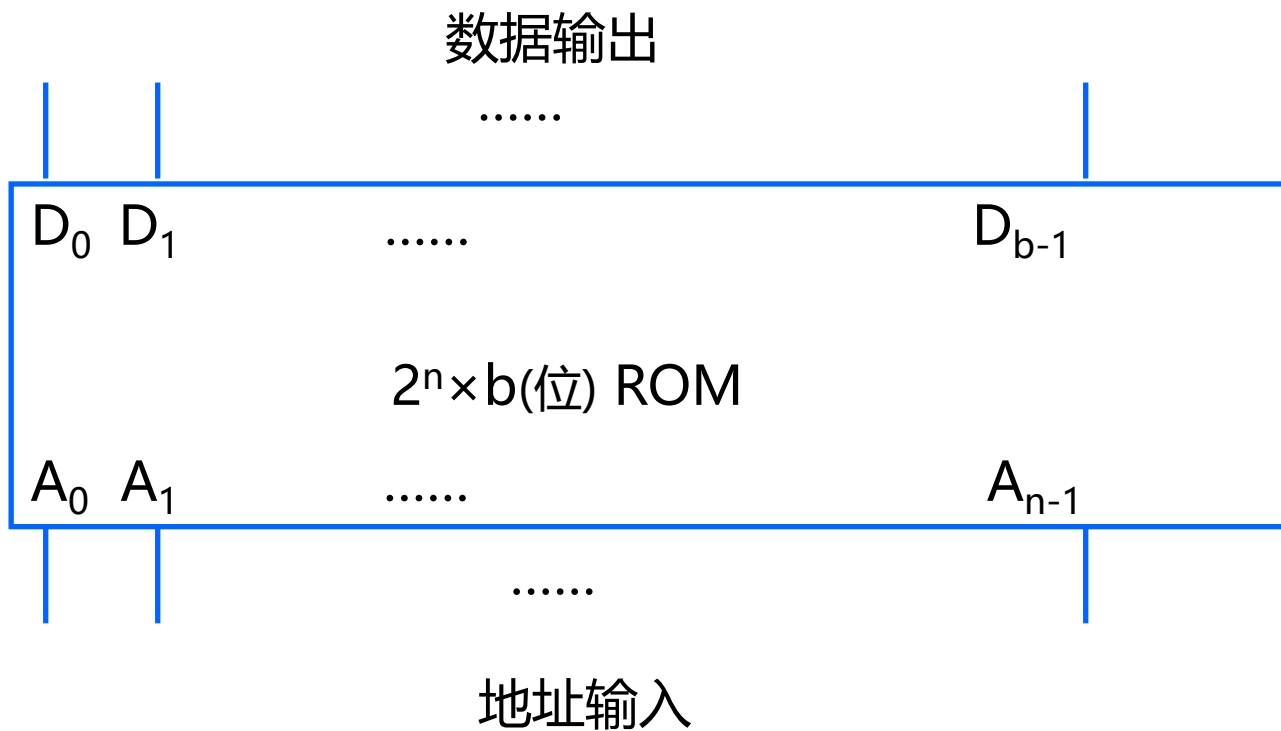
选通控制端S为低电平有效，即S=0时芯片被选中，处于工作状态；
S=1时芯片被禁止，Y≡0。

存储器

存储器的分类：

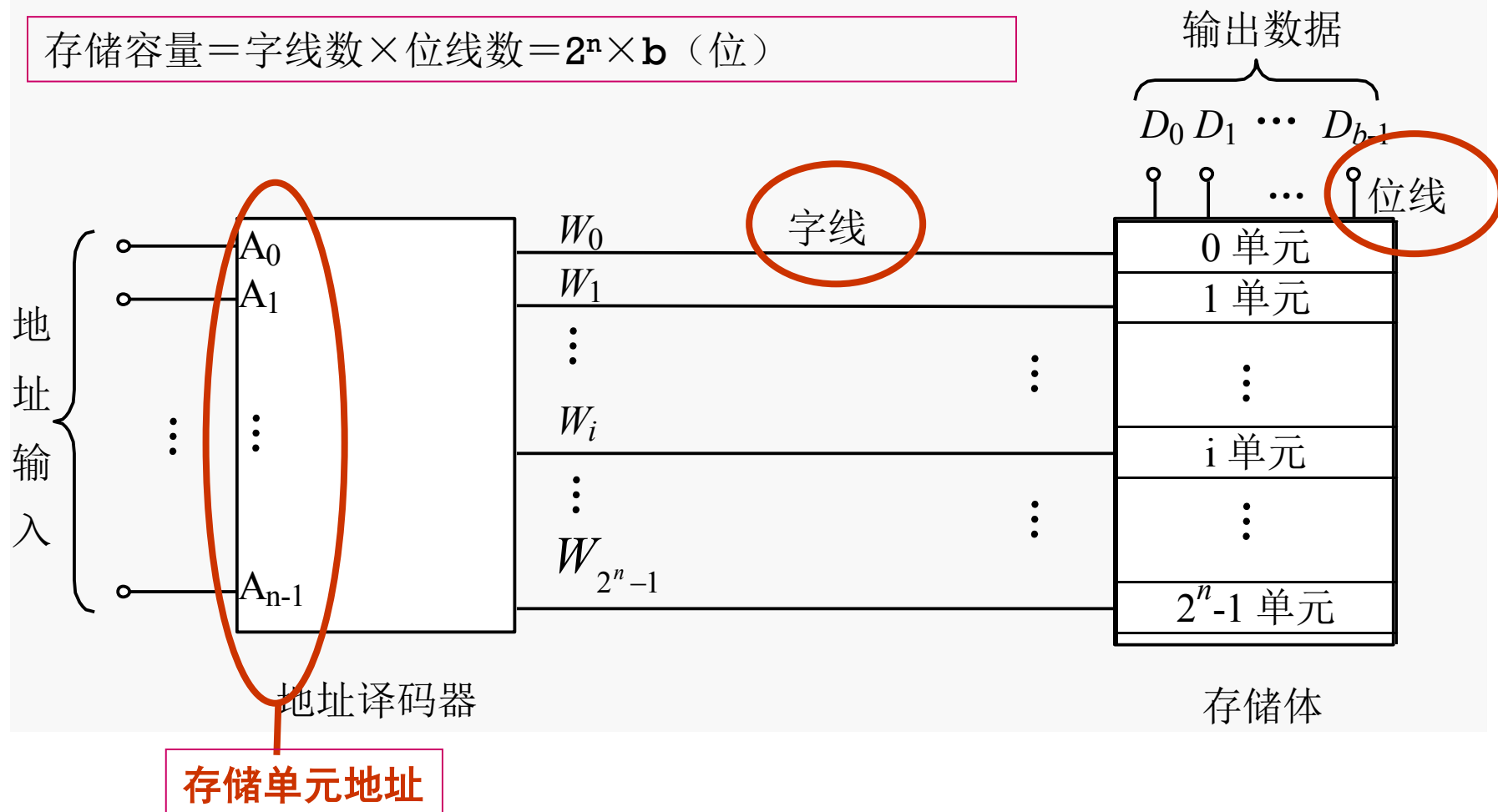
- RAM：在工作时既能从中读出（取出）信息，又能随时写入（存入）信息，但断电后所存信息消失。
- ROM：在工作时只能从中读出信息，不能写入信息，且断电后其所存信息仍能保持。
 - 掩膜ROM：不能改写。
 - PROM：只能改写一次。
 - EPROM：可以改写多次。

ROM的基本结构



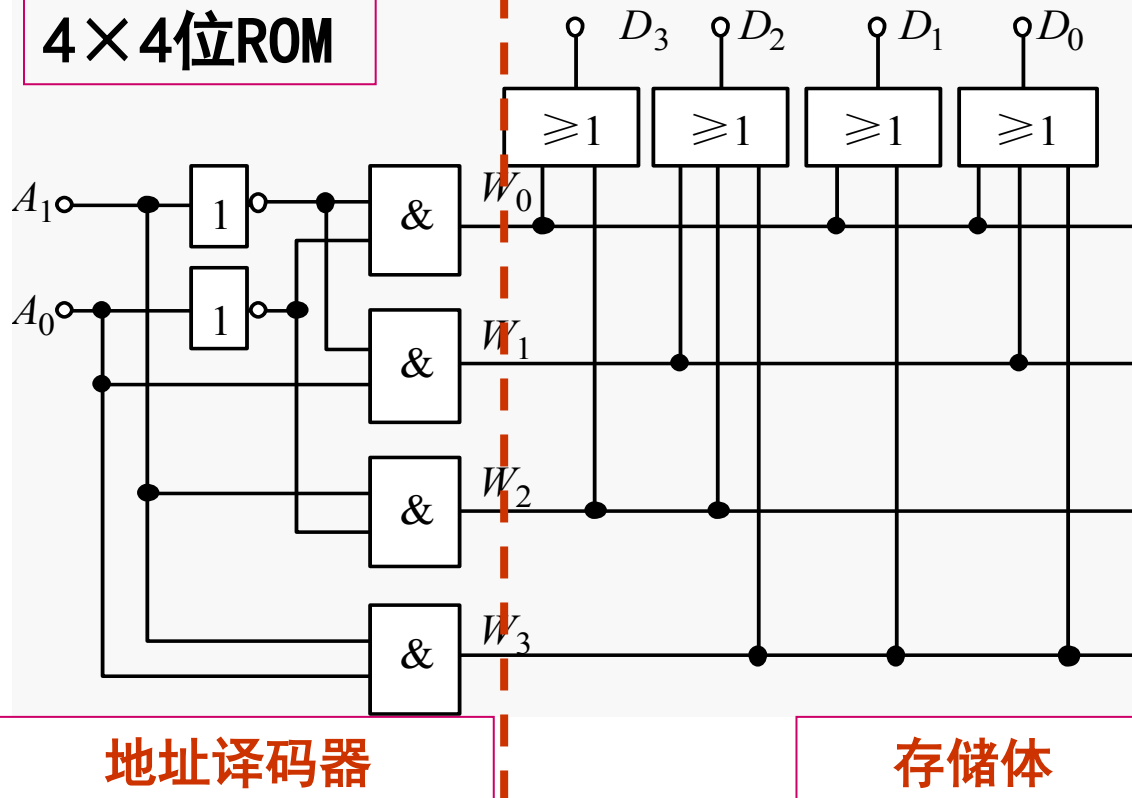
ROM内部结构示意图

存储容量 = 字线数 $\times$ 位线数 = $2^n \times b$ (位)



ROM的基本工作原理

4×4位ROM



$$W_0 = m_0 = \bar{A}_1 \bar{A}_0$$

$$W_1 = m_1 = \bar{A}_1 A_0$$

$$W_2 = m_2 = A_1 \bar{A}_0$$

$$W_3 = m_3 = A_1 A_0$$

$$D_3 = W_0 + W_2 = m_0 + m_2$$

$$D_1 = W_0 + W_3 = m_0 + m_3$$

$$D_2 = W_1 + W_2 + W_3 = m_1 + m_2 + m_3$$

$$D_0 = W_0 + W_1 + W_3 = m_0 + m_1 + m_3$$

存储内容

地 址		字 线				存 储 内 容			
A_1	A_0	W_0	W_1	W_2	W_3	D_3	D_2	D_1	D_0
0	0	1	0	0	0	1	0	1	1
0	1	0	1	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0	0
1	1	0	0	0	1	0	1	1	1

$$D_3 = W_0 + W_2 = m_0 + m_2$$

$$D_2 = W_1 + W_2 + W_3 = m_1 + m_2 + m_3$$

$$D_1 = W_0 + W_3 = m_0 + m_3$$

$$D_0 = W_0 + W_1 + W_3 = m_0 + m_1 + m_3$$

对于给定的地址，相应一条字线输出高电平，与该字线相连接的或门输出为1，未连接的或门输出为0。

ROM应用举例：用ROM作函数运算表

例：用ROM构成能实现函数 $y = x^2$ 的运算表电路。

设 x 的取值范围为0 ~ 15的正整数，则对应的是4位二进制正整数，用 $B = B_3B_2B_1B_0$ 表示。

根据 $y = x^2$ 可算出 y 的最大值是 $15^2 = 225$ ，可以用8位二进制数 $Y = Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$ 表示。

由此可列出 $Y = B^2$ 即 $y = x^2$ 的真值表。

真值表

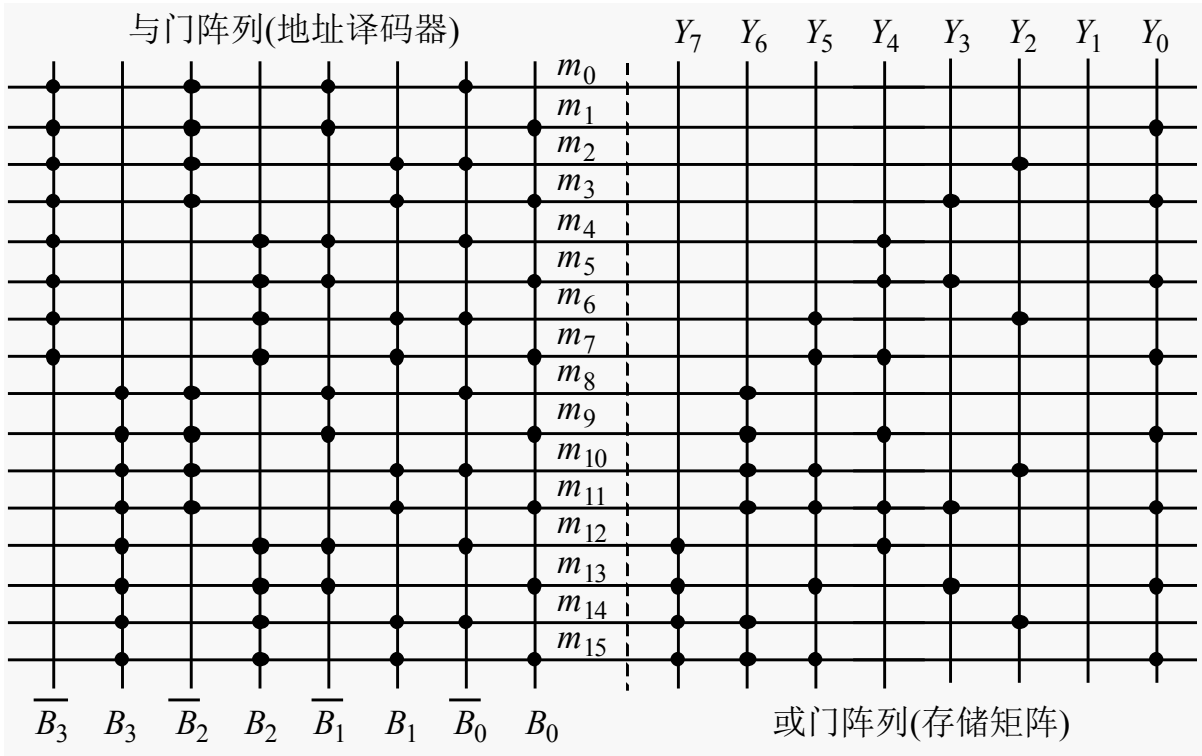
逻辑表达式

$$\begin{cases} Y_7 = \sum m(12,13,14,15) \\ Y_6 = \sum m(8,9,10,11,14,15) \\ Y_5 = \sum m(6,7,10,11,13,15) \\ Y_4 = \sum m(4,5,7,9,11,12) \\ Y_3 = \sum m(3,5,11,13) \\ Y_2 = \sum m(2,6,10,14) \\ Y_1 = 0 \\ Y_0 = \sum m(1,3,5,7,9,11,13,15) \end{cases}$$

输 入				输 出								注
B_3	B_2	B_1	B_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	十进制数
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	0	4
0	0	1	1	0	0	0	0	1	0	0	1	9
0	1	0	0	0	0	0	1	0	0	0	0	16
0	1	0	1	0	0	0	1	1	0	0	1	25
0	1	1	0	0	0	1	0	0	1	0	0	36
0	1	1	1	0	0	1	1	0	0	0	1	49
1	0	0	0	0	1	0	0	0	0	0	0	64
1	0	0	1	0	1	0	1	0	0	0	1	81
1	0	1	0	0	1	1	0	0	1	0	0	100
1	0	1	1	0	1	1	1	1	0	0	1	121
1	1	0	0	1	0	0	1	0	0	0	0	144
1	1	0	1	1	0	1	0	1	0	0	1	169
1	1	1	0	1	1	0	0	0	1	0	0	196
1	1	1	1	1	1	1	0	0	0	0	1	225

阵列图

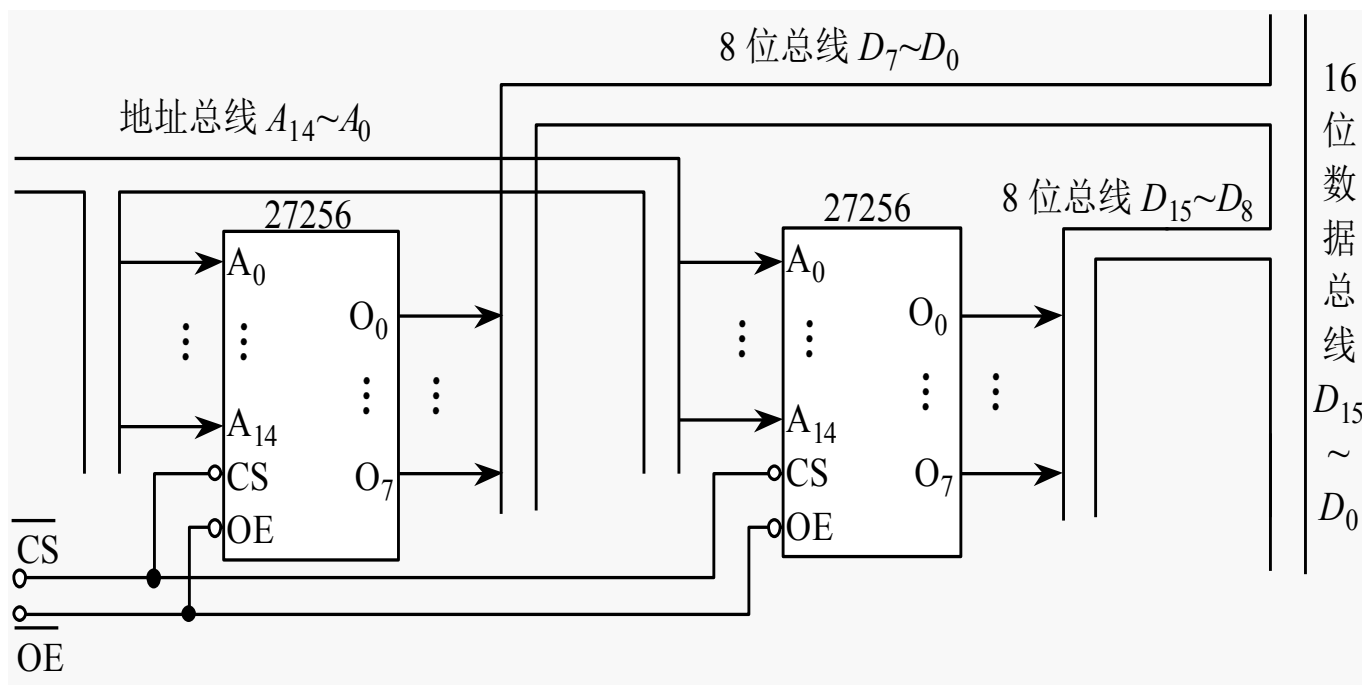
$$\left\{ \begin{array}{l} Y_7 = \sum m(12,13,14,15) \\ Y_6 = \sum m(8,9,10,11,14,15) \\ Y_5 = \sum m(6,7,10,11,13,15) \\ Y_4 = \sum m(4,5,7,9,11,12) \\ Y_3 = \sum m(3,5,11,13) \\ Y_2 = \sum m(2,6,10,14) \\ Y_1 = 0 \\ Y_0 = \sum m(1,3,5,7,9,11,13,15) \end{array} \right.$$



ROM的容量扩展

1、位扩展（字长的扩展）

用两片27256（32k × 8）扩展成32k × 16位EPROM

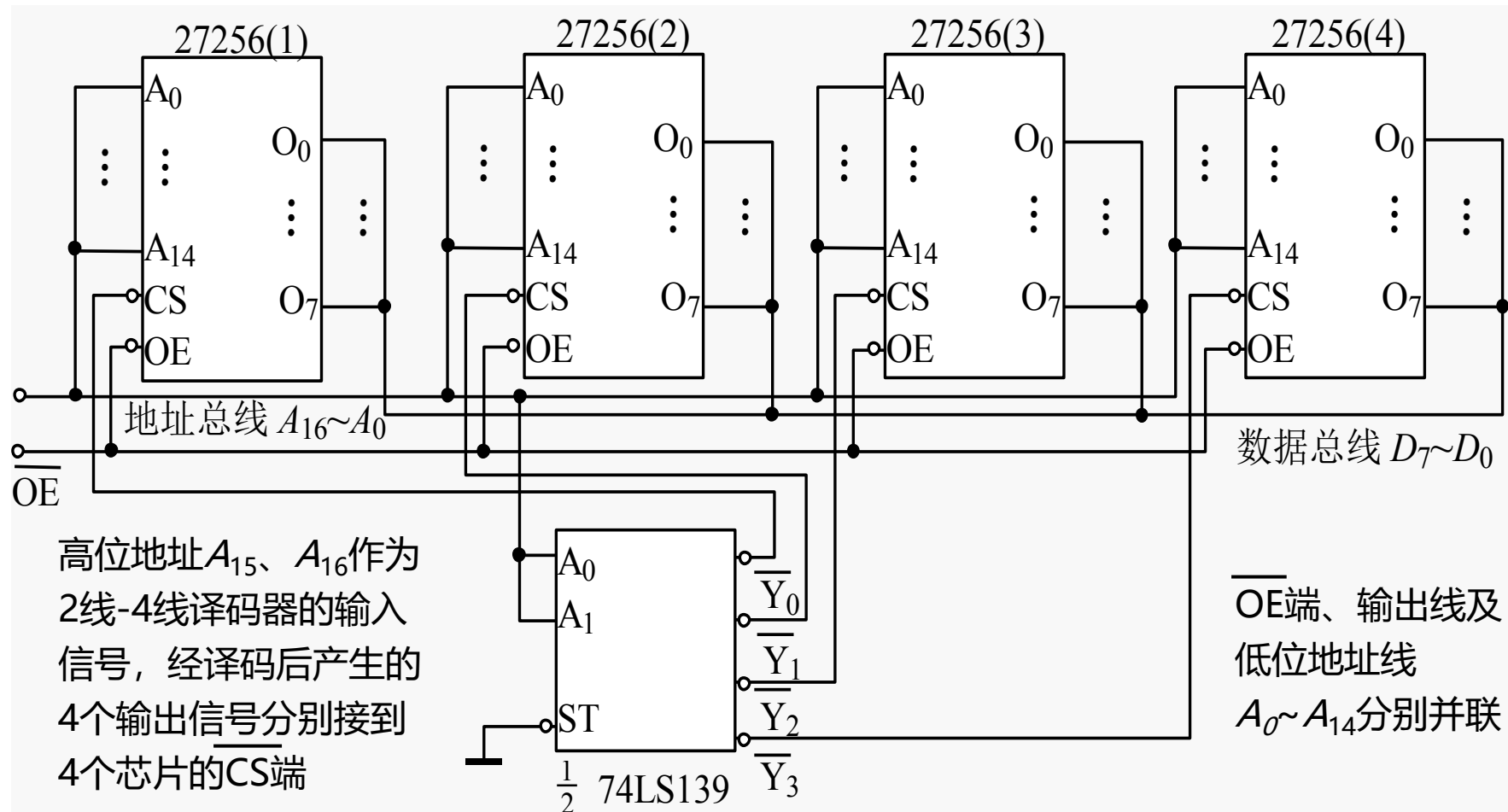


地址线及控制线分别并联

输出一个作为高8位，另一个作为低8位

2、字扩展（字数扩展，地址码扩展）

用4片27256扩展成 $4 \times 32k \times 8$ 位EPROM



可靠性代码

可靠性代码：代码传输时不易出错，或出现错误时容易被发现并进行校正，常见的有格雷码、奇偶校验码等。

1. 格雷码

格雷码：任何两个相邻代码之间只有一位码元不同，且0和最大数对应的格雷码之间也只有一位不同，首尾相连。

$$G_i = B_i \oplus B_{i+1}$$

典型4位格雷码 4位自然二进制码

	G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	1	0	0	1	0
3	0	0	1	0	0	0	1	1
4	0	1	1	0	0	1	0	0
5	0	1	1	1	0	1	0	1
6	0	1	0	1	0	1	1	0
7	0	1	0	0	0	1	1	1
8	1	1	0	0	1	0	0	0
9	1	1	0	1	1	0	0	1
10	1	1	1	1	1	0	1	0
11	1	1	1	0	1	0	1	1
12	1	0	1	0	1	1	0	0
13	1	0	1	1	1	1	0	1
14	1	0	0	1	1	1	1	0
15	1	0	0	0	1	1	1	1

2. 奇偶校验码

◆ **奇偶校验码(Parity Check)**: 一种校验代码传输正确性的方法, 由两部分组成:

- ① 需要传输的信息本身, n 位二进制代码。
- ② 1位奇偶校验位: 增加1位0或1使二进制代码中1的个数为奇数或偶性。
 - 奇校验: 1的个数为奇数
 - 偶校验: 1的个数为偶数

特点:

- ① 只能发现奇数个数传输错误, 没有错误定位和纠错能力, 发现错误后, 只能重传。
- ② 实现简单, 广泛应用。

十 进 制 数	8421 奇校验码			8421 偶校验码		
	信 息 码	校验位		信 息 码	校验位	
0	0 0 0 0	1		0 0 0 0	0	
1	0 0 0 1	0		0 0 0 1	1	
2	0 0 1 0	0		0 0 1 0	1	
3	0 0 1 1	1		0 0 1 1	0	
4	0 1 0 0	0		0 1 0 0	1	
5	0 1 0 1	1		0 1 0 1	0	
6	0 1 1 0	1		0 1 1 0	0	
7	0 1 1 1	0		0 1 1 1	1	
8	1 0 0 0	0		1 0 0 0	1	
9	1 0 0 1	1		1 0 0 1	0	

改进的奇偶校验码

双向偶校验:

垂直水平两个维度校验

0110100	1
1011010	0
0010110	1
1110101	1
1001011	0
1000110	1

0110100	1
1011010	0
0010111	1
1110101	1
1001011	0
1000110	1

(可纠错)

可纠正1位错误

0110100	1
1011010	0
0110100	1
1110101	1
1001011	0
1000110	1

(可检错)

可检出一部分偶数位错误

0110100	1
1011010	0
0110110	1
1110101	1
1001001	0
1000110	1

(可检错)

可检测出某行(列)上的奇数位

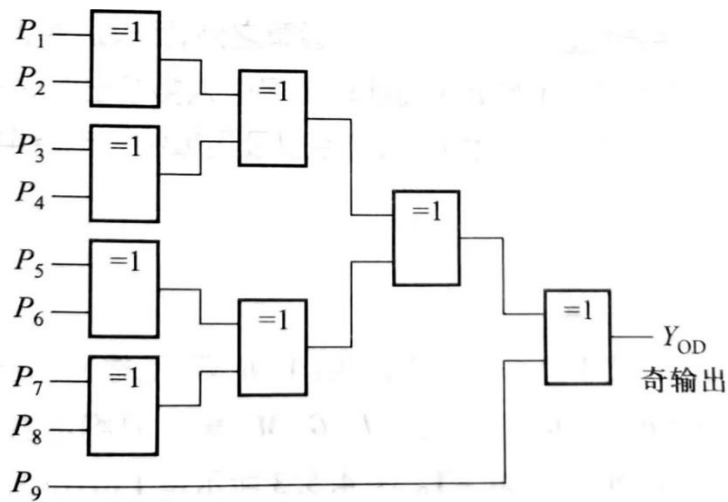
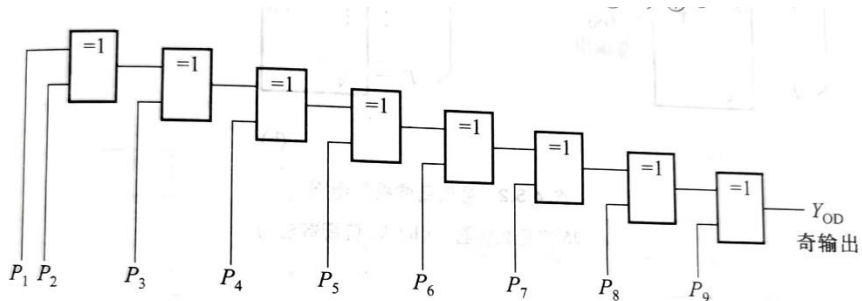
1110101	1
1011010	0
0010110	1
1110101	1
1001011	0
0000111	1

(不能检错)

不能检测出错误码分布在矩形4个顶点上的错误

奇偶检验电路

$$Y_{OD} = P_1 \oplus P_2 \oplus P_3 \oplus P_4 \oplus P_5 \oplus P_6 \oplus P_7 \oplus P_8 \oplus P_9$$

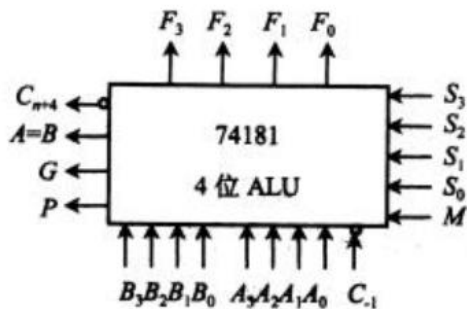


异或运算满足交换律和结合律，两种电路功能上等价

算术逻辑单元（ALU）

- 算术逻辑单元：简称ALU，是一种功能较强的组合逻辑电路，能进行多种算术运算和逻辑运算。
- ALU的一个基本逻辑结构是超前进位加法器。

SN74181型四位ALU功能表中



- 能执行16种算术运算和16种逻辑运算；
- M：状态控制端，当M=H，执行逻辑运算；M=L执行算术运算；
- $S_0 \sim S_3$ ：运算选择控制端，它决定电路执行哪种算术运算或哪种逻辑运算；
- $A_3 \sim A_0$, $B_3 \sim B_0$ ：参加运算的两个数；

SN74181型ALU功能表

功 能 表				
工作方式选择 输入 $S_3S_2S_1S_0$	负逻辑输入或输出		正逻辑输入或输出	
	逻辑运算 ($M=1$)	算术运算 ($M=0$) ($C_{-1}=0$)	逻辑运算 ($M=1$)	算术运算 ($M=0$) ($C_{-1}=1$)
0000	$\overline{A}$	A 减 1	$\overline{A}$	A
0001	$\overline{AB}$	AB 减 1	$\overline{A+B}$	A+B
0010	$\overline{A+B}$	$A\overline{B}$ 减 1	$\overline{AB}$	$A+\overline{B}$
0011	逻辑 1	减 1	逻辑 0	减 1
0100	$\overline{A+B}$	A 加 ($A+\overline{B}$)	$\overline{AB}$	A 加 $A\overline{B}$
0101	$\overline{B}$	AB 加 ($A+\overline{B}$)	$\overline{B}$	(A+B) 加 $A\overline{B}$
0110	$\overline{A\oplus B}$	A 减 B 减 1	$A\oplus B$	A 减 B 减 1
0111	$A+\overline{B}$	$A+\overline{B}$	$A\overline{B}$	$A\overline{B}$ 减 1
1000	$\overline{AB}$	A 加 ($A+B$)	$\overline{A+B}$	A 加 AB
1001	$A\oplus B$	A 加 B	$\overline{A\oplus B}$	A 加 B
1010	B	$A\overline{B}$ 加 ($A+B$)	B	($A+\overline{B}$) 加 AB
1011	A+B	A+B	AB	AB 减 1
1100	逻辑 0	A 加 A^*	逻辑 1	A 加 A^*
1101	$A\overline{B}$	AB 加 A	$A+\overline{B}$	(A+B) 加 A
1110	AB	$A\overline{B}$ 加 A	A+B	($A+\overline{B}$) 加 A
1111	A	A	A	A 减 1

(1) 1=高电平; 0=低电平; (2) *表示每一位均移到下一个更高位, 即 $A^*=2A$

实验要求实现的ALU

指令名称	Op码 [5:0]	指令功能	补充说明
ADD	100000	有符号数相加	使用超前进位加法器，注意根据结果置Over标志。
ADDU	100001	无符号数相加	使用超前进位加法器，Over标志恒为0。
SUB	100010	有符号数相减	使用超前进位加法器，注意根据结果置Over标志。
SUBU	100011	无符号数相减	使用超前进位加法器，Over标志恒为0。
SLL	000000	逻辑左移	A[4:0]为移位位数，在低位补零，Over标志恒为0。
SRL	000010	逻辑右移	A[4:0]为移位位数，在高位补零，Over标志恒为0。
SRA	000011	算术右移	A[4:0]为移位位数，Over标志恒为0。 若B操作数最高位为0，则高位补零，否则高位补1。
AND	100100	位与	Over标志恒为0。
OR	100101	位或	Over标志恒为0。
XOR	100110	位异或	Over标志恒为0。
NOR	100111	位或非	Over标志恒为0。

2.2.2 时序逻辑电路



时序电路: 在任何时刻的稳定输出，不仅与该时刻的输入信号有关，而且还与电路原来的状态有关。

触发器概述

在数字电路中，基本的工作信号只有0和1两种取值，作为存放二值逻辑信号的单元电路——**触发器**，对它的基本要求，即触发器的**基本特征**是：

- 有两个稳定的状态：0状态和1状态；
- 在不同的输入情况下，它可以被置成0状态或1状态；
- 当输入信号消失后，所置成的状态能够保持不变。

触发器的现态和次态：触发器接收输入信号之前的状态叫做现态，用 Q^n 表示。

触发器接收输入信号之后的状态叫次态，用 Q^{n+1} 表示。现态和次态是两个相邻离散时间里触发器输出端的状态。

触发器的类型

● 根据逻辑功能不同分为

RS 触发器

D 触发器

JK 触发器

T 触发器

T' 触发器

● 根据触发方式不同分为

电平触发器

边沿触发器

● 根据电路结构不同分为

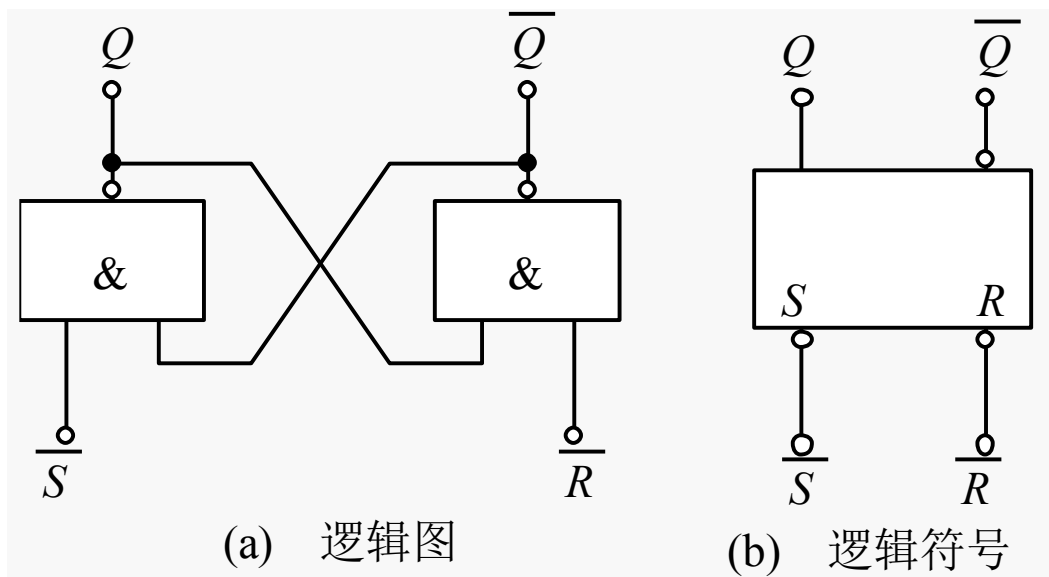
基本 RS 触发器

同步触发器

边沿触发器

用与非门组成的基本RS触发器

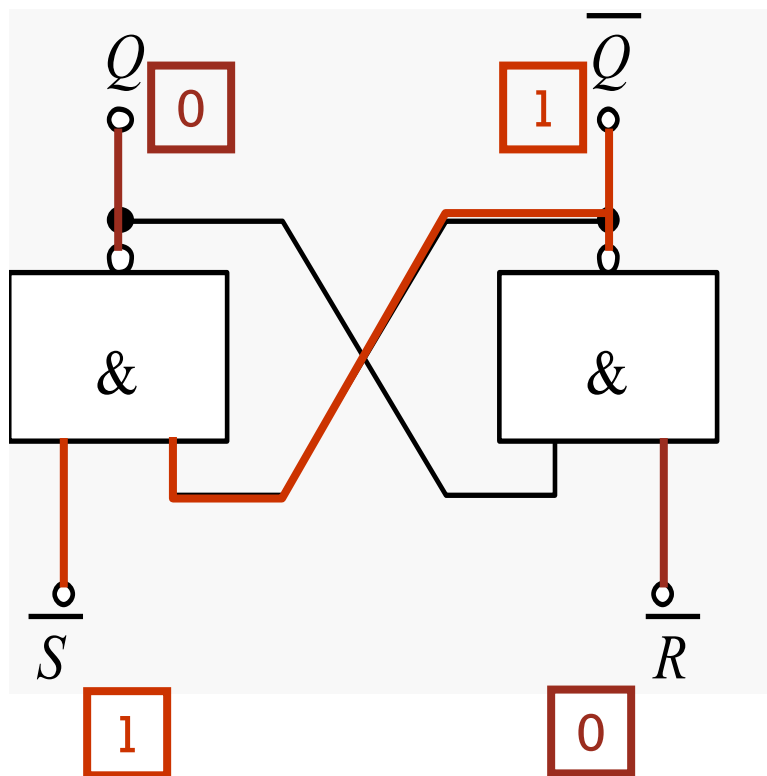
电路组成和逻辑符号



信号输入端，低电平有效。

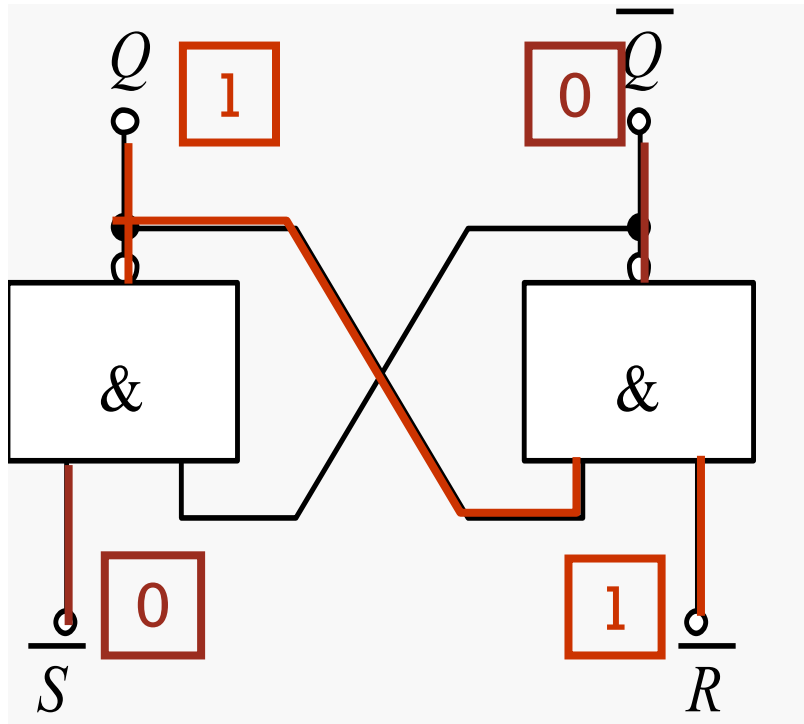
信号输出端， $Q=0$ 、 $\overline{Q}=1$ 的状态称0状态， $Q=1$ 、 $\overline{Q}=0$ 的状态称1状态。

工作原理



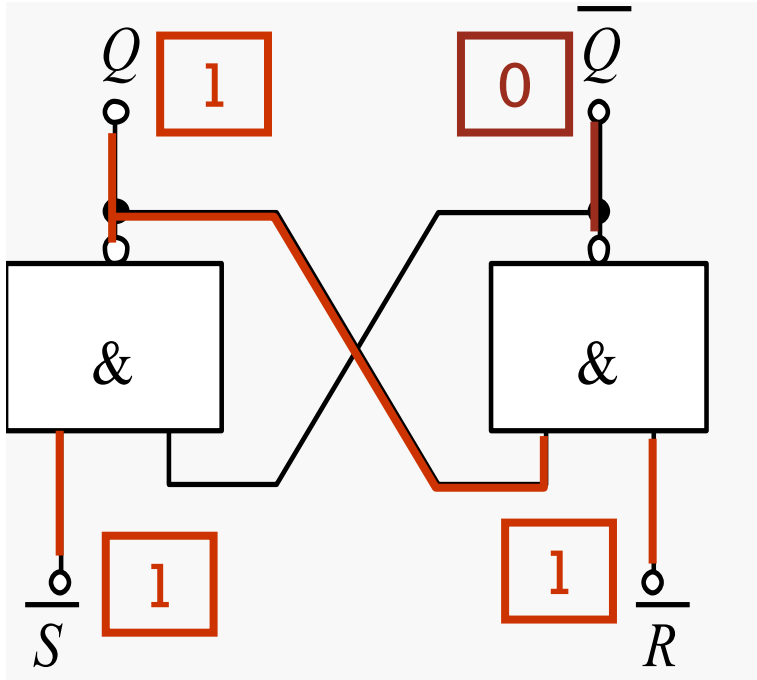
$\bar{R}$	$\bar{S}$	Q
0	1	0

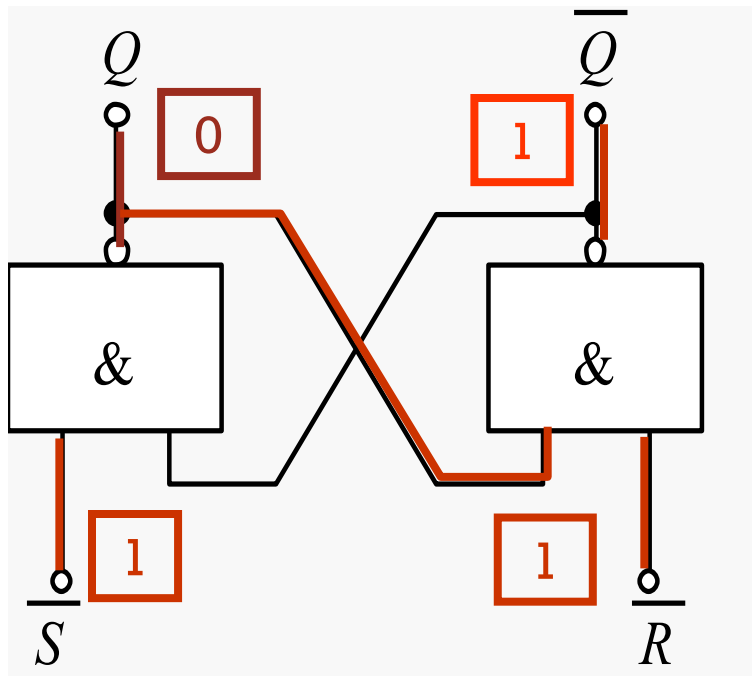
① $\bar{R} = 0$ 、 $\bar{S} = 1$ 时：由于 $\bar{R} = 0$ ，不论原来 Q 为0还是1，都有 $\bar{Q} = 1$ ；再由 $\bar{S} = 1$ 、 $\bar{Q} = 1$ 可得 $Q = 0$ 。即不论触发器原来处于什么状态都将变成0状态，这种情况称将触发器置0或复位。 $\bar{R}$ 端称为触发器的置0端或复位端(即Reset)。



$\bar{R}$	$\bar{S}$	Q
0	1	0
1	0	1

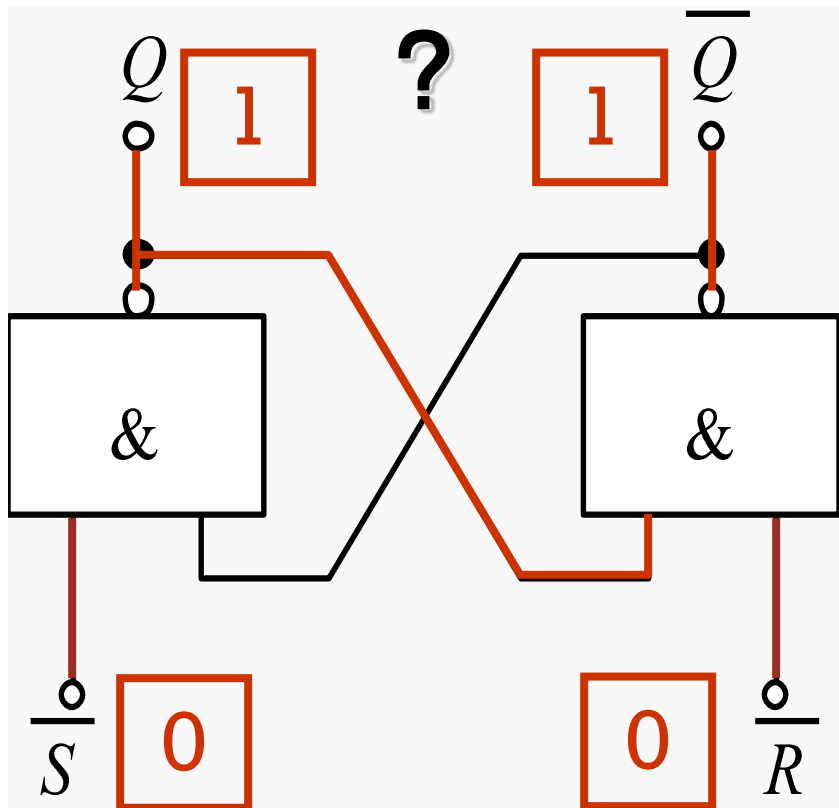
② $\bar{R} = 1$ 、 $\bar{S} = 0$ 时：由于 $\bar{S} = 0$ ，不论原来 Q 为0还是1，都有 $Q = 1$ ；再由 $\bar{R} = 1$ 、 $Q = 1$ 可得 $\bar{Q} = 0$ 。即不论触发器原来处于什么状态都将变成1状态，这种情况称将触发器置1或置位。 $\bar{S}$ 端称为触发器的置1端或置位端(即Set)。





$\overline{R}$	$\overline{S}$	Q
0	1	0
1	0	1
1	1	不变

③ $\overline{R} = 1$ 、 $\overline{S} = 1$ 时：根据与非门的逻辑功能不难推知，触发器保持原有状态不变，即原来的状态被触发器存储起来，这体现了触发器具有记忆能力。



$\bar{R}$	$\bar{S}$	Q
0	1	0
1	0	1
1	1	不变
0	0	不允许

④ $\bar{R} = 0$ 、 $\bar{S} = 0$ 时： $\bar{Q} = Q = 1$ ，不符合触发器的逻辑关系。并且由于与非门延迟时间不可能完全相等，在两输入端的0同时撤除后，将不能确定触发器是处于1状态还是0状态。所以触发器不允许出现这种情况，这就是**基本RS触发器的约束条件**。

特性表（真值表）

现态：触发器接收输入信号之前的状态，也就是触发器原来的稳定状态。

$\bar{R}$	$\bar{S}$	Q^n	Q^{n+1}	功能
0	0	0	不用	不允许
0	0	1	不用	
0	1	0	0	$Q^{n+1} = 0$
0	1	1	0	置 0
1	0	0	1	$Q^{n+1} = 1$
1	0	1	1	置 1
1	1	0	0	$Q^{n+1} = Q^n$
1	1	1	1	保持

次态：触发器接收输入信号之后所处的新的稳定状态。

次态 Q^{n+1} 的卡诺图

$\overline{\overline{RS}}$		Q^n			
		00	01	11	10
0		×	0	0	1
1		×	0	1	1

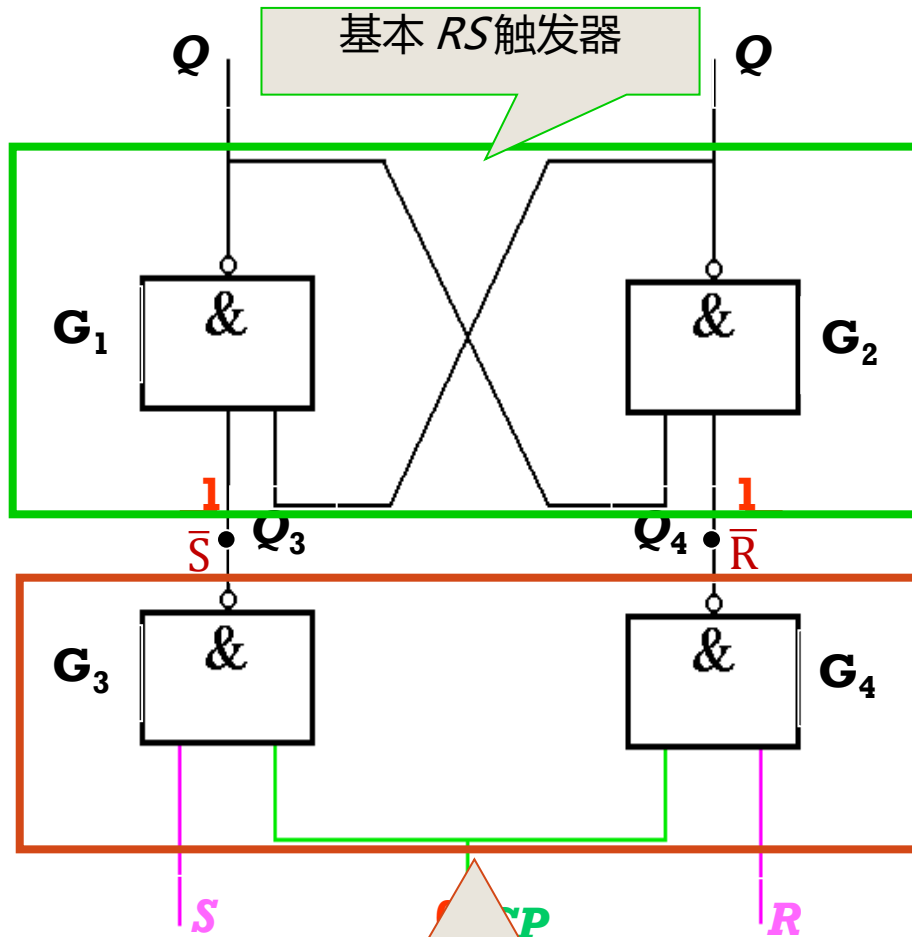
特性方程

$$\begin{cases} Q^{n+1} = \overline{\overline{S}} + \overline{R}Q^n = S + \overline{R}Q^n \\ \overline{\overline{R}}\overline{\overline{S}} = 0 \quad (RS = 0) \end{cases} \quad \text{约束条件}$$

触发器的特性方程：触发器次态 Q^{n+1} 与输入及现态 Q^n 之间的逻辑关系式

同步RS触发器

1. 电路结构与工作原理



增加了由时钟 CP 控制的门 G_3 、 G_4

工作原理

- $CP = 0$ 时, G_3 、 G_4 被封锁, 输入信号 R 、 S 不起作用。基本 RS 触发器的输入均为 1, 触发器状态保持不变。
- $CP = 1$ 时, G_3 、 G_4 解除封锁, 将输入信号 R 和 S 取非后送至基本 RS 触发器的输入端。

特性表

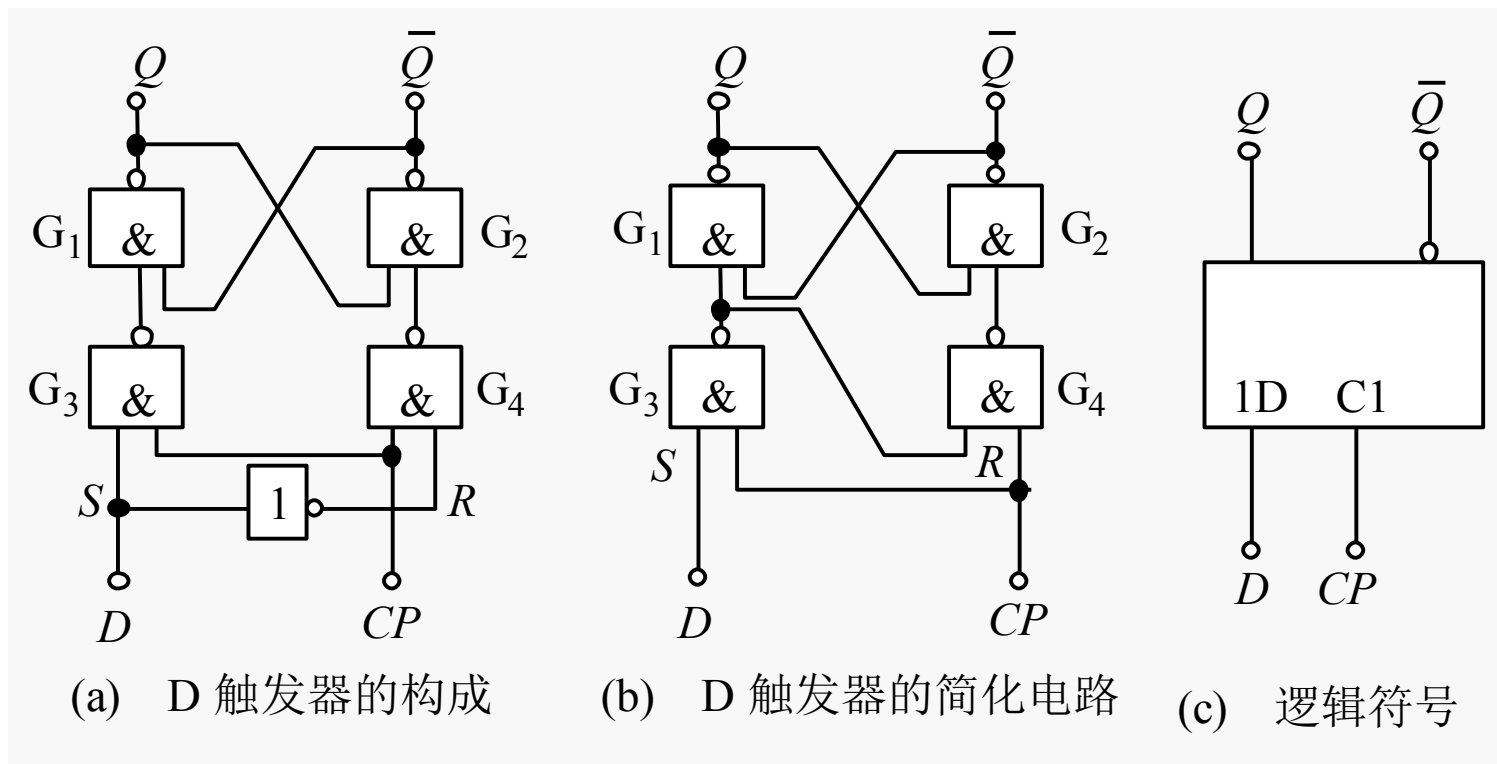
CP	R	S	Q^n	Q^{n+1}	功能
0	×	×	×	Q^n	$Q^{n+1} = Q^n$ 保持
1	0	0	0	0	$Q^{n+1} = Q^n$ 保持
1	0	0	1	1	
1	0	1	0	1	$Q^{n+1} = 1$ 置 1
1	0	1	1	1	
1	1	0	0	0	$Q^{n+1} = 0$ 置 0
1	1	0	1	0	
1	1	1	0	不用	不允许
1	1	1	1	不用	

特性方程

$$\begin{cases} Q^{n+1} = S + \bar{R}Q^n \\ RS = 0 \text{ (RS不能同时为1)} \end{cases}$$

CP=1期间有效

同步D触发器



将 $S=D$ 、 $R=\bar{D}$ 代入同步RS触发器的特性方程，得同步D触发器的特性方程：

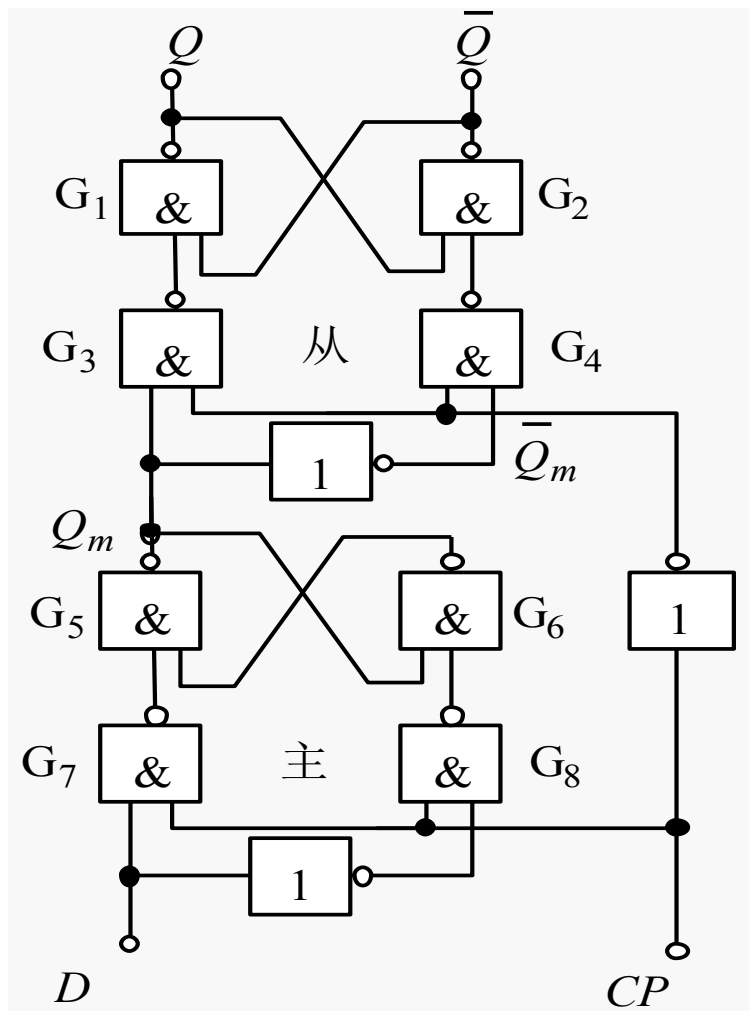
$$Q^{n+1} = S + \bar{R}Q^n = D + \bar{D}Q^n = D \quad CP=1 \text{期间有效}$$

无空翻触发器——边沿触发器

工作特点:

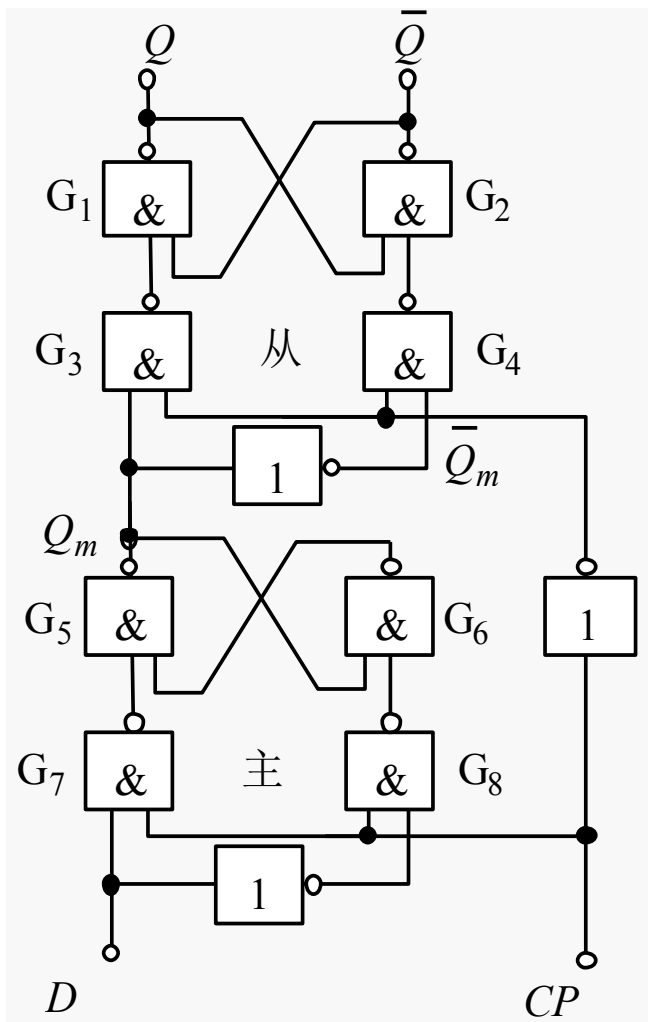
- 只能在 CP 上升沿(或下降沿)时刻接收输入信号
- 电路状态**只能在 CP 上升沿(或下降沿)时刻翻转**
- 克服了空翻, 可靠性和抗干扰能力强, 应用范围广

边沿D触发器



工作原理

- $CP=0$ 时, 门 G_7 、 G_8 被封锁, 门 G_3 、 G_4 打开, 从触发器的状态取决于主触发器: $Q=Q_m$ 、 $\bar{Q}=\bar{Q}_m$, 输入信号 D 不起作用。
- $CP=1$ 时, 门 G_7 、 G_8 打开, 门 G_3 、 G_4 被封锁, 从触发器状态不变, 主触发器的状态跟随输入信号 D 的变化而变化, 即在 $CP=1$ 期间始终都有 $Q_m=D$ 。

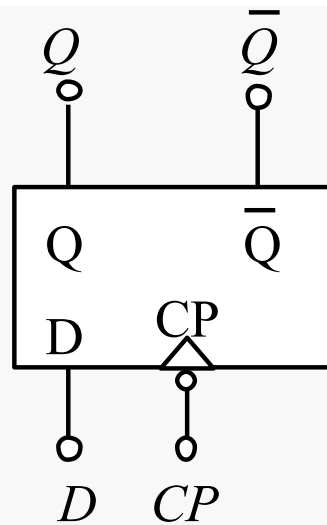


- **CP下降沿到来时**，封锁门G₇、G₈，打开门G₃、G₄，主触发器锁存CP下降时刻D的值，即 $Q_m = D$ ，随后将该值送入从触发器，使 $Q = D$ 、 $\bar{Q} = \bar{D}$ 。
- **CP下降沿过后**，主触发器锁存的CP下降沿时刻D的值被保存下来，而从触发器的状态也将保持不变。

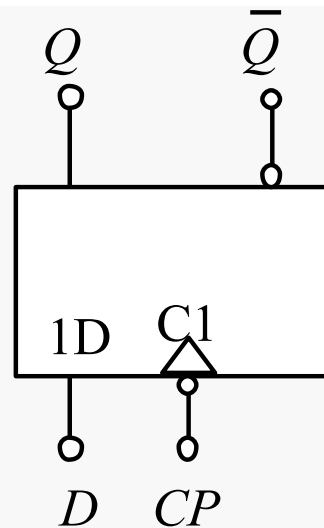
综上所述，边沿D触发器的特性方程为：

$$Q^{n+1} = D \quad \text{下降沿时刻有效}$$

逻辑符号

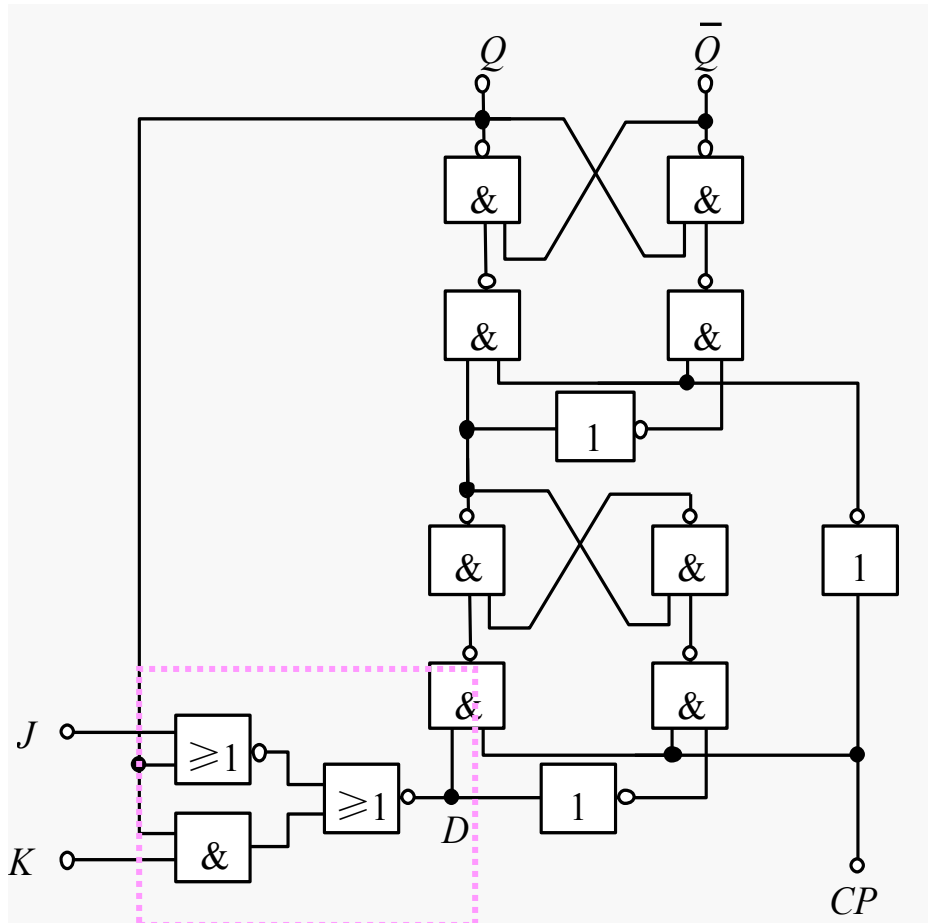


曾用符号



国标符号

边沿JK触发器



$$\begin{aligned}
 D &= \overline{J + Q^n + KQ^n} \\
 &= (J + Q^n) \cdot \overline{KQ^n} \\
 &= (J + Q^n)(\overline{K} + \overline{Q^n}) \\
 &= J\overline{Q^n} + \overline{K}Q^n + J\overline{K} \\
 &= J\overline{Q^n} + \overline{K}Q^n
 \end{aligned}$$

$$\begin{aligned}
 Q^{n+1} &= D \\
 &= J\overline{Q^n} + \overline{K}Q^n \quad CP \text{下降沿时刻有效}
 \end{aligned}$$

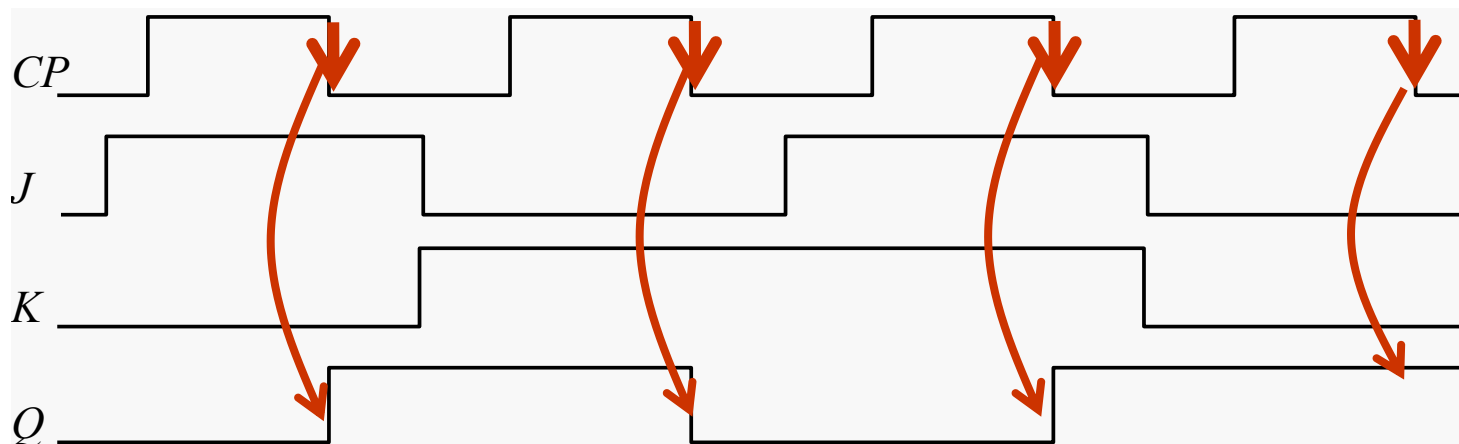
在边沿D触发器的基础上，增加了三个门，并将Q反馈给输入，便构成了边沿JK触发器。

特性表

$$Q^{n+1} = J\bar{Q}^n + \bar{K}Q^n$$

J	K	Q^n	Q^{n+1}	功能
0	0	0	0	$Q^{n+1} = Q^n$ 保持
0	0	1	1	
0	1	0	0	$Q^{n+1} = 0$ 置 0
0	1	1	0	
1	0	0	1	$Q^{n+1} = 1$ 置 1
1	0	1	1	
1	1	0	1	$Q^{n+1} = \bar{Q}^n$ 翻转
1	1	1	0	

时序图



小结

- 触发器是数字电路的极其重要的基本单元。
- 触发器有两个稳定状态，在外界信号作用下，可以从一个稳态转变为另一个稳态；无外界信号作用时状态保持不变。
- 触发器可以作为二进制存储单元使用。
- 各种不同逻辑功能的触发器的特性方程为：

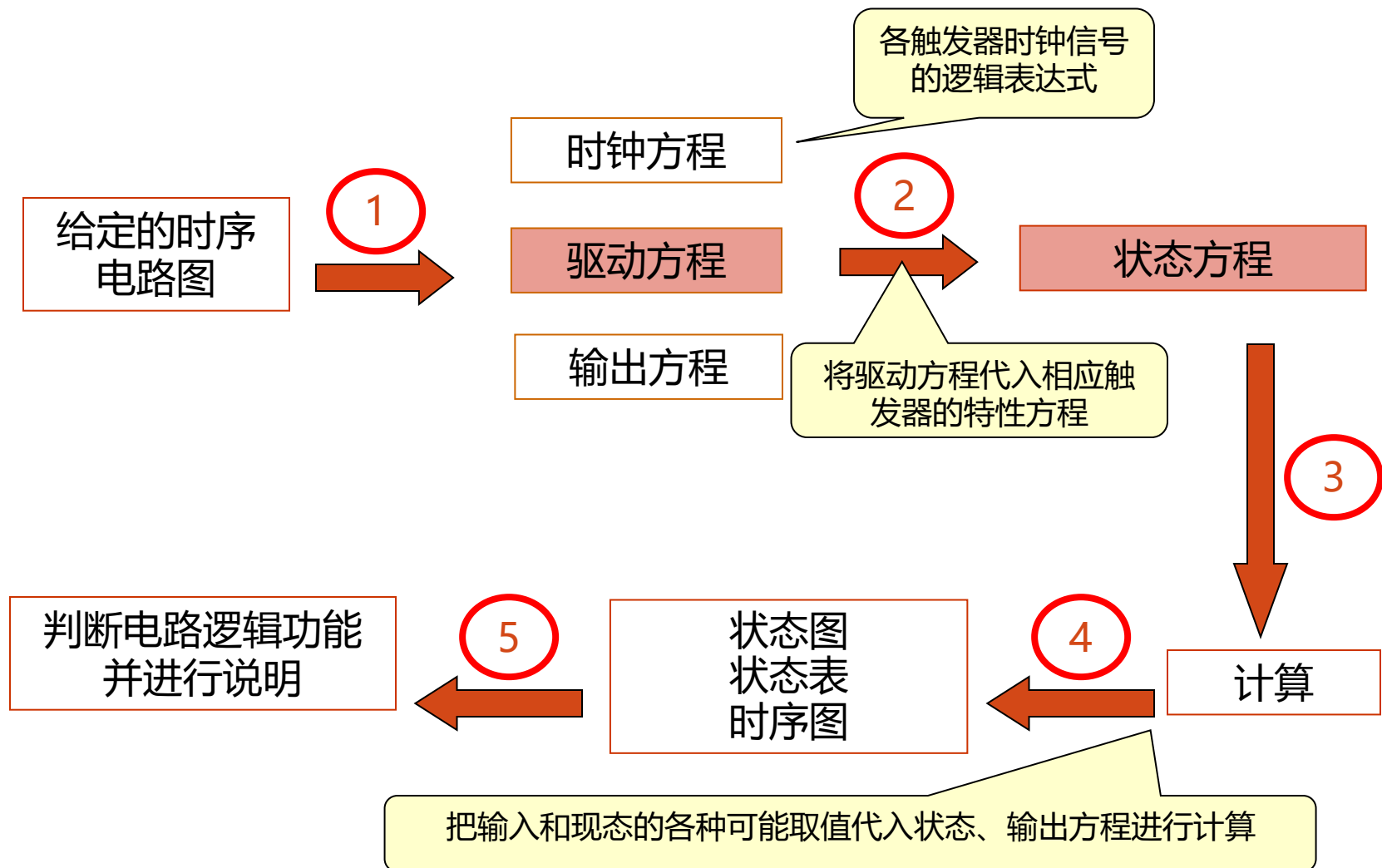
RS触发器： $Q^{n+1} = S + \bar{R}Q^n$ ，其约束条件为： $RS = 0$

JK触发器： $Q^{n+1} = J\bar{Q}^n + \bar{K}Q^n$

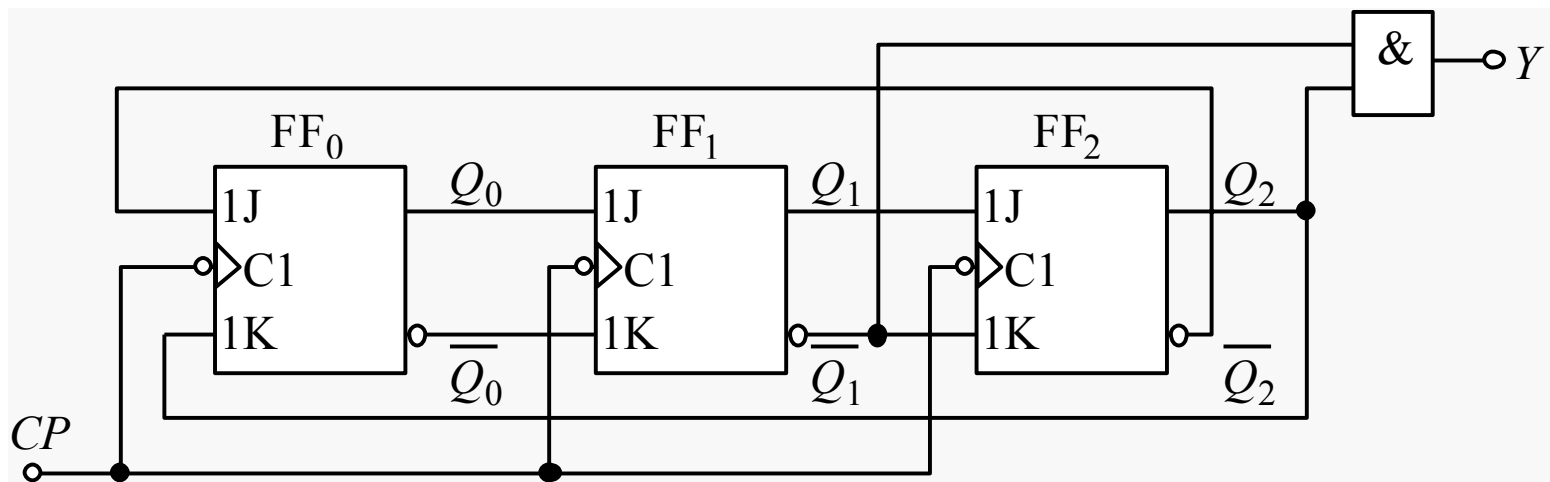
D触发器： $Q^{n+1} = D$

时序逻辑电路的分析方法

■ 时序电路的分析步骤：



例



时钟方程: $CP_2 = CP_1 = CP_0 = CP$

同步时序电路的时钟方程可省去不写。

1

写方程

输出方程: $Y = \overline{Q_1}^n Q_2^n$

驱动方程:
$$\begin{cases} J_2 = Q_1^n & K_2 = \overline{Q_1}^n \\ J_1 = Q_0^n & K_1 = \overline{Q_0}^n \\ J_0 = \overline{Q_2}^n & K_0 = Q_2^n \end{cases}$$

2

求状态方程

$$\begin{cases} J_2 = Q_1^n & K_2 = \overline{Q_1^n} \\ J_1 = Q_0^n & K_1 = \overline{Q_0^n} \\ J_0 = \overline{Q_2^n} & K_0 = Q_2^n \end{cases}$$

JK 触发器的特性方程：

$$Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$$

将各触发器的驱动方程代入，即得电路的状态方程：

$$\begin{cases} Q_2^{n+1} = J_2\overline{Q_2^n} + \overline{K_2}Q_2^n = Q_1^n\overline{Q_2^n} + Q_1^nQ_2^n = Q_1^n \\ Q_1^{n+1} = J_1\overline{Q_1^n} + \overline{K_1}Q_1^n = Q_0^n\overline{Q_1^n} + Q_0^nQ_1^n = Q_0^n \\ Q_0^{n+1} = J_0\overline{Q_0^n} + \overline{K_0}Q_0^n = \overline{Q_2^n}\overline{Q_0^n} + \overline{Q_2^n}Q_0^n = \overline{Q_2^n} \end{cases}$$

$$\begin{cases} Q_2^{n+1} = Q_1^n \\ Q_1^{n+1} = Q_0^n \\ Q_0^{n+1} = \overline{Q_2^n} \\ Y = \overline{Q_1^n} Q_2^n \end{cases}$$

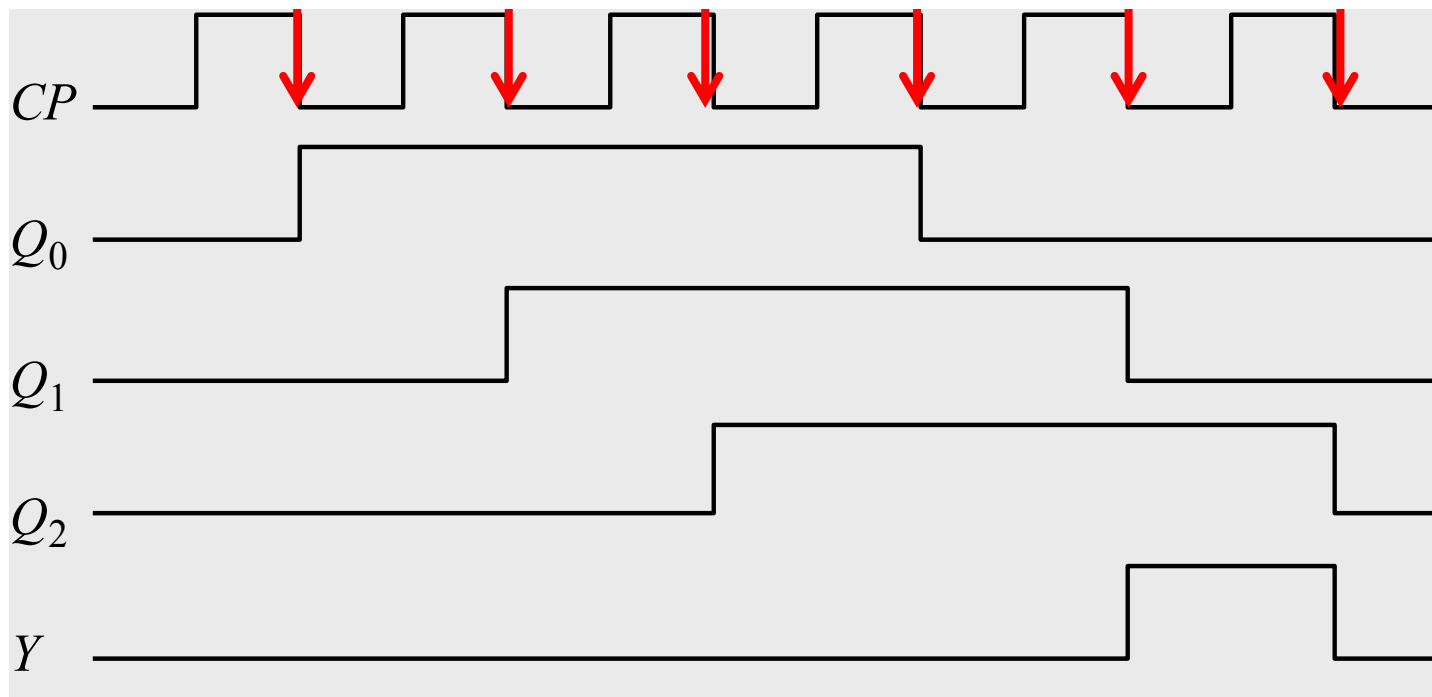
现 态			次 态			输 出
Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y
0	0	0	0	0	1	0
0	0	1	0	1	1	0
0	1	0	1	0	1	0
0	1	1	1	1	1	0
1	0	0	0	0	0	1
1	0	1	0	1	0	1
1	1	0	1	0	0	0
1	1	1	1	1	0	0

注意：这里是按自然顺序给出了 $Q_2^n Q_1^n Q_0^n$ 的全部取值，但在CP脉冲作用下 $Q_2^n Q_1^n Q_0^n$ 的取值顺序和取值，则取决于 $Q_2^n Q_1^n Q_0^n$ 初值和状态方程。

4

画时序图、状态图

时序图



$$\begin{cases} Q_2^{n+1} = Q_1^n \\ Q_1^{n+1} = Q_0^n \\ Q_0^{n+1} = \overline{Q_2}^n \end{cases}$$

0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	0	0	1	0

$$Y = \overline{Q_1}^n Q_2^n$$

状态图

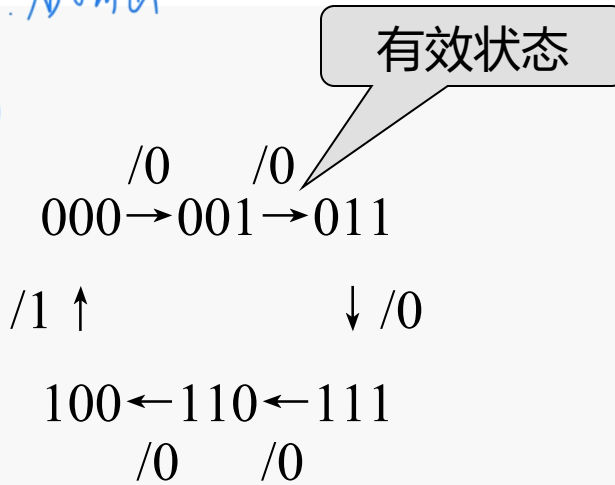
排列顺序: *左输入, 右输出*

$Q_2^n Q_1^n Q_0^n$

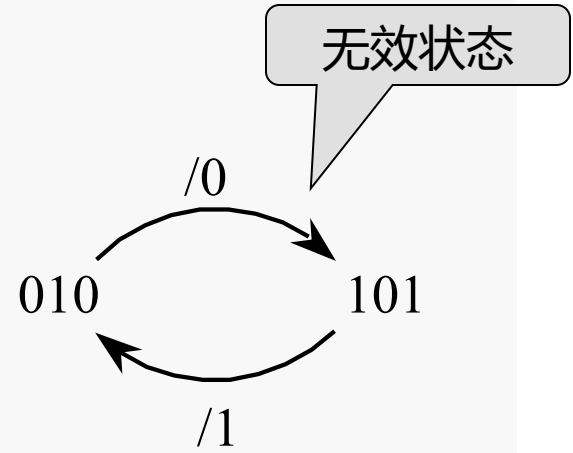
$/Y$

$$\begin{cases} Q_2^{n+1} = Q_1^n \\ Q_1^{n+1} = Q_0^n \\ Q_0^{n+1} = \overline{Q_2^n} \end{cases}$$

$$Y = \overline{Q_1^n} Q_2^n$$



(a) 有效循环



(b) 无效循环

能自启动: 在时序电路中, 虽然存在无效状态, 但无效状态没有形成循环。

不能自启动: 在时序电路中, 有无效状态存在, 且无效状态之间形成了循环。这种电路一旦因干扰落入了无效循环, 正常的工作状态将被迫终止, 直到电路重新启动, 否则就再也不会到有效状态了。

相邻只有一位不同

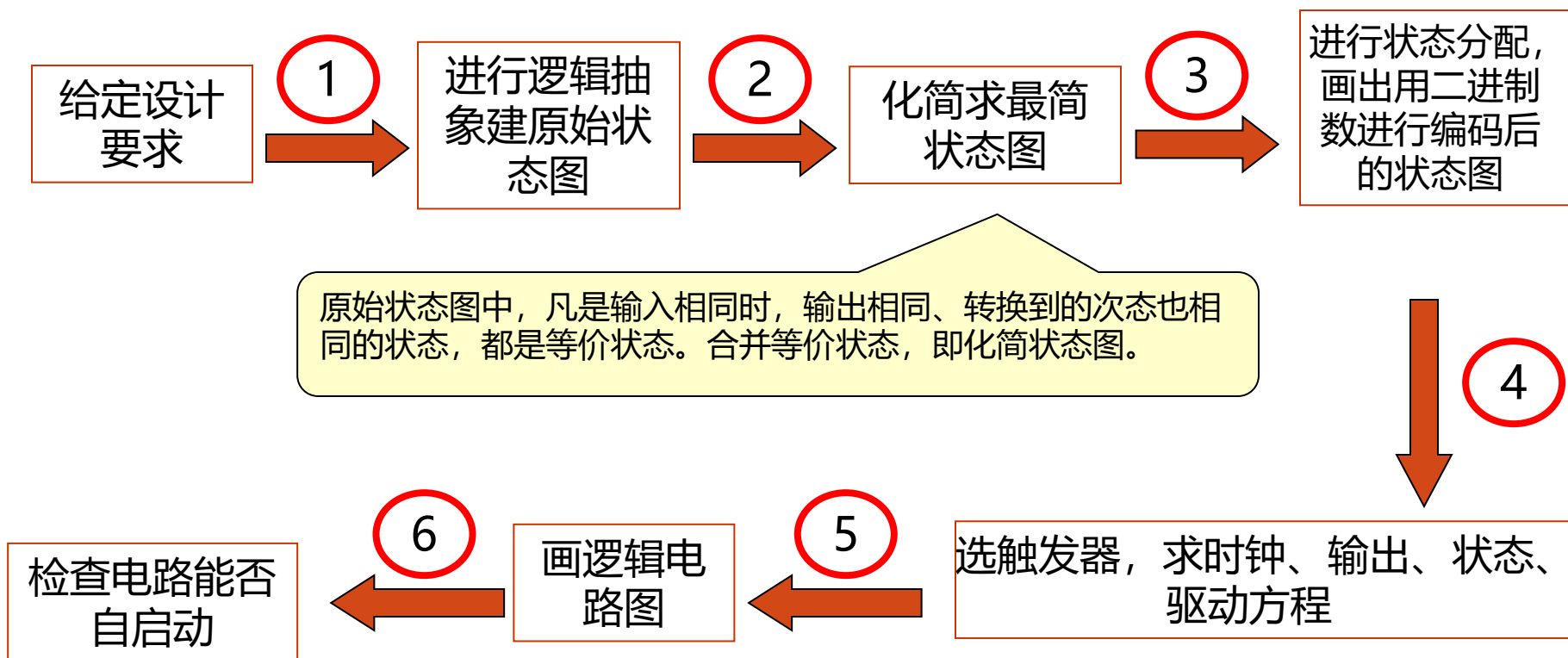
有效循环的6个状态分别是0~5这6个十进制数字的格雷码，并且在时钟脉冲 CP 的作用下，这6个状态是按递增规律变化的，即：

$000 \rightarrow 001 \rightarrow 011 \rightarrow 111 \rightarrow 110 \rightarrow 100 \rightarrow 000 \rightarrow \dots$

所以这是一个用格雷码表示的六进制同步加法计数器。当对第6个脉冲计数时，计数器又重新从000开始计数，并产生输出 $Y=1$ 。

时序逻辑电路的设计方法

■ 时序电路的设计步骤：



例

设计一个串行数据检测电路，当连续输入3个或3个以上1时，电路的输出为1，其它情况下输出为0。例如：

输入 X 101100111011110
输出 Y 000000001000110

1

建立原始状态图

设电路输入为0时状态为 S_0 。开始时处于状态 S_0 。

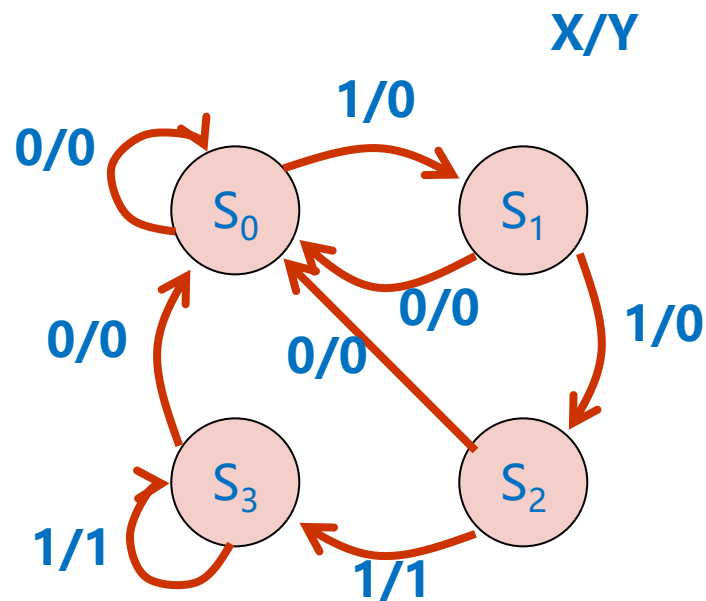
第一次输入1时，由状态 S_0 转入状态 S_1 ，并输出0；

若继续输入1，由状态 S_1 转入状态 S_2 ，并输出0；

如果仍接着输入1，由状态 S_2 转入状态 S_3 ，并输出1；

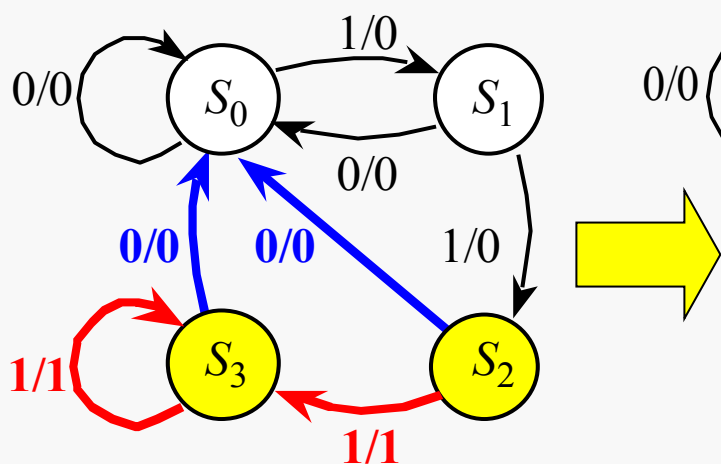
此后若继续输入1，电路仍停留在状态 S_3 ，并输出1。

电路无论处在什么状态，只要输入0，都应回到初始状态，并输出0，以便重新计数。

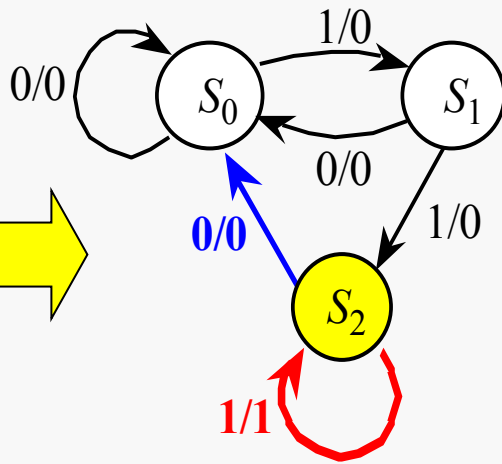


2

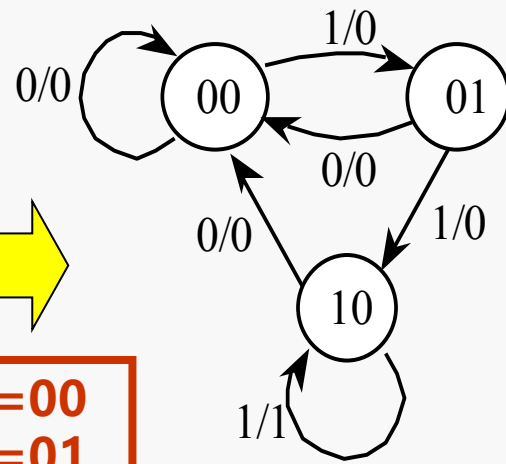
状态化简



(a) 原始状态图



(b) 简化状态图



(c) 二进制状态图

$S_0=00$
 $S_1=01$
 $S_2=10$

原始状态图中，凡是在输入相同时，输出相同、要转换到的次态也相同的状态，称为**等价状态**。状态化简就是将多个等价状态合并成一个状态，把多余的状态都去掉，从而得到最简的状态图。

所得原始状态图中，状态 S_2 和 S_3 等价。因为它们在输入为 1 时输出都为 1，且都转换到次态 S_3 ；在输入为 0 时输出都为 0，且都转换到次态 S_0 。所以它们可以合并为一个状态，合并后的状态用 S_2 表示。

选触发器，求时钟、输出、状态、驱动方程

选用2个 CP 下降沿触发的 JK 触发器，分别用 FF_0 、 FF_1 表示。采用同步方案，即取： $CP_0=CP_1=CP$ 。

求输出方程

$X \backslash Q_1^n Q_0^n$		00	01	11	10
		0	0	×	0
	1	0	0	×	1

Y 的卡诺图

$$Y = XQ_1^n$$

$Q_1^n Q_0^n$		X			
		00	01	11	10
0	0	0	0	×	0
	1	1	0	×	0

(a) Q_0^{n+1} 的卡诺图

$$Q_0^{n+1} = X\overline{Q_1^n}\overline{Q_0^n}$$

求状态方程

$X \backslash Q_1^n Q_0^n$		00	01	11	10
		00	00	$\times$	00
0	1	01	10	$\times$	10

(b) $Q_1^{n+1} Q_0^{n+1}$ 的卡诺图

$X \backslash Q_1^n Q_0^n$		00	01	11	10
0		0	0	×	0
1		0	1	×	1

(b) Q_1^{n+1} 的卡诺图

$$Q_1^{n+1} = XQ_0^n\overline{Q_1^n} + XQ_1^n$$

$$\begin{cases} Q_0^{n+1} = X\bar{Q}_1^n\bar{Q}_0^n + 0 \cdot Q_0^n \\ Q_1^{n+1} = XQ_0^n\bar{Q}_1^n + XQ_1^n \end{cases}$$

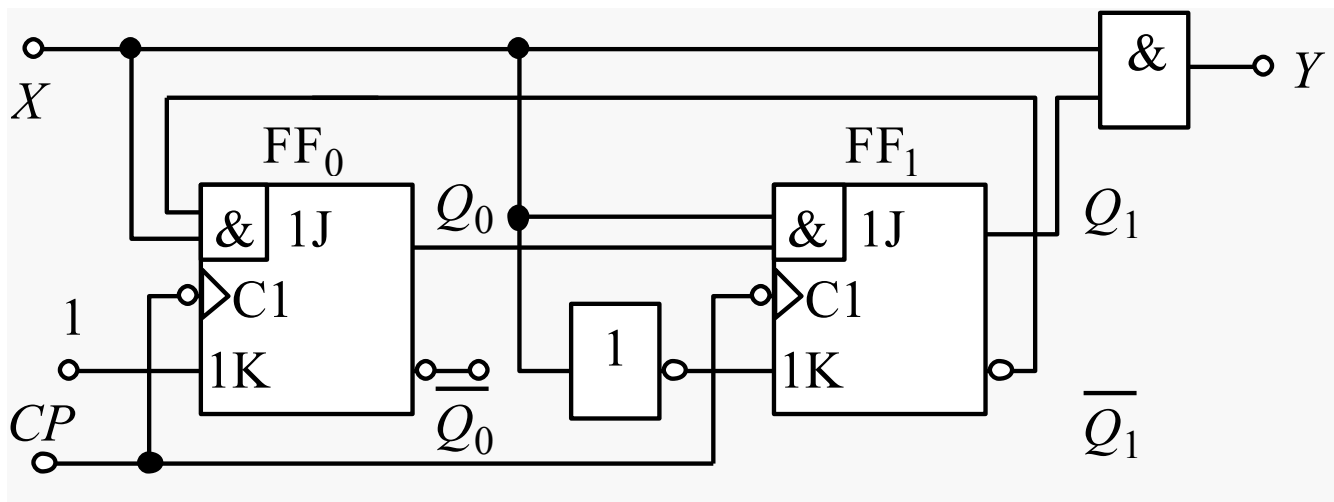
比较，得驱动方程：

$$\begin{cases} J_0 = X\bar{Q}_1^n & K_0 = 1 \\ J_1 = XQ_0^n & K_1 = \bar{X} \end{cases}$$

$$Q^{n+1} = J\bar{Q}^n + \bar{K}Q^n$$

5

电路图



6

检查电路能否自启动

将无效状态11代入输出方程 $Y = XQ_1^n$ 和状态方程计算：

$$00 \xleftarrow{0/0} 11 \xrightarrow{1/1} 10$$

电路能够自启动。

计数器

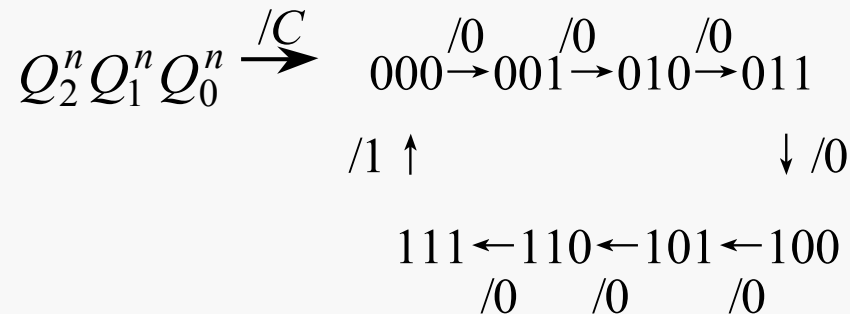
能够记忆输入脉冲个数的电路称为计数器。

特点：输入信号主要是计数脉冲CP，输出通常是状态函数

3位二进制数同步加法计数器

1、状态图

排列顺序：



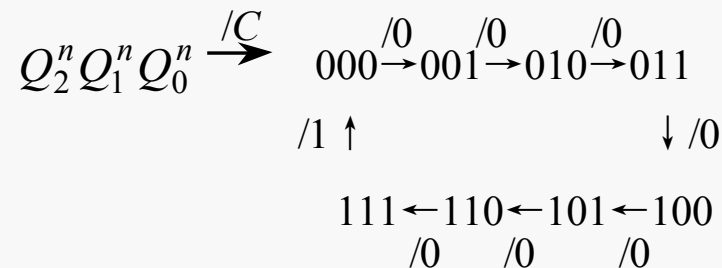
2、选择触发器，求时钟方程、输出方程和状态方程

(1) 选用3个CP下降沿触发的JK触发器，分别用FF₀、FF₁、FF₂表示。

(2) 时钟方程： $CP_0 = CP_1 = CP_2 = CP$

(3) 输出方程： $C = Q_2^n Q_1^n Q_0^n$

排列顺序：



(4) 状态方程

$$\left\{ \begin{array}{l}
 Q_0^{n+1} = \overline{Q_0^n} \\
 Q_1^{n+1} = \overline{Q_1^n} Q_0^n + Q_1^n \overline{Q_0^n} \\
 Q_2^{n+1} = \overline{Q_2^n} Q_1^n Q_0^n + Q_2^n \overline{Q_1^n} + Q_2^n \overline{Q_0^n}
 \end{array} \right.$$

$$Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$$

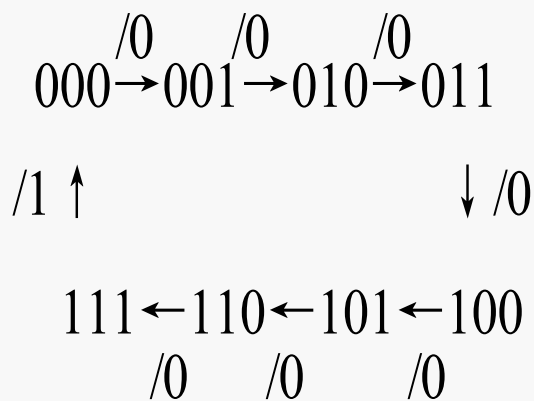
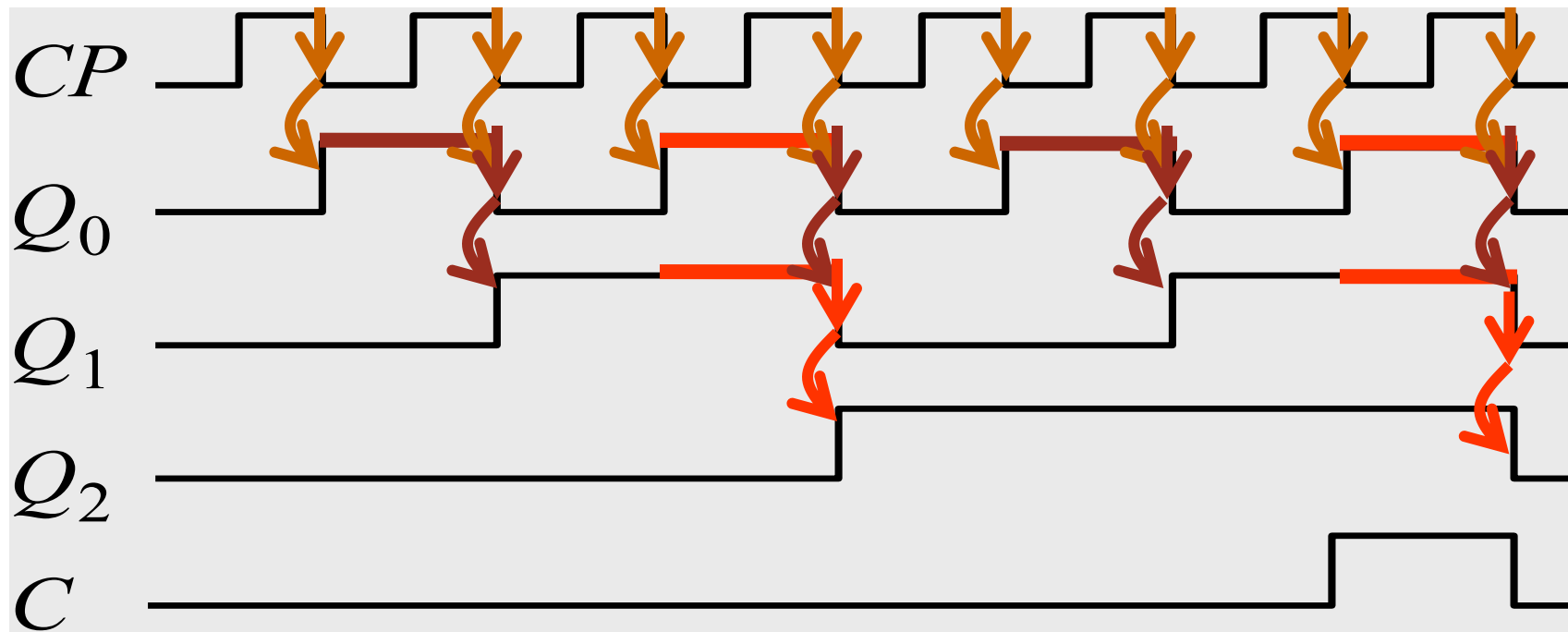
驱动方程

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = Q_0^n$$

$$J_2 = K_2 = Q_1^n Q_0^n$$

时序图



FF₀每输入一个时钟脉冲翻转一次

FF₁在Q₀=1时, 在下一个CP触发沿到来时翻转。

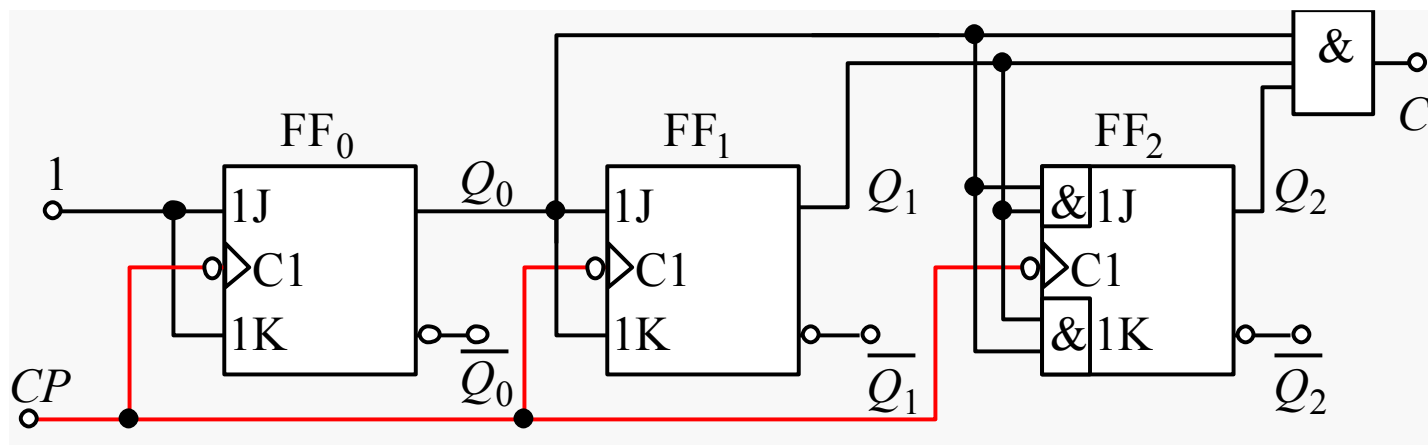
FF₂在Q₀=Q₁=1时, 在下一个CP触发沿到来时翻转。

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = Q_0^n$$

$$J_2 = K_2 = Q_1^n Q_0^n$$

4、画逻辑电路图



由于没有无效状态，电路能自启动。

5、推广到 n 位二进制同步加法计数器

驱动方程

$$\begin{cases} J_0 = K_0 = 1 \\ J_1 = K_1 = Q_0^n \\ J_2 = K_2 = Q_1^n Q_0^n \\ \dots\dots\dots \\ J_{n-1} = K_{n-1} = Q_{n-2}^n Q_{n-3}^n \cdots Q_1^n Q_0^n \end{cases}$$

输出方程

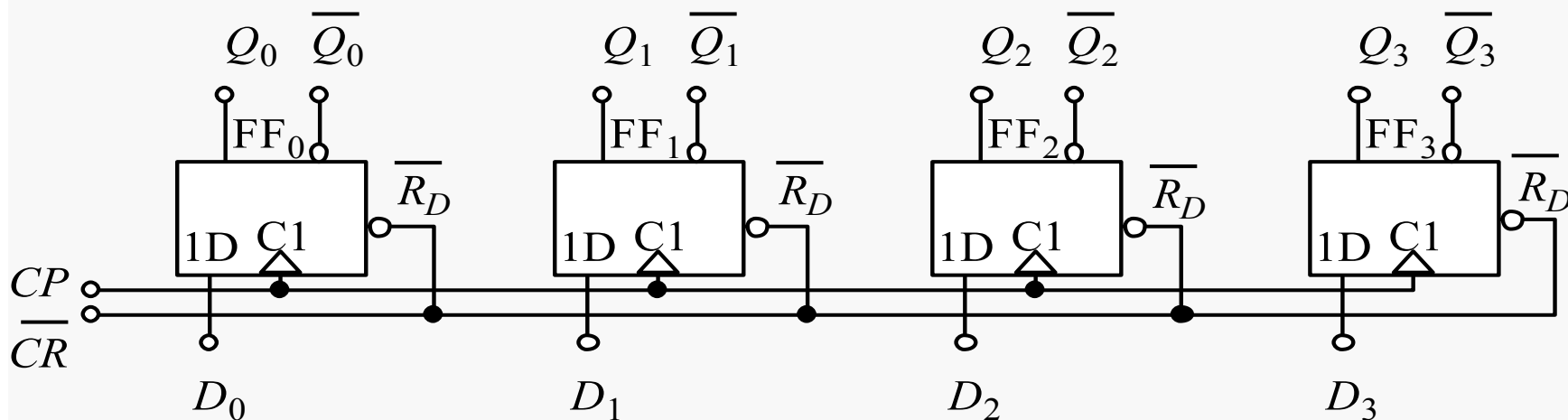
$$C = Q_{n-1}^n Q_{n-2}^n \cdots Q_1^n Q_0^n$$

寄存器和读写存储器

- 寄存器的特点与分类
- 基本寄存器
- 移位寄存器
- 读写存储器

基本寄存器

4边沿D触发器



(1) 清零。 $\overline{CR}=0$ ，异步清零。即有：

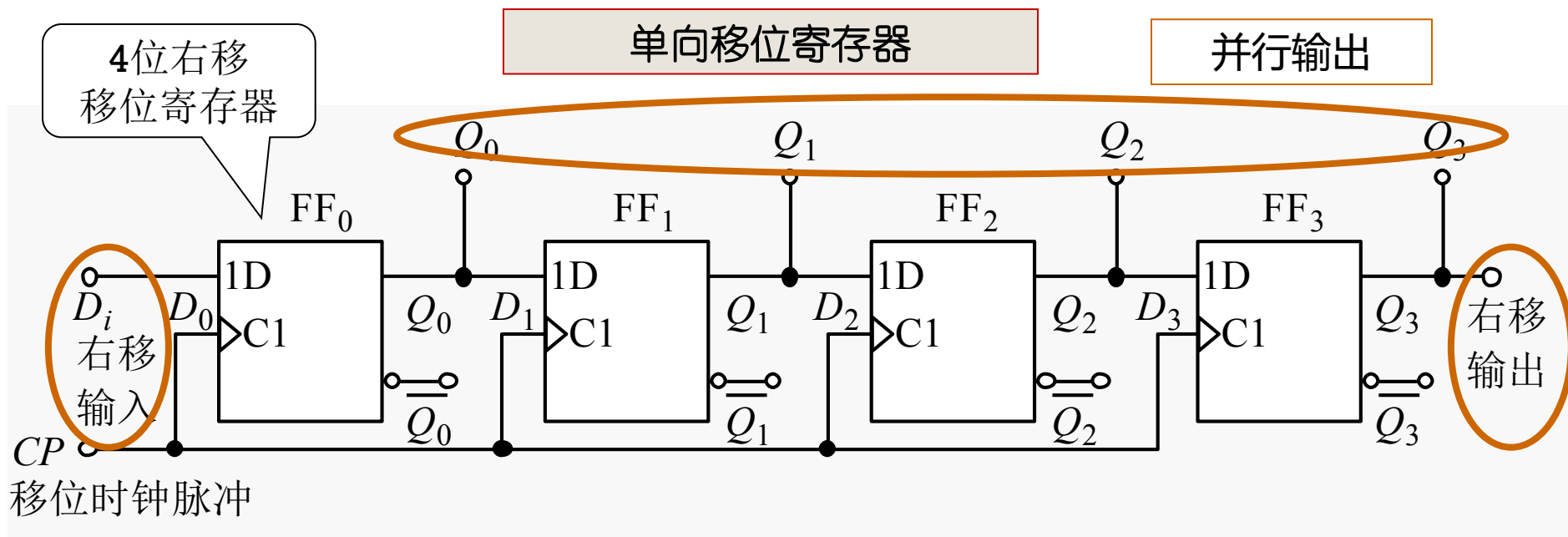
$$Q_3^n Q_2^n Q_1^n Q_0^n = 0000$$

(2) 送数。 $\overline{CR}=1$ 时， CP 上升沿送数。即有：

$$Q_3^{n+1} Q_2^{n+1} Q_1^{n+1} Q_0^{n+1} = D_3 D_2 D_1 D_0$$

(3) 保持。在 $\overline{CR}=1$ 、 CP 上升沿以外时间，寄存器内容将保持不变。

移位寄存器



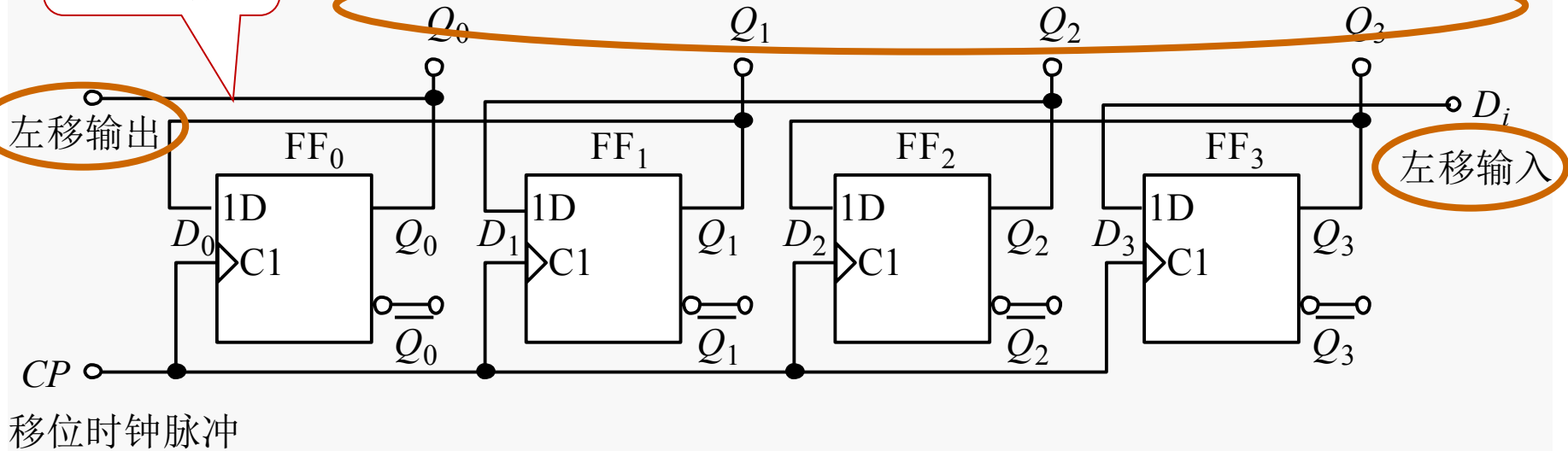
时钟方程: $CP_0 = CP_1 = CP_2 = CP_3 = CP$

驱动方程: $D_0 = D_i, D_1 = Q_0^n, D_2 = Q_1^n, D_3 = Q_2^n$

状态方程: $Q_0^{n+1} = D_i, Q_1^{n+1} = Q_0^n, Q_2^{n+1} = Q_1^n, Q_3^{n+1} = Q_2^n$

并行输出

4位左移
移位寄存器



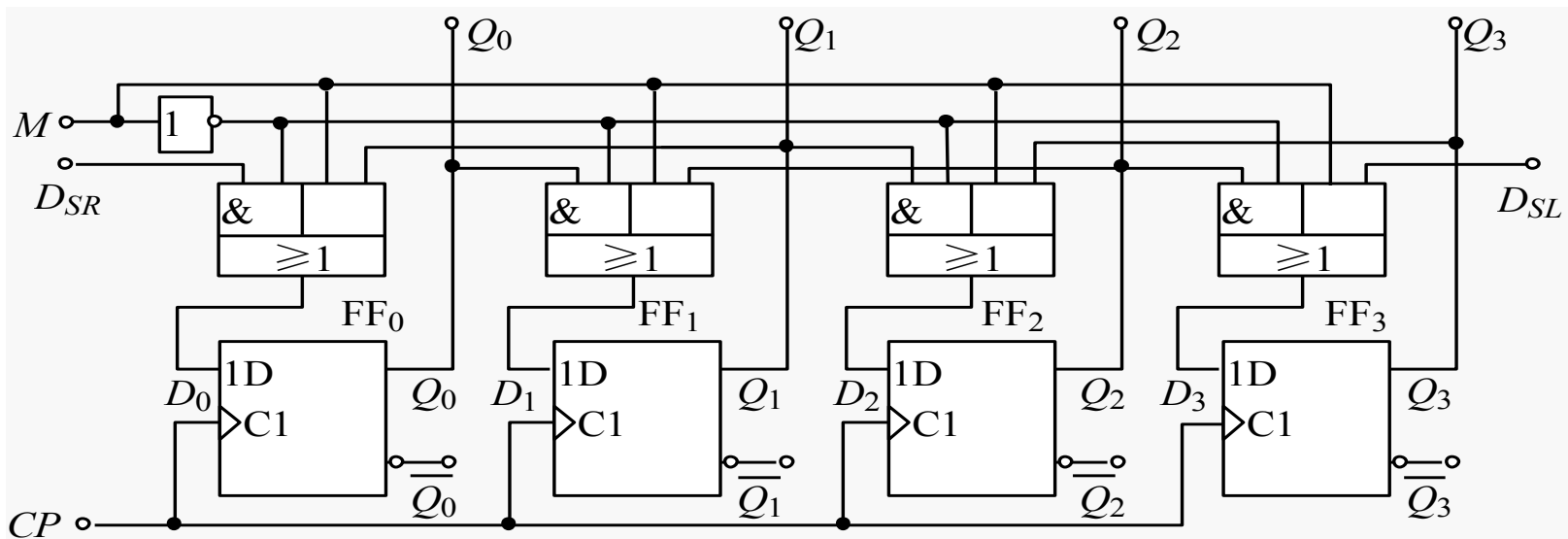
时钟方程: $CP_0 = CP_1 = CP_2 = CP_3 = CP$

驱动方程: $D_0 = Q_1^n, D_1 = Q_2^n, D_2 = Q_3^n, D_3 = D_i$

状态方程: $Q_0^{n+1} = Q_1^n, Q_1^{n+1} = Q_2^n, Q_2^{n+1} = Q_3^n, Q_3^{n+1} = D_i$

双向移位寄存器

把左移和右移移位寄存器组合起来，加上移位方向控制信号，便可构成双向移位寄存器。



$$\begin{cases} Q_0^{n+1} = \overline{M}D_{SR} + MQ_1^n \\ Q_1^{n+1} = \overline{M}Q_0^n + MQ_2^n \\ Q_2^{n+1} = \overline{M}Q_1^n + MQ_3^n \\ Q_3^{n+1} = \overline{M}Q_2^n + MD_{SL} \end{cases}$$

M=0时右移

$$\begin{cases} Q_0^{n+1} = D_{SR} \\ Q_1^{n+1} = Q_0^n \\ Q_2^{n+1} = Q_1^n \\ Q_3^{n+1} = Q_2^n \end{cases}$$

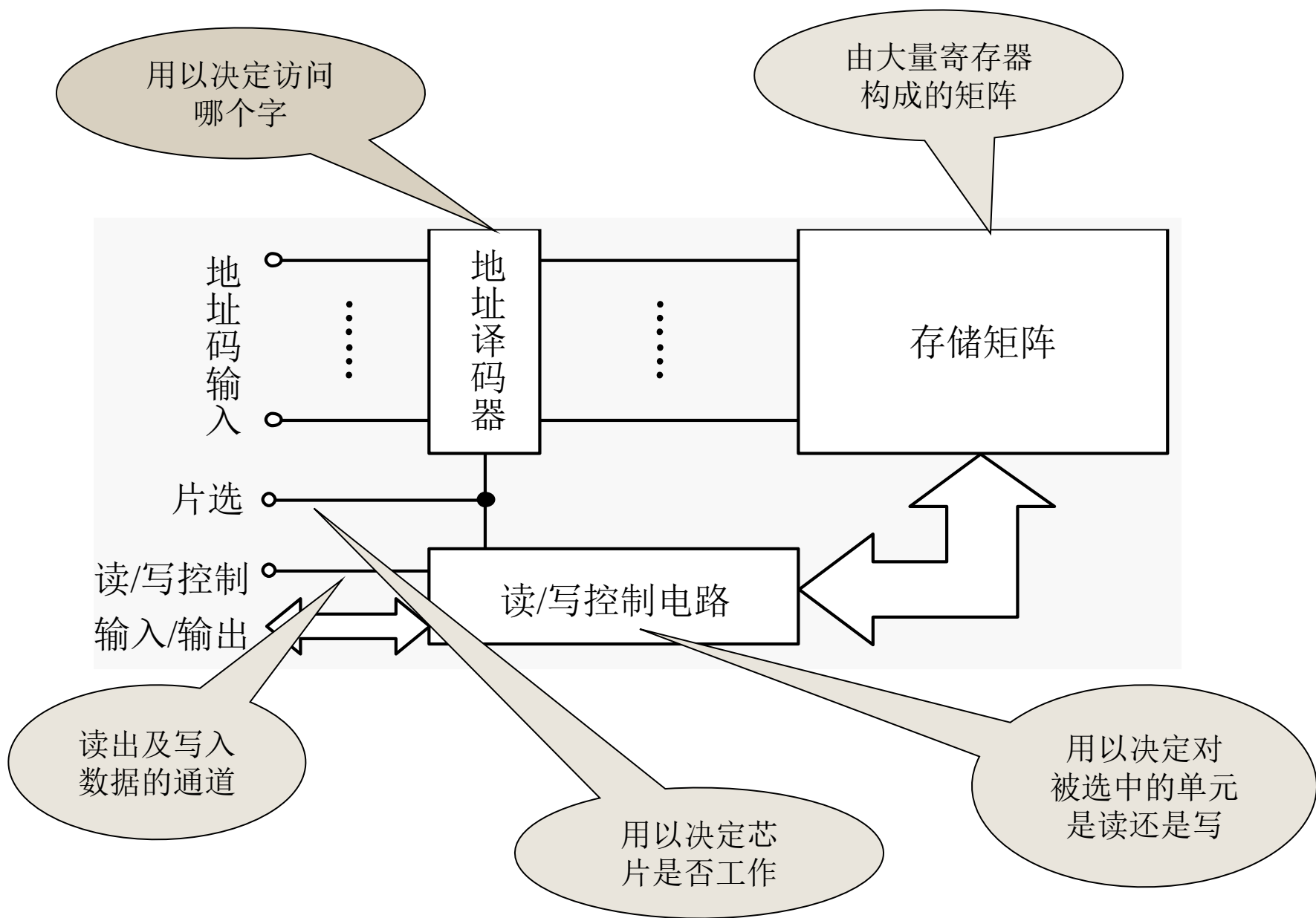
M=1时左移

$$\begin{cases} Q_0^{n+1} = Q_1^n \\ Q_1^{n+1} = Q_2^n \\ Q_2^{n+1} = Q_3^n \\ Q_3^{n+1} = D_{SL} \end{cases}$$

读/写存储器

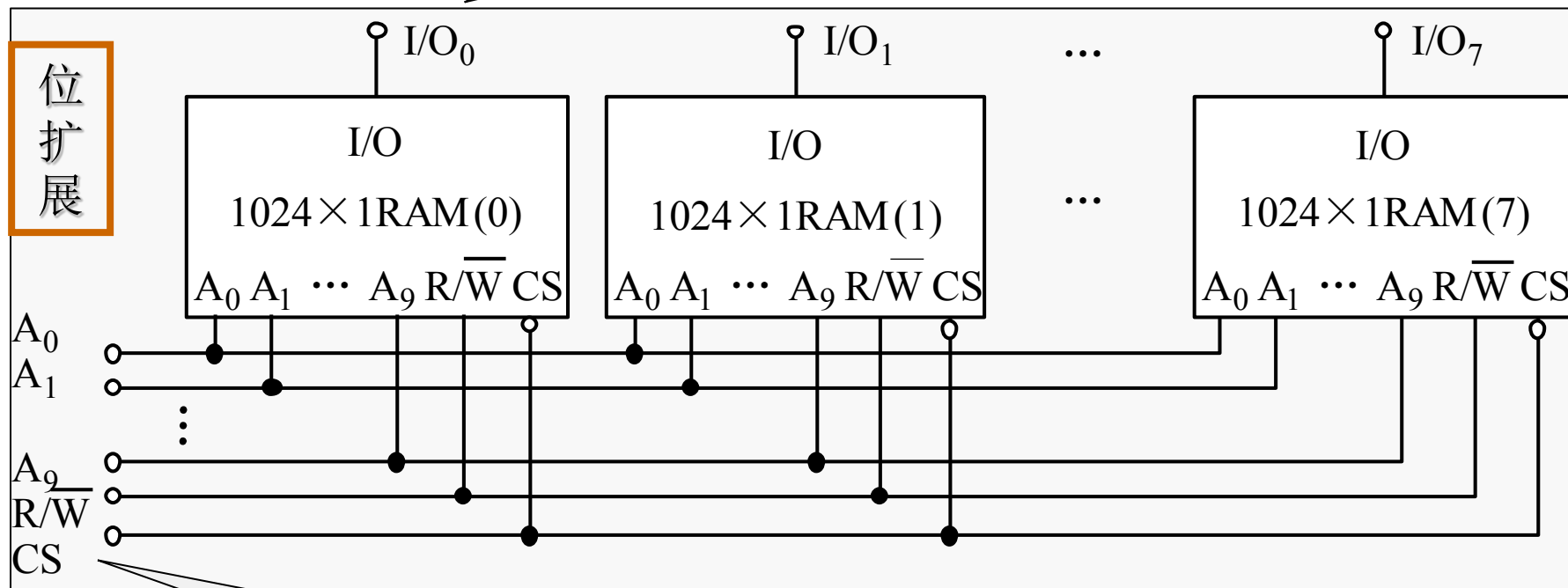
1.RAM的结构

- RAM是由许许多多的基本寄存器组合起来构成的大规模集成电路。
- RAM中的每个寄存器称为一个字，寄存器中的每一位称为一个存储单元。
- RAM的容量：寄存器的个数（字数）与寄存器中存储单元个数（位数）的乘积。



2. RAM容量的扩展

输入 / 输出 (I/O) 分开
使用作为字的各个位线

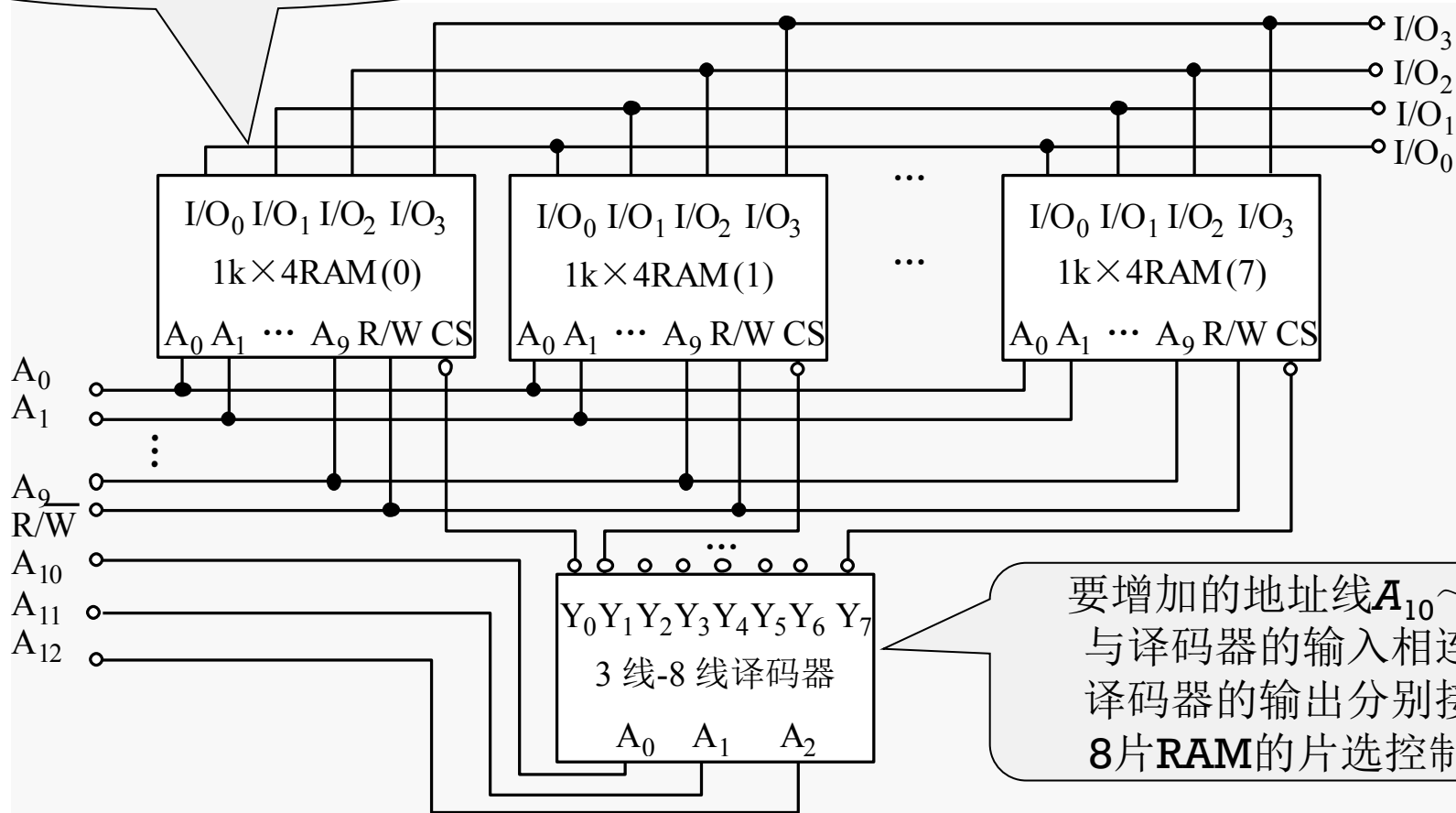


将地址线、读 / 写线和
片选线对应地并联在一起

字扩展：字数的扩展可以利用外加译码器控制芯片的片选输入端来实现。

例：8片1K*4位RAM扩展为8K*4位RAM

输入 / 输出 (I/O) 线并联



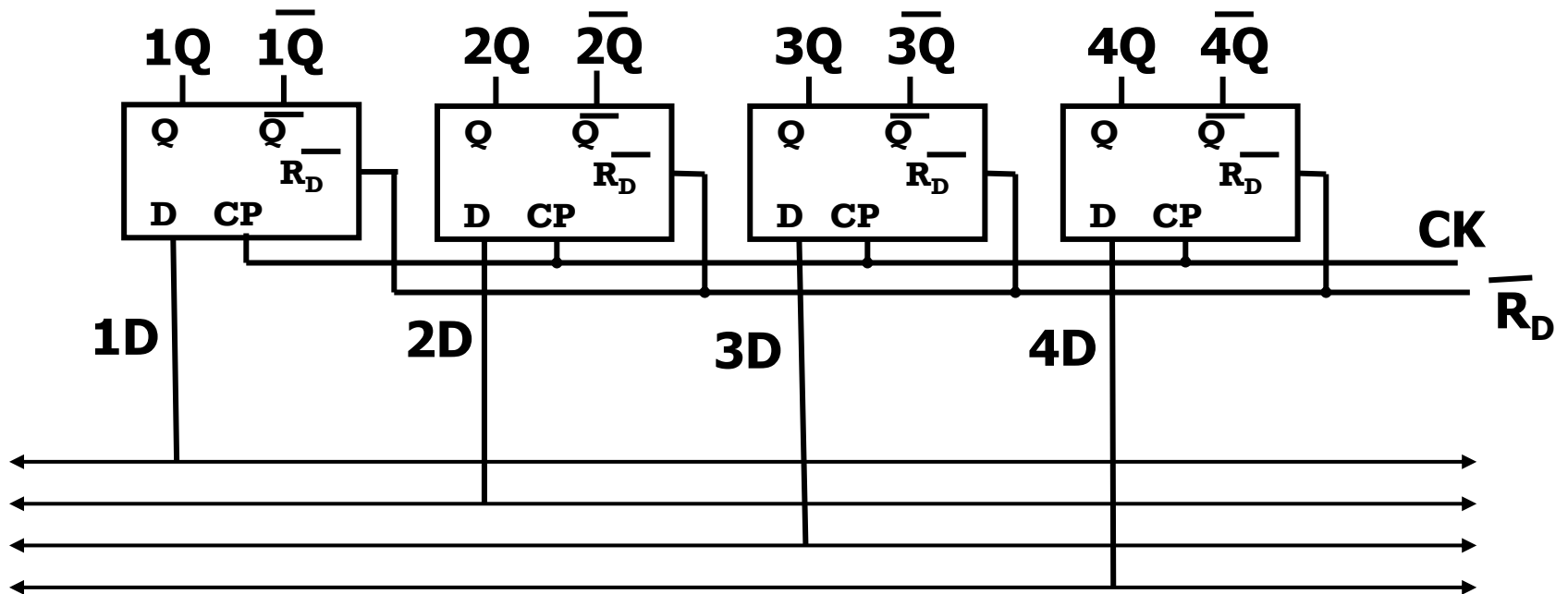
要增加的地址线 $A_{10} \sim A_{12}$ 与译码器的输入相连，译码器的输出分别接至 8片RAM的片选控制端

数据通路的时序

- **数据通路** (Data Path) : 数据在功能部件之间传送的路径称为数据通路, 路径上的部件称为数据通路部件, 如ALU、寄存器等。每个部件既有输入信号线, 也有输出信号线。
- **数据总线**: 在CPU的各个组成部分之间以及与外部设备 (如内存、I/O接口等) 之间传输数据, 数据总线的信号传输是双向的, 既可以作为输入信号, 也可以作为输出信号。

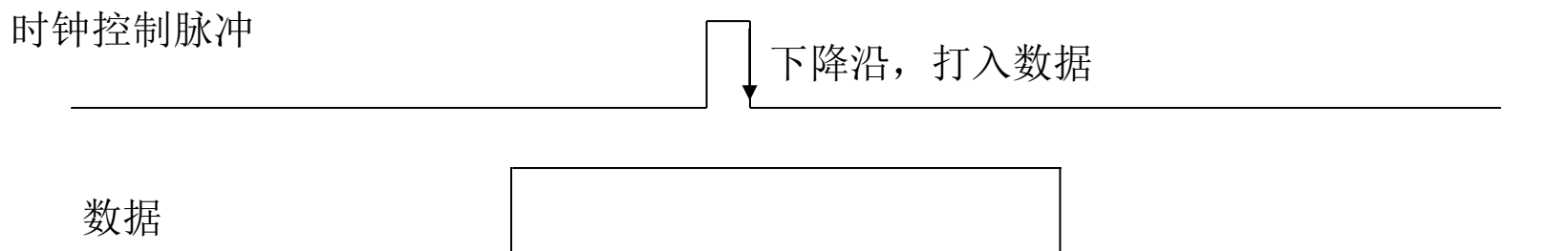
数据通路上的寄存器

连接总线的逻辑部件的详细情况，以寄存器为例：

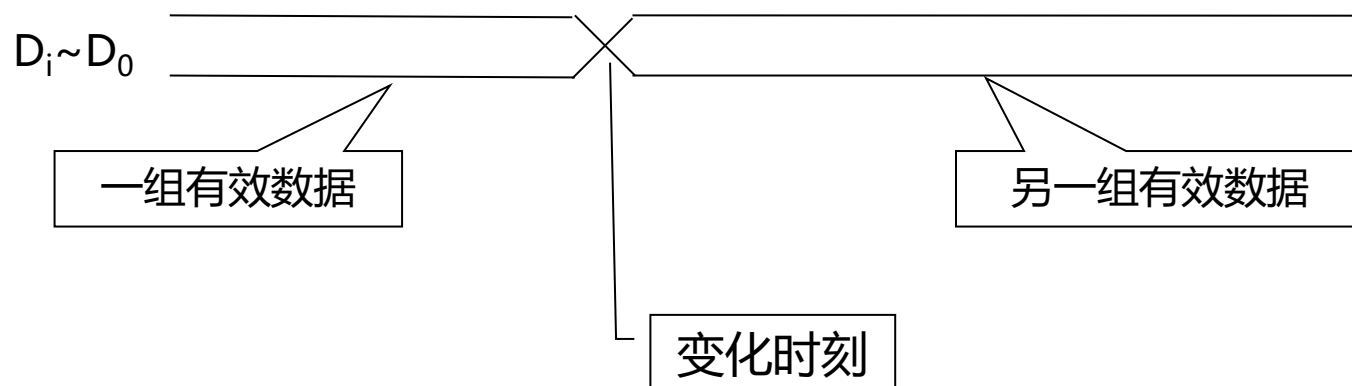


实现ALU运算的数据通路

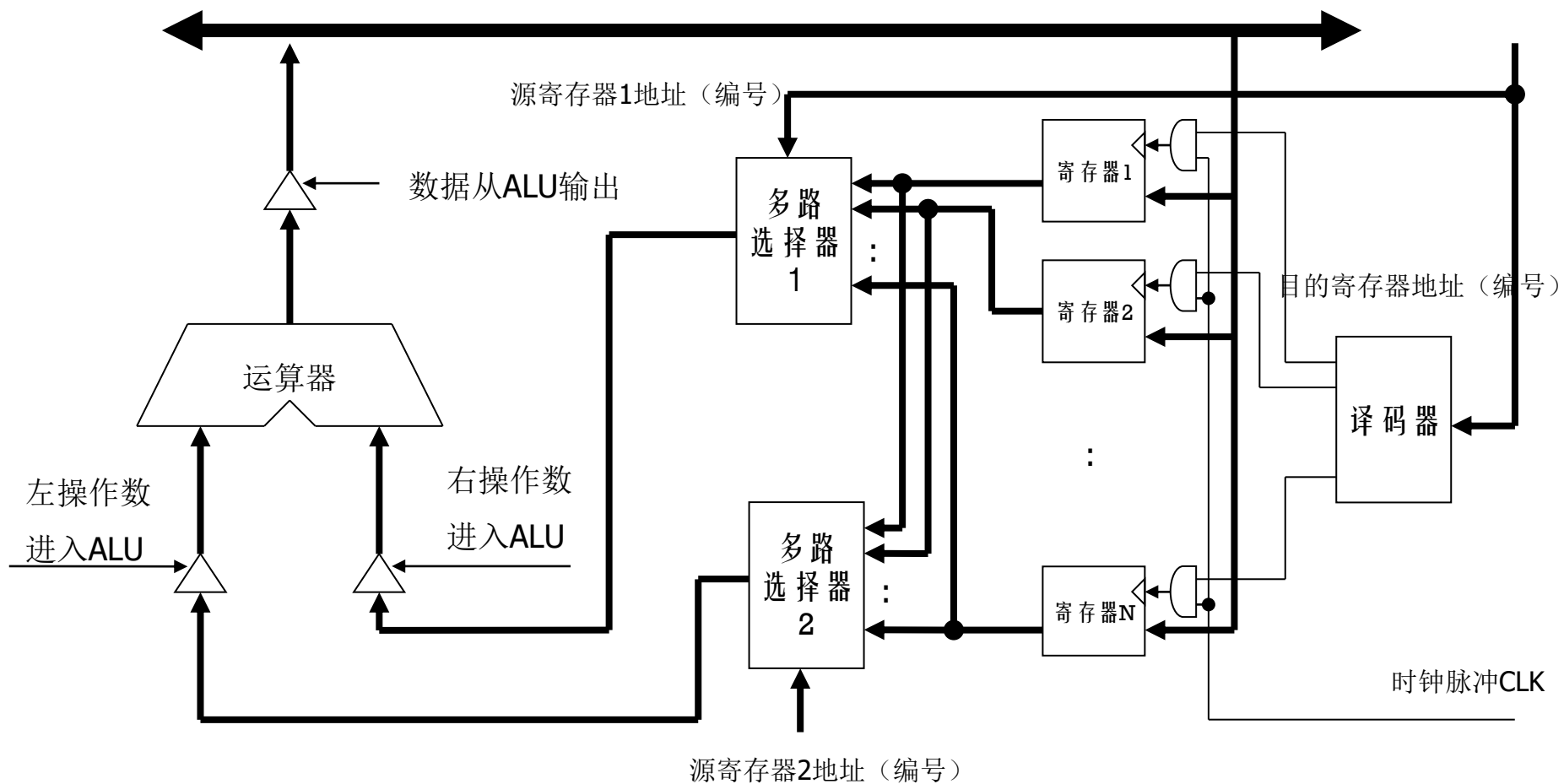
- 时序的表示：用时序图表示各个信号之间时间的约束关系。
- 数据信号的宽度应该覆盖控制信号，控制信号起作用的时间越短越好。



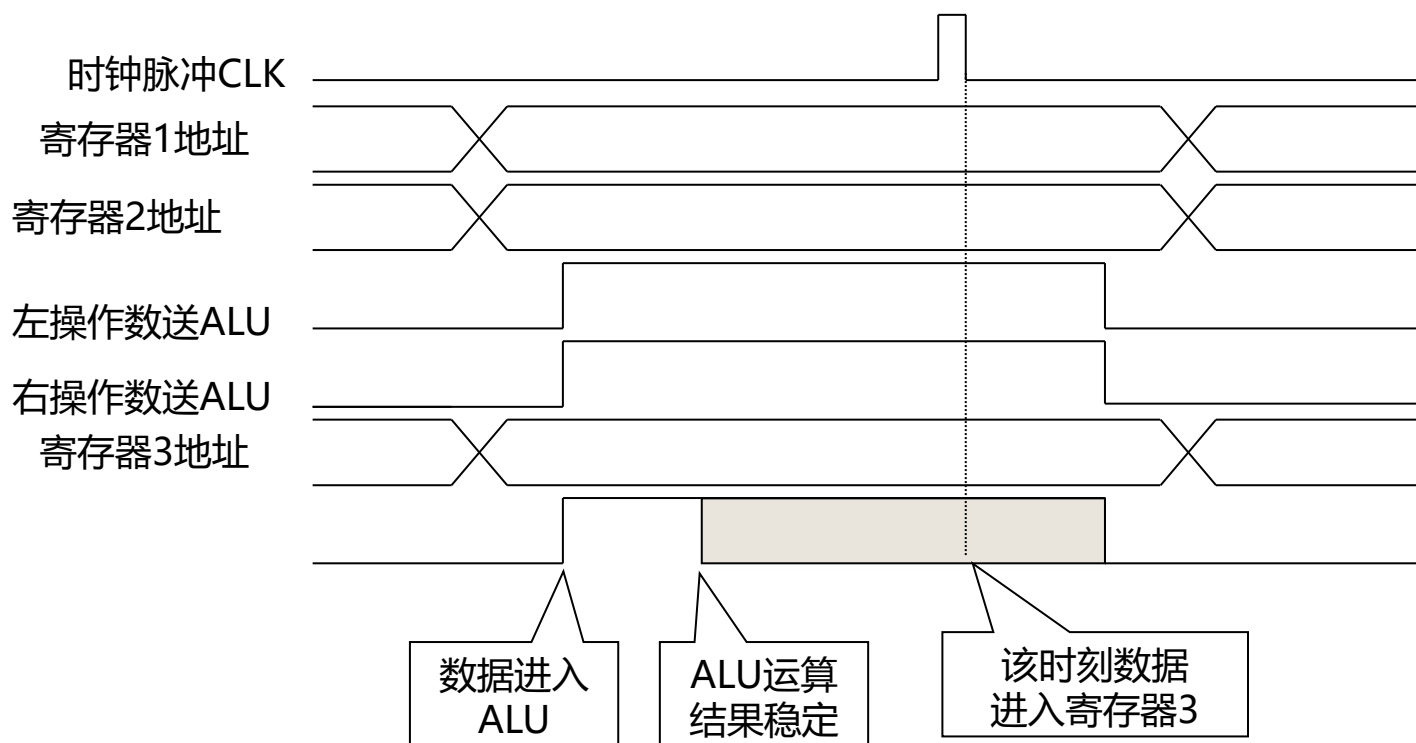
- 数据线时序的表示：



一个实现运算过程的简单数据通路



- 从寄存器1和寄存器2分别取数据送ALU的左右操作数
 - 需要的信号：寄存器1的地址、寄存器2的地址、左操作数进入ALU、右操作数进入ALU，这些信号的长度要足够长。
- 计算结果存入寄存器3
 - 需要的信号：寄存器3的地址，时钟脉冲

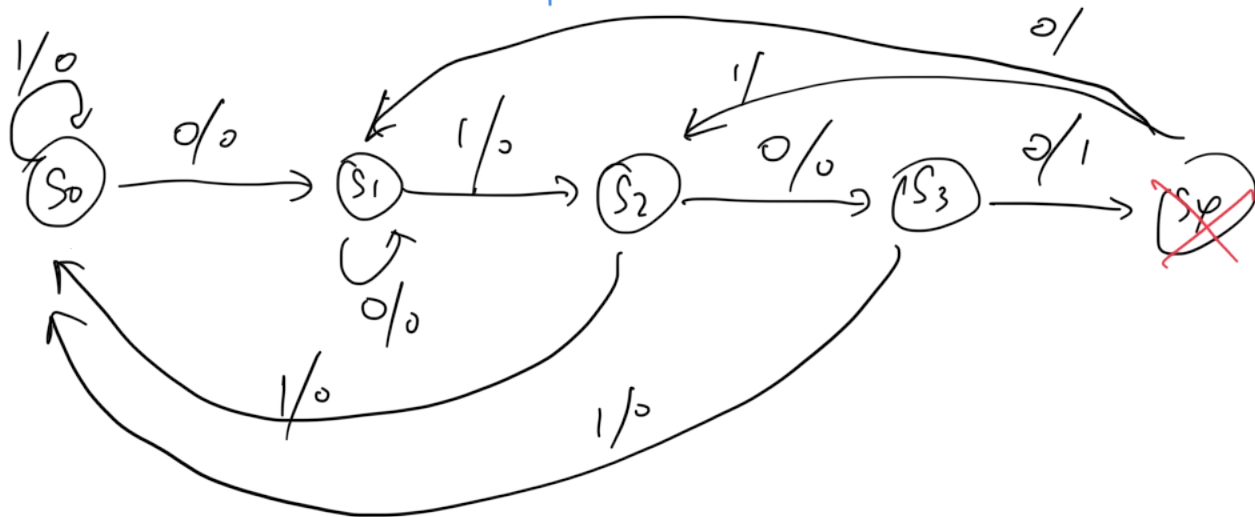


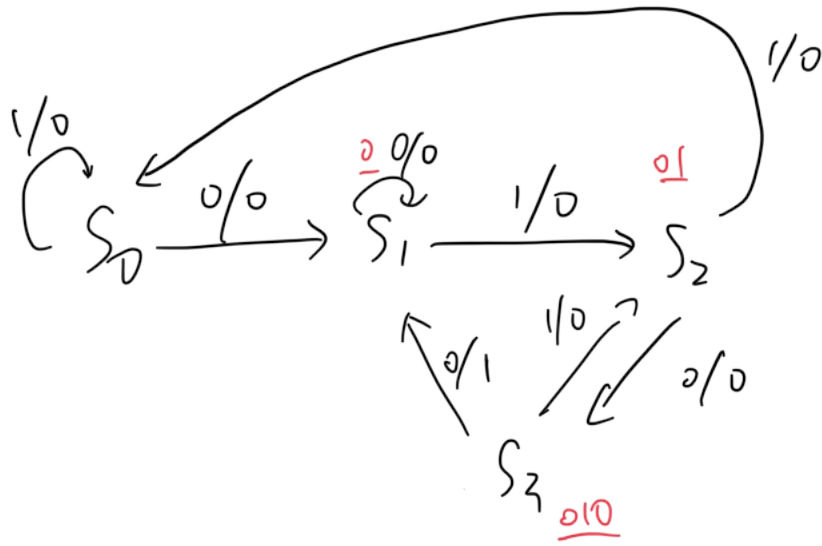
JK 触发器. 设计电路检测 0100 信号

$X: 000100100^2$

$Y: 0000010010$

① 画状态图





4个状态, $2^2 = 4$ 个状态